# HackPit

Full Capability Assessment & Agreed Plan

**Date:** 26 July 2026

**Scope:** backend · frontend · pipeline · docker · docs · knowledge base

**Measured against:** OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB/CTF · bug bounty · client pentests

**HackPit — Capability Assessment & Build Log**

# HackPit — Capability Assessment & Build Log

**Date:** 2026-07-26 **Scope:** every backend module, the frontend, the pipeline, the docker stack, the docs, and the live KB **Targets this is measured against:** OSCP · PNPT · eCPPT · HTB CPTS · CRTP · HTB boxes/CTF · bug bounty · real-world client pentests.

**Latest (build #12, 2026-08-03).** A frontend-to-backend coverage sweep found **ten endpoints with no caller at all** — D19's "built and then invisible" one layer deeper, including both depth halves of build #8 — and all ten are now reachable. The project also has its **first CI**: the 56-file hermetic suite, the frontend build and an eslint-baseline check run on every push, with the corpus those tests need supplied by a derived, equivalence-proven fixture rather than skipped (**D26**). See Build #12 at the end of Part III.

**Status note (updated).** This began as an assessment (Part I) and an agreed plan (Part II). **All four phases of that plan have since been built**, and a fifth followed — the execution substrate (Phase 1), the state model (Phase 2), the drive-speed gaps (Phase 3), the content-and-goal-specific work (Phase 4: KB Route-B ingest, the evasion/OPSEC channel, the HTTP repeater, pivot/tunnel routing, exam-mode report templates), and **Phase 5**, which closed the two things Part I had left standing: a real PTY (added as a *second* surface, so the auditable one survives) and the recommended KB enrichment (PortSwigger Academy, HackTricks Cloud, the HTB slice, and a CVE→exploit index). Part I has been trimmed to what genuinely remains; the completed blockers, decisions and phases live in **Part III — Build Log**.

---

## 0. Verdict

The **architecture is genuinely strong** — the safety engineering and the LLM-grounding discipline are better than most commercial tooling.

The original problem this assessment found was *reach*: nothing in HackPit was fake or stubbed, but the container it executed into contained seven programs, so it could not finish a real HTB box, touch a real AD domain, run a bug-bounty recon sweep, or complete an OSCP-style engagement. **That execution substrate has now been rebuilt (Phase 1), the loop reasons over structured state instead of stdout tails (Phase 2), the drive-speed gaps are closed (Phase 3), the content/goal work is done (Phase 4), and the interaction and knowledge gaps Part I left standing are closed (Phase 5).** See Part III.

Since then **Phase 5** closed the PTY gap (a real terminal as a *second* surface, keeping the auditable one) and finished the KB enrichment the assessment recommended, and the **Windows execution backend** (a WinRM driver) has since made the CRTP toolset, the OSCP AD set and real internal pentests *executable* — and made the existing AD attack-path graph run **live** instead of on synthetic data. What remains is a short list of *deliberately deferred* items — a VPS for blind callbacks, and the last few thin KB categories — each deferred for a stated reason, not left undone. The Windows/CRTP execution target is **no longer among them**: it is closed by the WinRM driver against an external VM you run (see the Windows execution backend in Part III and D9). Risk-tiered approval is also gone — **decided against** (see D5). See §1 and Part II's "Explicitly deferred".

---

# 1. Remaining gaps

The blocker sections for everything the five phases fixed have been removed (see Part III). What is left is genuinely small.

### 1.1 The PTY gap — now closed, without losing the transcript

This section previously read "no true PTY — deliberate partial", and framed it as a trade: readable logged transcripts *or* full-screen tooling. **Phase 5 rejected that trade.** The premise was wrong — the two are not alternatives if they are separate surfaces.

`:kali` is unchanged: still a persistent shell delimiting each command with a sentinel over a plain pipe, still producing the clean, escape-free, per-command transcripts reports are built from. Alongside it, `:terminal` allocates a **real PTY** in the same open sandbox, so `vim`, `top`, an interactive `msfconsole`, a raw `evil-winrm` shell and `python -c 'pty.spawn'` upgrades all render. Both are audited; both carry identical containment. See Part III, Phase 5.

### 1.2 The human-approval model — settled, not open

**Current state:** every command needs individual `approved=true`; anything the danger heuristic flags additionally needs `dangerous_ack`; `:kali` and `:terminal` are human-driven (typing *is* the approval). Phase 3 added **one-keystroke approve** ( `Enter` = approve · `S` = skip · `Esc` = stop; a dangerous command never fires on `Enter` alone).

This section previously proposed **risk-tiered approval** — auto-running "passive" commands, batch-approving a plan of read-only ones — as open work. **That is now decided against** (D5). Per-command approval is the project's single load-bearing safety property in engagement mode: Wall A is down, the sandbox reaches the internet, the host and the LAN, and nothing else bounds *where* a command can go. A tier that auto-runs anything would remove the only gate on the one mode that needs it most, and it would do so by trusting a classifier — a new component, on the wrong side of the safety boundary, to decide when the human can be skipped. The convenience it buys is a keystroke that has already been optimised away.

Per-command approval is therefore **standing policy, not a deferred item**, and does not appear in the deferred list. The prerequisite work that would have enabled tiering ( `_looks_like_host()`, Phase 3 step 12) was worth doing on its own merits — the scope check is trustworthy now regardless. **Unchanged and non-negotiable: never remove the gate for anything the danger heuristic flags.**

---

# 2. Feature-by-feature audit

### Companion (KB / search / attack paths)

| Component | State | Notes |
|---|---|---|
| KB load + serve | **Solid** | 1,601 entries in memory, exclusions dropped at the door and re-filtered defensively at query time. |

| Component | State | Notes |
|---|---|---|
| Hybrid search ( `pipeline/search.py` ) | **Solid** | BM25 (stdlib) + `nomic-embed-text` cosine, weighted RRF (lex 1.0 / vec 0.5 — correct call for a KB full of exact identifiers), tier boost, whole-query title bonus. Degrades to lexical when Ollama is down instead of 500-ing. |
| Attack-path composition | **Solid, genuinely novel** | Writeup-first mode + KB-first mode. Retrieval seeded per-phase so no phase starts empty. Step eligibility filtering ( `is_step_eligible` ) is unusually careful — excludes writeups, defensive-hardening guides, tool-install meta-docs, personal logs, grab-bags. |
| Grounding / validation | **Best-in-class** | Cited `entry_id` s must resolve or the step downgrades to `ai_suggested` ; commands come from the KB entry, never the model; `target_adaptation` is dropped entirely if it names an FQDN not in the supplied facts. |
| `substitute_target` | **Excellent** | CIDR sentinel to avoid double-rewrite, host-position gating so `.env.example` and `<script>` survive, refuses to rewrite what it can't rewrite confidently. |
| `foreign_refs` | **Excellent** | Honesty marker — reports a foreign host/AD domain rather than guessing a replacement. Right call. |
| Channel-2 context grounding | **Good** | Writeup/methodology content as reasoning background, strictly separated from the step pool, with a leak guard over model output. |
| Engagement sessions | **Works** | SQLite, per-step checked + pasted results, debounced autosave. |
| Report generation | **Very good, now multi-template (Phase 4)** | Evidence built **programmatically**, not by the LLM — the model was observed mis-transcribing a port number. Exam/format templates: OSCP (per-host walkthrough + a proof.txt table spliced from state), CPTS (exec-summary + findings register), H1/Bugcrowd (impact-first + a CVSS 3.1 score *computed*, not asserted). proof.txt/local.txt are real per-host state (auto-captured when a command reads a flag file, or pasted). Collision-proof fences; lab vs REAL-TARGET labelled; detection footprint appended grounded-only; optional red-team OPSEC roll-up (D10). |
| Assistant chat | **Works** | Session-aware, KB-grounded, deterministic citation extraction (scans the reply for ids that resolve). |
| Scripts arsenal | **Works** | 1,028 scripts deduped from the KB across 6 groups. |

## Cockpit (execution)

| Component | State | Notes |
|---|---|---|
| Gated executor | **Excellent design, now properly tooled** | Four ordered gates, mode split, argv-only (never a shell), SSE streaming, run records persisted. The image it execs into now ships ~63 of the 73 catalogued tools (Phase 1). |
| Isolation gate | **Real** | `assert_isolation_proven()` structurally inspects every attached network for `internal: true` . Not a comment — an actual check. |
| Engagement mode | **Correct** | Explicit entry required, fail-closed on unknown/exited id, per-command approval re-checked even on the prevalidated path (belt-and-suspenders in `iter_run` ). |

| Component | State | Notes |
|---|---|---|
| Scope model ( `scope.py` ) | **Good** | Hosts, `*.wildcards` , CIDRs, `!exclusions` , `*` opt-out. Fail-closed. No DNS at match time (avoids the "resolve an out-of-scope name to decide it's out of scope" leak). The `_looks_like_host()` false-positives are fixed (Phase 3). |
| Recon-driven expansion | **Good** | Mines run output for hosts, sorts by scope, in-scope join the live allowed set, out-of-scope surfaced read-only. Cannot widen the scope. Capped and never silently truncating. |
| Orchestrator loop | **Now state-grounded** | Proposes one command, never executes. Feeds on the structured state model + live task tree (Phase 2) instead of stdout tails. |
| Engagement state model | **New (Phase 2)** | hosts/services/endpoints/credentials/findings, upsert-only, fed by output parsers + loot-file ingest; drives the planner and a UI panel. Executes nothing (AST-asserted). |
| `:kali` | **Now a persistent shell (Phase 3)** | One long-lived `docker exec -i sh` ; `cd` /env/background jobs persist. Same containment (hardcoded open container, human-only, audited, no isolation gate). Deliberately no PTY — that is what keeps its transcripts clean (§1.1). |
| `:terminal` | **Real PTY, second surface (Phase 5)** | `pty.fork()` inside the same open box, driven over a framed WebSocket to xterm.js; full-screen tools render and resize. Containment mirrors `:kali` point for point; the raw stream is audited. Does not replace `:kali` (§1.1). |
| `:exploits` | **New (Phase 5)** | Version-keyed CVE → exploit lookup over the sandbox's local exploit-db catalogue — 47k exploits, 25k distinct CVEs. Read-only; executes nothing. |
| Live sessions | **Well-designed, now tooled** | Start is a gated command; stdin is human-only and source-scan locked. |
| HTTP repeater | **New (Phase 4)** | Compose/send/replay/diff. Argv-only curl inside the hardcoded open box (no shell parses a request field; body on stdin), human-only + source-scan locked like `:kali` , scope-checked against a named engagement, every send run-recorded. |
| Pivot / tunnels | **New (Phase 4)** | chisel/ligolo-ng lifecycle (human-only start/stop) + pure route resolution and a **visible** proxychains rewrite applied *before* the approval screen. A tunnel's subnet enters scope only via an explicit, audited amendment; recon expansion still cannot widen. |

## AD graph + orchestration

| Component | State | Notes |
|---|---|---|
| BloodHound parser | **Genuinely good** | Handles v4/v5/CE, zip/dir/json/bytes/mapping, reconciles naming drift, synthesizes DCSync from `GetChanges` + `GetChangesAll` , emits coverage warnings for missing collection methods. Real work. |
| Path engine | **Correct** | BFS shortest path over abusable edges only, abuse-rank tie-break, k-shortest-ish alternatives. |
| Technique catalog | **Good** | 25 edge kinds, KB-grounded with catalog fallback, target-type specialization ( `GenericAll` on a group → `AddMember` ). Each edge now also carries a **native Windows variant** (PowerView/Rubeus/Mimikatz) for live WinRM execution alongside the Linux impacket/evil-winrm one. |
| Orchestrator | **Excellent safety design** | The model picks an **edge index**, never a command. Cannot invent a host, cannot author a command, cannot reach an edge outside the collection. A pick outside the list is refused rather than repaired. Now runs **live** — the approved command executes over WinRM on a selected Windows target (proposes, never auto-fires; regression-locked for the WinRM path too). |

| Component | State | Notes |
| --- | --- | --- |
| `advance` endpoint | **Excellent** | Advancement requires a `run_id` that was **approved** and **exited 0**, verified server-side — transport-agnostic, so the WinRM path advances the walk identically. |
| Execution tooling | **Now present + verified live** | impacket, certipy-ad, bloodyad, netexec, evil-winrm, bloodhound.py, kerbrute, responder, mitm6 are in the image (Phase 1). The **WinRM driver** (Windows execution backend) runs the abuse **live** against a real Windows/AD box you run in VMware — the graph is no longer synthetic-only. **Verified against a real domain controller in build #9** (2026-07-31, `backend/livefire.log` : 45 PASS / 0 FAIL / 3 NOT-RUN): a live WinRM round trip reached a real `corp.local` DC and returned `corp\administrator` , and `bloodhound-python` collected the real forest into the typed graph (84 nodes / 625 edges), asserted to be the operator's actual domain rather than `sample_data` . Unit tests remain hermetic (WinRM mocked) — the live run is the separate evidence, and it found four defects the green hermetic suite could not see. |

### Detection footprint

**Read-only, honest, well-sourced.** Real SigmaHQ rule UUIDs, real ATT&CK ids (Enterprise v19.1), and a verifier script ( `pipeline/detection_sources.py --verify` ) that re-checks every id against live upstream. The "describes what blue sees, never how to be seen less" line is enforced in code. **Scale:** ~133 command specs, 19 distinct ATT&CK techniques — a good purple-team *flavour*, not a coverage tool.

**CRTP conflict — now reconciled (Phase 4, D10).** CRTP includes AMSI/logging evasion, which the panel used to refuse outright. It now has an **additive second channel**: the blue-team detection view is byte-for-byte unchanged and still passes the never-prescribe guard, and a separate, opt-in `opsec` channel adds the offensive half — what makes a command loud, the quieter tradecraft, and, mandatorily, *what still records it*. The OPSEC channel has its own guard (it may never advise disabling/clearing/tampering with a sensor); the LLM may extend it for uncatalogued commands, marked `ai_suggested` . See Part III, Phase 4.

### Code scan (SAST)

**Well-built and genuinely safe.** Static-only invariant asserted at every `_spawn()` and again by `test_codescan_safety.py` . Deliberately orthogonal — imports nothing from the engagement/executor/scope model. **Limits — now widened (post-assessment):** the offline bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added ( `rules/hackpit-languages.yaml` : command injection, SQLi, unsafe deserialization, code-eval, file-inclusion, SSRF), plus a **ruleset picker** (bundled / per-language / a registry pack for the full online catalogue). Still offline-first. See "Post-assessment refinements".

### Frontend

16 routes, real API wiring throughout, SSE streaming, no mocked data layer ( `cockpitSample.ts` is the only sample content, explicitly labelled). New in Phases 1–3: the arsenal availability band, per-run time-budget + detach controls, the engagement-state/task-tree panel, the credential-vault "use" action, and the persistent `:kali` shell UI. **Gaps:** no global current-target/engagement context, no engagement export/import, no multi-target view, and the accepted 10-error `react-hooks` lint baseline (documented; `next build` passes exit 0).

## 3. Knowledge base assessment

**2,617 entries** (1,601 at the time of the assessment; +66 in Phase 4, +950 in Phase 5) · **median body 3,000 chars.**

**Source distribution:**

| Source | Entries |
|---|---|
| hacktricks | 715 (45%) |
| madstuff | 260 |
| some-hacking-resources | 233 |
| writeups | 59 |
| payloadsallthethings | 49 |
| claude-bug-bounty | 46 |
| peh-notes | 43 |
| htb-writeups | 41 |
| claude-red | 39 |
| oscp-cpts-notes | 36 |
| decepticon | 33 |
| htb-academy | 13 |
| **hackpit-authored** | **12** |
| htb-box-pdfs | 9 |
| galaxy-checklist | 7 |
| htb-my-resources | 5 |
| shodan-dorks | 1 |

*Phase 5 added:* hacktricks-cloud **578** · portswigger **372**.

**Tiers:** 2,265 tier-3 · 241 tier-2 · **111 tier-1** (your own) — see finding 2. **Ranking rebalanced (post-assessment):** tier is now a *substance-gated* nudge, not a blanket trust prior — a thin tier-1 stub no longer outranks richer, more relevant lower-tier content, and a command-rich page wins close calls regardless of tier. See "Post-assessment refinements".

### Findings

1. **PayloadsAllTheThings payload depth — now recovered (Phase 4, D11/D12).** PATT's 66 `.txt` payload lists + the shodan dork list are ingested at **payload-level granularity** (64 `payload-set` + 2 `dork-list` entries, `no_merge`-guarded so consolidation can never collapse them again), each a searchable entry over a full sidecar corpus mounted read-only at `/payloads`; 56 `oscp_tools` files went to the Scripts Arsenal. KB 1,601 → 1,667. See Part III, Phase 4.

2. **"Grounded in your own notes" is mostly "grounded in HackTricks."** Search boosts tier-1 ( `search.py:58` ), but with 111 tier-1 entries that lever has little to pull on. **Growing tier-1 is the highest-leverage KB work still available — still open**, and now the *only* open KB item. Phase 5 grew the KB by 950 entries, all tier-3, so the ratio moved the wrong way: the fix is writing, not ingesting.
3. **Empty categories — mostly fixed (Phase 5).** Was: cloud 2 · mobile 2 · iot 2 · forensics 1 · ics 1 · phishing 1 · supply-chain 1. Now **cloud 535** and **supply-chain 47** (HackTricks Cloud), and web/API coverage is deep (PortSwigger). **Still thin:** mobile · iot · forensics · ics · phishing — none of which is on the target list, so they stay unfilled by choice rather than oversight.
4. **HTB Academy — narrowed, not closed (Phase 5).** The local folder is an 18-module *slice*, not the CPTS syllabus, and HTB content is proprietary so nothing is scraped. Auditing the slice against what had been ingested found two files silently lost — one tracked in git but absent from disk, one lost purely for having a `.txt` extension — both now recovered. Closing this properly needs the course itself.
5. **Missing as retrievable topics — fixed (Phase 5).** API security (GraphQL/REST), OAuth/SSO, race conditions, business logic, request smuggling, prototype pollution, deserialization and SSRF→cloud-metadata chains all now resolve, most to PortSwigger Academy material with the lab that drills each one alongside.
6. **No CVE/exploit index — fixed (Phase 5).** `:exploits` answers service+version → CVE → public exploit over the sandbox's own exploit-db catalogue, version-compared rather than substring-matched.

Item 2 is the only KB item still open. Items 3–6 were the recommended next batches; they were never in scope for the first four phases and were built in Phase 5.

### Recommended ingestion, in priority order — status

1. **PortSwigger Web Security Academy** — the single best web-security source, and structured. **Done (Phase 5, +372).**
2. **HTB Academy modules** properly (CPTS syllabus). **Partially — the local slice is fully ingested; the syllabus needs the course.**
3. **HackTricks Cloud. Done (Phase 5, +578).**
4. **PayloadsAllTheThings re-ingested at payload-level granularity** (see §4). **Done (Phase 4).**
5. **An exploit-db / CVE mapping** keyed on service+version. **Done (Phase 5) — and deliberately not as KB prose; see Part III.**

---

## 4. Source-usage audit — "did I use my sources properly?"

Measured on disk against the built KB, not inferred.

### 4.1 The headline finding: the pipeline only reads `*.md`

`pipeline/ingest.py:229` → `sorted(source_path.rglob("*.md"))` ; `pipeline/ingest_notes.py:599` likewise. **Every `.txt` , `.json` , `.py` , `.sh` , `.csv` , `.yaml` file in every source tree was invisible to the pipeline.**

| Source folder | Files | `.md` | KB entries | Non-md skipped | Of which is *real content* |
|---|---|---|---|---|---|
| `hackdic` | 807 | 806 | 715 | 1 | — |

| Source folder | Files | .md | KB entries | Non-md skipped | Of which is *real content* |
|---|---|---|---|---|---|
| `madstuff` | 798 | 797 | 260 | 1 | — |
| `some hacking resources` | 233 | 233 | 233 | 0 | — |
| **`PayloadsAllTheThings`** | 481 | 134 | 49 | 347 | **66 `.txt` payload lists + 28 `.py` scripts** |
| `oscp-cpts-notes` | 288 | 85 | 36 | 203 | 174 PNG screenshots (+ git internals) |
| `oscp_tools` | 99 | 1 | 0 | 98 | 28 `.exe` binaries, ~22 `.ps1` / `.sh` / `.py` scripts |
| `HTB_academy` | 48 | 16 | 13 | 32 | 3 PNGs — effectively nothing |
| `shodan-dorks` | 32 | 1 | 1 | 31 | **1 `.txt` dork list** |
| `Hacking-Tools` | 31 | 1 | 0 | 30 | nothing — git internals only |
| `PRACTICAL ETHICAL HACKING NOTES` | 110 | 45 | 43 | 65 | images |
| `more new resources` (writeups) | 118 | 72 | 100 | 46 | images |
| `htb my resources` | 5 | 5 | 5 | 0 | — |
| `htb boxes` | 10 (pdf) | 0 | 9 | — | PDF path handled separately |
| **`PentestTools`** | **0** | 0 | 0 | — | **empty folder** |

**Scale:** stripping git internals, images and compiled binaries, the genuinely lost knowledge was roughly **95 files** — chiefly **PATT's 66 `.txt` payload lists** (the pipeline ingested the *explanations* and discarded the *payloads* — backwards for bug bounty), the **shodan-dorks `.txt`**, and **~22 `oscp_tools` scripts**. **Fixed (Phase 4, D11/D12):** an additive Route-B ingester ( `pipeline/ingest_corpora.py` ) folded all 66 payload lists + both dork lists into the KB as `payload-set` / `dork-list` entries with full sidecar corpora, and 56 `oscp_tools` files into the Scripts Arsenal — without rewriting a single existing entry (verified byte-identical) and idempotently. Two env traps hit and handled: `rglob` silently omitted 2 of 66 files (OneDrive dehydration → union with `git ls-files` ) and 22 files were AV-locked (→ git-blob recovery).

### 4.2 Consolidation is lossy for some sources

`madstuff` 797 md → 260 (33%). `oscp-cpts-notes` 85 md → 36 (42%). `PayloadsAllTheThings` 134 md → 49 (37%). Some is correct deduplication into HackTricks; a 3:1 collapse on your OSCP/CPTS notes is worth an eyeball — those are exactly the entries you'd want surviving as tier-1/tier-2.

> **Audited (post-assessment) — healthy dedup, no loss.** A read-only provenance audit of the OSCP set: **36 survived as their own entries (33 with real commands)**, **12 folded into stronger canonical pages** (PayloadsAllTheThings/peh-notes/some-hacking-resources — preserved as `also_covered_in` + variants, not deleted), and the remaining ~37 were **same-technique OS/lab splits merged together** (the GitBook ships Linux/Windows/lab variants as separate files). **Zero** OSCP entries were dropped as low-value (only 11 were dropped KB-wide). Also corrected: `oscp-cpts-notes` is a **tier-3 public GitBook repo**, not your own writing — your genuine tier-1 comes from `writeups` / `peh-notes` / `htb-my-resources`. **Decision: no re-ingest** — re-adding with `no_merge` would just recreate duplicate technique pages that hurt search. See "Post-assessment refinements".

### 4.3 The biggest missed idea — PentestGPT's Pentest Task Tree — is now BUILT

The assessment's single highest-value reasoning-layer recommendation was PentestGPT's task tree: a *persistent, hierarchical, status-tracked engagement state the model reads from and writes back to every turn.* **This shipped in Phase 2** ( `backend/state/tasks.py` ) — layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a`, updated as results arrive via model-proposed operations that code validates. It is retained here as the rationale for what was built. (Implementation note: HackPit's variant has the model return *validated operations* rather than rewriting the whole tree, so one bad response cannot silently rewrite the plan.)

### 4.4 The structural pattern

**The pipeline is excellent at *techniques* and lossy on *lists, workflows and payload corpora*.** A checklist is a *sequence*, a dork list is a *set*, a payload collection is a *corpus* — all three were forced through a technique-shaped, markdown-only schema and collapsed. **Both fixes shipped (Phase 4):** the Route-B ingester accepts non-markdown inputs, and the second entry shapes ( `meta.kind` = `payload-set` / `dork-list`, `no_merge` -guarded) survive consolidation intact. The `checklist` shape is defined and available; no checklist source was in the D12 scope, so none were ingested yet.

---

## 5. What is genuinely excellent — do not refactor

- **The safety architecture is real.** Source-scan regression locks on `:kali` and session stdin, four ordered gates, clean lab/engagement mode split, `assert_isolation_proven()` as a structural runtime check. Better than most commercial tooling. (The Phase 1–3 work preserved every one of these invariants; the test suite grew with it.)
- **The grounding discipline is the best idea in the project.** Grounded-vs- `ai_suggested` labelling; entry_ids must resolve or the step is downgraded; commands come from the KB, never the model; **evidence spliced programmatically into reports rather than written by the LLM**; `foreign_refs` as an honesty marker; the Channel-2 leak guard.
- **The AD orchestrator picks an EDGE, not a command.** A safety design that also improves output quality. The Phase-2 task tree follows the same principle (validated ops, not free-form rewrites).
- `advance` **requires an approved, exit-0 run** — evidence-based state transition, verified server-side.
- **The consolidation pipeline** — alias-key + cosine match, structural merge, idempotent, provenance preserved.
- **The detection footprint's framing and sourcing** — real Sigma UUIDs with a drift verifier.
- `substitute_target` — CIDR sentinel, host-position gating, refusal to rewrite what it can't rewrite confidently.

- **The BloodHound parser** — v4/v5/CE reconciliation with honest coverage warnings.

---

## 6. Security posture of HackPit itself

- **No authentication anywhere.** No `Depends`, no key, no session. CORS restricted to `localhost:3000`. Honestly documented in code.
- On a laptop this is fine. **On a VPS attack box it is an unauthenticated RCE endpoint** — `POST /cockpit/kali` (and the new persistent-shell routes) run arbitrary `sh -c` in a container with full network reach to your host and LAN. Auth is a hard blocker before any non-localhost deployment.
- Secrets handling is correct: `llm_config.json` gitignored, API key written to disk but never returned, collector preview redacts the password/hash. The engagement-state DB (which now holds **real captured credentials**, by decision) is gitignored like the rest.
- The repo is code-only; KB, sources, sessions DB and `.env` are all gitignored. A prior full-history audit confirmed no secrets were ever committed.
- **Known:** a self-scan flagged SSRF and CORS misconfiguration in HackPit's own backend (memory obs #1496). Worth revisiting before any exposure.

---

## 7. What the assessment examined

**Read in full:** every non-test backend module (executor, allowlist, config, sandbox, engagement, scope, kali, session, runstore, router, models; attack_path, main, orchestrator, llm, report, chat, context_channel, sessions; all of adgraph, arsenal, detection, codescan), `pipeline/search.py`, `docker/Dockerfile.sandbox`, `docker-compose.yml`, the QA report, and the live KB.

**Structurally mapped:** the frontend (routes, API surface, component→endpoint wiring, demo/mock scan — none found beyond the labelled `cockpitSample.ts`); the test files; `pipeline/consolidate.py`.

**Source trees inspected on disk:** `hexstrike-ai` (151 `@mcp.tool` defs), `mantishack`, `Decepticon`, `PentestGPT` (task-tree prompt read directly), `claude-red`, `claude-bug-bounty`, `Galaxy-Bugbounty-Checklist`, `hackdic`, `htb boxes`, the writeup trees, `some hacking resources`. File counts and markdown ratios measured per tree.

**Verified live (assessment):** container tooling (37-tool probe), SYN-scan capability, nuclei templates, container capabilities, KB statistics, ingester file-type globs.

---

# PART II — WHAT REMAINS

*Decided with Zaid, 2026-07-26. All five phases and every decision that drove them are in Part III. The build is complete; this section is only what is intentionally left — some of it deferred, some of it rejected outright.*

## Standing policy (unchanged, still governs the project)

These are not open work — they are the rules the project runs by, and building is finished, so they no longer need a decision table. Kept here as the active policy:

- **D2 — Windows + Docker Desktop; a VPS is added later, only for bug-bounty callbacks.** Docker Desktop is a real Linux VM (WSL2); the one thing it can't give is a publicly reachable listener for blind SSRF/XXE/RCE. The repeater (Phase 4) sends and reads the *direct* response; the VPS piece is still deferred until bounty work needs it.
- **D5 — Per-command approval stays. Risk-tiering is rejected, not deferred.** One-keystroke approve shipped (Phase 3). Tiering — auto-running "passive" commands, batch-approving read-only plans — was reconsidered and **decided against** (§1.2): in engagement mode per-command approval is the *only* thing bounding where a command may go, and a classifier deciding when to skip the human is a new component on the wrong side of that boundary. Every execution surface since preserves this — repeater, tunnels (rewrite *visible before* approval), and Phase 5's `:terminal` (human-only, no agent path).
- **D8 — HexStrike is a reference, never an execution backend.** Its tools bypass all four gates, the target-lock, the scope model and the audit trail. Rejected, not deferred.
- **D9 — CRTP Windows execution: CLOSED via a WinRM driver against an external VM (was "accept the gap").** The original decision accepted that CRTP's PowerShell/.NET tooling can't run on the Linux container. That gap is now closed the honest way: **no Windows VM lives inside the project** — HackPit *drives* one you run in VMware over WinRM (Model A), a new execution transport behind the same gates. Windows-only tools now report runnable when a Windows target is selected (reconciled per active target); on a Linux run they are still N/A and marked. The AD graph executes live against that target. See the Windows execution backend in Part III.

## Explicitly deferred (with the reason)

- **A VPS for blind out-of-band callbacks** (D2) — a NAT'd laptop has no public listener; deferred until bounty work needs it.
- **A Windows execution target for CRTP** (D9) — **DEMONSTRATED LIVE** (build #9, 2026-07-31): the WinRM driver executes CRTP/AD work live on an external VMware VM you run. No longer "built but only unit-tested" — a real PowerShell round-trip lands on the profile host over WinRM, the whole-script danger classifier fires on real input, the credential leaks into nothing, and output ingests into state (Task 1, 14/14). See build #9 in Part III. The one caveat still open is a full Windows-target C2 callback (Task 4), which needs a routable listener.
- **The AD graph run off real collection, not synthetic data** — **DEMONSTRATED LIVE** (build #9): the gated, scope-locked `bloodhound-python` collected the real `corp.local` (84 nodes / 625 edges), it parsed into the typed graph rather than `sample_data`, and the route to Domain Admin was computed on the real graph (Task 2, 10/10). The live abuse walk then executed a real DCSync over the domain — four NTLM hashes including `krbtgt` — through the danger red-confirm, with real loot ingesting into state and the walk advancing only on

the approved exit-0 run (Task 3). Four defects that only a real domain could surface were found and fixed (credential-in-record leak, impacket parser-name mismatch, KB grounder arming a free edge, collector FQDN classifier), each regression-locked. The native-tooling abuse variants (Rubeus/mimikatz *on* the DC) stay not-run: a bare Server 2022 has no offensive tooling and staging it was out of scope.

- **Growing tier-1** (§3, finding 2) — the only KB item still open, and the one that cannot be solved by ingesting: it needs Zaid's own writing.
- **The HTB Academy syllabus proper** (§3, finding 4) — the local slice is fully ingested; the rest is proprietary and needs the course.
- **Thin categories with no target on the list** — mobile · iot · forensics · ics · phishing (§3, finding 3). Unfilled by choice.
- **C2-framework install + the persistence evasion/OPSEC half** — **DONE** (build #4, no longer deferred): Sliver is installed and wired to a gated panel, and the persistence OPSEC notes were backfilled. Empire and weevely remain catalogued reference-only by choice.
- **Live-fire verification of the C2 and tunnel surfaces (build #4) — DEMONSTRATED IN LAB, with exact caveats.** A controlled lab exists ( `docker/proof/live_fire_proof.sh` + `live-fire-lab.yml` , four containers on `internal: true` networks) and drives the surfaces through their shipped gated entry points: **66 passed, 0 failed, 3 not-run.** What is demonstrated live: an implant that actually builds and **a beacon that calls back**; a **proxychained request routed through the pivot** into a subnet the operator box has no route to, using the argv `wrap_command` produced; all three lifecycle gates (Sliver server, both pivot kinds, DNS listener) refusing on all three limbs with nothing spawned; every listener coming up, being confirmed bound by `ss` inside the container, and staying bound for the whole phase; the pivot subnet entering scope only via the explicit amendment, with the deep target confirmed unreachable beforehand; the DNS key absent from the pasteable one-liner; and **containment holding live, measured from inside the implant host** — no internet, no host, C2 only. What is **not** demonstrated, and why, kept deliberately separate from the above:
    - **The three defects the first live-fire run found are FIXED (2026-07-28), and the fixes are what the numbers above measure.** The Sliver implant build now runs through the console `sliver-server` hosts (Sliver 1.5 has no `generate` subcommand on either binary) and decides `generated` vs `failed` by reading the artifact back rather than trusting a console exit code; both listener lifecycles now hold a console binary's stdin open and report a status they **observed**. Fixing them surfaced two more of the same family, also fixed: a stop that killed the `docker exec` client while leaving the server running inside the container (next start: `EADDRINUSE` ), and a `proxychains` config shipped pointing at Tor rather than the SOCKS5 port HackPit's own rewrite targets.
    - **iodine's IP-over-DNS tunnel is now demonstrated** carrying traffic through the gated surface, confirmed DNS-encapsulated on the wire (2026-07-28). **dnscat2's handshake is decisively an upstream defect** in the pinned build — `tcpdump` shows the server itself answering `NXDOMAIN` to correct queries, reproduced over loopback in one container — sitting above HackPit's proven-bound listener. What stays operator-side: a **delegated DNS zone** (a domain with an NS record pointed at a publicly reachable listener, the D2 VPS gap), which neither tunnel can exercise without it.
    - **ligolo's interface routing** needs `session` then `start` typed into the proxy's console. HackPit holds that console's stdin open and deliberately never types into it, so completing a ligolo session stays the operator's step; the routed-traffic claim is carried by chisel, whose server needs no console. **Detonating an artifact on an instrumented Windows host** remains untouched (below), and pairs with the D9 VM. The original list, for the record:
    - **Catch a live Sliver beacon.** The server lifecycle starts and stops and is audited, but no implant has ever called back. Needs a listener reachable from a victim host — i.e. the same VPS gap as D2, or a lab VM on a

routable segment. Until then the implant argv is only as correct as the catalog's documented syntax; a wrong flag surfaces as `status='failed'`, not as a containment failure.

  - **Stand up a real DNS tunnel.** `dnscat2-server` and `iodined` start inside the engage sandbox and the client one-liner is handed back, but no delegated zone exists to test against. Needs an authoritative zone the operator controls with an NS record pointed at the listener. `iodined` also needs a TUN device in the engage sandbox — plausible given its `NET_ADMIN`, but unproven.
  - **Run a generated artifact on a Windows box and watch the telemetry.** The evasion engine emits artifacts and the paired footprints claim specific Event IDs and Sysmon codes. Those claims are transcribed from ATT&CK v19.1 and SigmaHQ, not observed here. The honest version of this feature is only fully honest once someone detonates an artifact on an instrumented host and confirms the footprint matches what actually landed in the log. This is the natural pairing with the D9 Windows VM. None of this blocked the build shipping — the containment properties are what the test suite locks, and those are provable without live infrastructure. Build #7 converted the containment half from "locked by tests" to "holds live", and the efficacy half from "documented" to "three specific defects, named and reproducible" and then to "fixed, with the callback and the routed traffic actually observed". What remains before build #4 could be called field-ready on these surfaces is the DNS session and a delegated zone.

- **An independent review of build #4's Tasks 8–13 — DONE** (no longer outstanding). Run in a fresh session (the safety classifier had begun refusing subagent dispatch partway through the build, and it refuses on accumulated context). It found **no containment hole** — no agent path, no shell, the container never request-influenced, no delivery primitive — and five substantive defects, all fixed: a multi-technique request that built one artifact and described a different one; the T1690/T1685 mis-mapping above; two stub headers describing memory patching when the stubs are managed-reflection bypasses; a no-agent-path test whose positive control counted the wrong variable (99 files reported, 5 actually scanned); and a `_gate_request` comment asserting a scope-gate behaviour that does not hold. Plus seven minor items — an image smoke test that could not fail, unpinned installs, dead fields, weak assertions. Every fix carries a test that fails without it.
- **Gate-integrity Criticals 2 and 3 — DONE (build #5, 2026-07-27).** Both closed; full detail in Part III. **Critical 2** (the WinRM first-token classification — the sharpest known hole in the tool, because it defeated the red-confirm by moving a cmdlet one token to the right on the one path that reaches a real domain-joined host) is fixed by classifying the whole joined PowerShell script, deliberately without parsing it, with the gate and the transport now deriving that string from one shared function. **Critical 3** (source-scan locks covering a fraction of the tree) is fixed by one shared scanner: whole-tree `rglob`, repo-relative allow-lists, an AST pass for indirection, and per-lock planted-violation controls — coverage per lock goes 30 of 69 modules to 67. The plan's third task, the one that matters longest, is written down in `backend/AGENTS.md`. **`I2`, the ungated tunnel listener start, was outside build #5's plan but has since been closed** — `start_tunnel` now runs the real `validate_request` before spawning, `TunnelStartRequest` carries `approved` / `dangerous_ack` defaulting false, and the route 403s a safety refusal; detail in Part III.
- **Pinning the build-#4 install layer** (audit M2) — **DONE** (build #7, no longer deferred). All three float-installs are pinned to real resolved values: dnscat2 to commit `42f8d783…`, the gems to `trollop:2.9.10 salsa20:0.1.3 sha3:2.2.4 ecdsa:1.2.0`, `donut-shellcode==1.1`. `test_evasion.py` now asserts full pinning (three installs, three `ARG` pins, a real 40-char SHA, no floating form left behind) rather than tolerating a declared gap. The fresh build also surfaced a smoke test that **had never passed** — `sliver-server version | grep -qi sliver`, against a binary that prints only `v1.5.42 - <sha>` — so that layer had not built since the check was added; both sliver checks now match `v${SLIVER_VERSION}`, which is strictly stronger. `hackpit-kali:build7` builds clean with every pinned tool resolving and smoke-testing by its real invoked name.
- **The prevalidated path's danger re-check — DONE** (build #7). `iter_run(prevalidated=True)` re-checked approval but not danger; nothing was exposed (the single such caller validates first) but the asymmetry was a

trap for the second caller. It now re-checks danger in all three modes using the same per-mode classifier `validate_request` uses, regression-locked in `test_prevalidated_gates.py`.

- **Kali VM as an execution target; HexStrike as an execution backend; risk-tiered / batch approval — rejected, not deferred** (D8, D5, §1.2).
- **Authentication before any non-localhost deployment** (§6) — no decision needed until a VPS enters the picture, but a hard blocker the moment it does. `:terminal` makes this sharper: an unauthenticated *interactive terminal* onto a box that reaches the host and LAN. **Still outstanding, and now quantified (2026-07-31).** A Cloudflare Tunnel + Access + app-level-auth build was scoped in full and then deliberately not taken. The scoping established the exact state of the gap: `backend/main.py` has **no authentication of any kind on any route** — no `Depends`, no HTTPBasic, no API key — and the CORS allowlist is a *browser* policy that stops nothing which is not a browser. The sharpest edge is the single WebSocket route `/cockpit/terminal/ws`, a live PTY into the Kali container: gating HTTP alone would leave a free shell. Localhost-only remains the default and the mitigation.
- Screenshot capture, recon diffing/monitoring, checklist-driven runs.
- **A README refresh** (build #12) — three items, held for a session of their own: state plainly that `data/kb` (the ~2,743-entry technique/workflow dataset) is **gitignored and not in the repo**, so a fresh clone's search is empty until the KB is obtained from the author; mention the CI now that the suite runs on every push; and correct the stale counters (README says 2,621 entries / 110 tools / 42 test files — measured: **2,743 / 115 / 56**).
- **Continuous integration — DONE** (build #12, no longer outstanding). `.github/workflows/ci.yml` runs the 56-file hermetic suite, `next build` and an eslint-baseline check on every push and PR, with the live ATT&CK/Sigma drift verifier on a weekly schedule rather than the merge path. The KB the fingerprint locks need is supplied by a derived, equivalence-proven fixture (D26) rather than skipped. All three jobs have since been **run on GitHub and passed**, the drift job by manual dispatch after it was found never to have executed at all.

## reconFTW — recon gaps mined, and what to skip

reconFTW (six2dez, MIT) was reviewed for what HackPit's *recon* was missing — its thinnest area (the incorporation narrative is in Part III). Respecting the constraint that governs everything here — HackPit is human-gated, so "incorporate" means adopt reconFTW's **knowledge**, never its auto-runner — the concrete gaps, where each landed, and rough effort:

- **URL-triage pipeline** — `gf` + `qsreplace` + `unfurl` (arsenal `web`) + a KB methodology entry. The single biggest bug-bounty gap: HackPit harvested URLs but had no triage step. *Low — done.*
- **Subdomain permutations** — `gotator`, `regulator`, `subwiz` (arsenal `recon`). An entire enum technique HackPit had none of. *Low–med — done.*
- **Mass resolution** — `puredns` + `massdns` + `dnsvalidator`, with a baked resolver list (arsenal `recon` + image). HackPit had only `dnsx`. *Med — done.*
- **JS mining** — `jsluice`, `subjs`, `getjs` (arsenal `web`): endpoint/secret extraction, not just crawling. *Low–med — done.*
- **ASN → CIDR expansion** — `asnmap`, `mapcidr` (arsenal `recon`) for wide-target scope work. *Low — done.*
- **External OSINT** — `trufflehog`, `github` / `gitlab-subdomains`, `dorks_hunter`, `gitdorks_go`, `msftrecon`, in a **new `osint` category** (HackPit had no OSINT home). *Low — done.*
- **Bucket/cloud enum** — `s3scanner`, `cloud_enum` (arsenal `cloud`): external asset discovery, distinct from the posture scanners. *Low–med — done.*

- **Injection completers** — `commix`, `SSTImap`, `nomore403`, `crlfuzz` (arsenal `web`): fill the cmd-injection / SSTI / 403-bypass / CRLF tool gaps (skills existed; tools didn't). *Low — done*.
- **Recon ordering** — the subdomain-enum sequence and the URL→gf-triage→fuzz pipeline, as two KB `checklist` entries (`recon-methodology-*`) + an advisory note in the composer's real-target prompt. The higher-leverage half — tools without the ordering is the cheap half. *Low–med — done*.

**Skip — conflicts with the human-gated model (rejected, not deferred):** the `reconftw.sh` runner and `reconftw.cfg` engine (autonomous chaining is the exact thing HackPit rejects), `interlace` (parallel command execution), `notify` (autonomous alerts), monitor/incremental/diff auto-rescan, the axiom/Ax fleet, `reconftw_ai` and faraday. `inscope` is redundant — `scope.py` is stronger (fail-closed, no DNS at match time). `interactsh`/OOB stays deferred on the VPS (D2, unchanged). `waymore` was considered and left out: `gau` + `waybackurls` + `katana` already cover URL harvesting.

---

# PART III — BUILD LOG (Phases 1–5, shipped 2026-07-26)

Branch `sandbox-kali-image`. Full hermetic safety suite green throughout (36 test files); both Docker proofs 4/4 (lab still egress-less; engage fully open); browser-verified with Ollama ( `qwen3:8b` ).

## Decisions taken (D1–D22)

Every decision that drove the build. D1/D3/D4/D6/D7 were the Phase-1 build; D9 was a standing accepted gap that is **now built** (the Windows execution backend); D2/D5/D8 are standing policy (Part II); D10/D11/D12 were Phase-4 work, now built; D13 is this document; D14/D15 shaped Phase 5; **D16/D17 are build #4** — D16 amends D10 and is the one policy reversal in the project's history.

- **D1 — Fix the sandbox image; HackPit *is* the attack box.** A Kali VM's advantages (root, raw sockets, VPN, `/etc/hosts` ) are all reachable in Docker via capabilities + `/dev/net/tun`. *Built (Phase 1).*
- **D3 — Capabilities + root on the ENGAGE sandbox only.** Lab keeps `cap_drop: ALL` + non-root. *Built.*
- **D4 — All three sandboxes get the new toolset; only the lab's *network* isolation stays.** *Built.*
- **D6 — Catalog describes reality, not aspiration.** *Built.*
- **D7 — Startup reconciliation check.** *Built.*
- **D9 — CRTP Windows execution.** Was "accept the gap; no Windows VM." Now **closed by the WinRM driver** (Windows execution backend): HackPit drives an external VMware VM you run — no Windows VM inside the project. Windows-only tools run when a Windows target is selected; still N/A + marked on a Linux run. *Built (the AD graph executes live).*
- **D10 — Allow evasion/OPSEC content as an additive second channel, keeping the blue view.** *Built (Phase 4).*
- **D11 — Fix the KB gap via Route B (additive merge), not a rebuild.** *Built (Phase 4).*
- **D12 — Ingest PATT payloads + shodan dorks into the KB; `oscp_tools` into the Scripts Arsenal.** *Built (Phase 4).*
- **D13 — Plan lives in this document.** *Done.*
- **D14 — The PTY is a SECOND surface, never a replacement.** The old framing treated auditable transcripts and full-screen tooling as alternatives. They are only alternatives on one surface. `:kali` keeps its sentinel; `:terminal` gets the pty; both are audited and identically contained, and a test asserts `kali.py` never grows a pty. *Built (Phase 5).*
- **D15 — The CVE index is a lookup table, not a KB batch.** "Find the exploit for this version" wants an exact, deterministic table; embedding it as prose would make the one query it exists to answer fuzzy. Its own data shape, search path and surface — and it executes nothing. *Built (Phase 5).*
- **D16 — Lift the OPSEC sensor-tamper ban; the honesty marker becomes the sole invariant.** Amends D10. The OPSEC channel previously refused any sensor-blinding phrasing, which became incoherent once build #4 shipped an engine that emits AMSI-patch and ETW-blind artifacts: the tool would have been producing artifacts it was forbidden to describe. The ban is lifted **entirely**, and two invariants carry the whole weight

instead — every note must name what **still records** the activity, and the blue-view footprint is always produced alongside and can never be suppressed. The blue-side describe-never-prescribe guard and all four execution gates are **unchanged**. A posture shift on a public repo, taken deliberately. *Built (build #4).*

- **D17 — Split the gating for C2 rather than gate it uniformly.** Server and listener **lifecycle** is human-only with no red-confirm (operator infrastructure on the operator's own sandbox — no target is touched, so clicking start is the approval); **artifact generation** is a fully gated command (approval + scope + red-confirm) because it produces something that will run on someone else's machine. Uniform gating would have meant either a meaningless confirm on every server start or no confirm on a payload build. *Built (build #4).*
- **D18 — The loop may REASON, but it still never executes, and approval stays per-command.** Build #8 makes the proposer genuinely deeper — working memory (a tried/failed ledger), hypothesis-first proposals, a scored candidate frontier, failure diagnosis, a refute-first critic, domain-specialist routing, fingerprint-keyed retrieval, and a model-tier config lever. Every one of those produces **proposals and rationale only**. The decision recorded here is the boundary they were built against and re-locked to: `orchestrator.py` and the whole new `reasoning/` package have **no path to any execution surface**, the frontier holds untried leads rather than a queue that fires, and there is **no auto-approve / batch-approve / run-the-chain** anywhere — a human still approves every single command. Deeper proposer, identical autonomy. *Built (build #8), regression-locked by an extension of the loop's source-scan onto the whole reasoning package.*
- **D19 — The launcher lives ON the home page, not on its own route.** Roughly a dozen surfaces were built and then invisible — `:arsenal`, `:c2`, `:tunnels`, `:windows`, `:repeater`, `:scripts`, `:code-scan` and the AD graph were reachable only by typing the URL, and the operator's own notes had flagged it twice before it was fixed. Both placements were built as mocks and compared. The deciding argument was that *a page you have to navigate to fixes discoverability only if you remember to navigate to it* — so the index belongs on the screen you already land on. Cost measured before committing: +1,024px on a page already ~1,936px, i.e. ~2 screens to ~3. The hero, the stat counters and the intro are untouched. *Built (build #11).*
- **D20 — Operator identity is configuration, never a constant.** This repo is public, and a real name, email or OSID written into source is in the public git history permanently. Identity therefore lives in a gitignored `backend/operator.json` with env overrides, read through two deliberately separate accessors: `public_profile()` (name + handle) for the browser, `report_identity()` (adds OSID / handle / contact) for a report handed to an examiner. A page has no use for an OSID, so it never receives one. The report block is *spliced by code*, never prompted for — handing the model a real name or an OSID invites it to transcribe or invent one. *Built (build #11).*
- **D21 — A source is DISTILLED into the KB, never parroted into it, and a source that teaches something wrongly is rewritten or dropped.** Set while folding in the first external corpus of live-hunting transcripts. Three rules came out of it and now govern every enrichment batch. (a) **Check before writing:** every candidate technique is grepped against the whole KB first, and anything already covered is skipped and reported as skipped — that batch added 13 entries while deliberately re-ingesting nothing of the 24 existing Java-deserialization, 19 cache-poisoning, 11 open-redirect/OAuth or 3 mass-assignment rows. (b) **Zero is a valid result:** a source that yields nothing is reported as yielding nothing, because a batch padded to look productive is worse than a small one. (c) **Correctness outranks fidelity to the source:** the corpus taught "price tampering" while conflating it with bounty manipulation and discussing selling exploits instead of reporting them; the concept was kept and rewritten from scratch, and that framing was not carried across. *Applied (KB enrichment, 2026-07-31).*
- **D22 — A command is gated on what it RUNS, not on what it is named.** Set while cataloguing the pivot tools. The danger heuristic classified `argv[0]` and stopped there, so `weevely` demanded the red-confirm and `proxychains -q weevely …` demanded nothing — while `cockpit/tunnels.py` builds exactly that argv for the operator to approve. The consequence was precisely inverted: **routing a command through a tunnel made its**

**gate weaker, so the further into a network you reached, the less the confirm applied.** The rule now is that a wrapper is peeled off and the command that actually executes is what gets classified, with the wrapper named in the reason so the warning still matches the argv on screen. The corollary is the part worth keeping: a catalogued entry is classified by **what it does**, which is why `proxychains` is clean (it routes; it carries nothing) while `sshuttle` sits beside chisel and ligolo (it needs no agent and no listener, which makes it the quietest way to put a subnet inside reach — not the least dangerous one). *Built (KB repo batch, 2026-08-01), regression-locked through `validate_request` rather than through the predicate, with both halves asserted: the wrapped dangerous command must fire, and a wrapped benign one must stay silent.*

- **D23 — 0xdf is drawn from under the project-wide distil-not-parrot rule; the standing ban on it was reviewed and reversed.** When the fingerprint corpus was first scoped, `ingest_exploitation_writeups.py` carried a hard sourcing line naming 0xdf, IppSec and individual HTB walkthroughs as sources *never to draw from* — and the batch declined the ingest on that line. The operator reviewed that position and reversed it: a third-party writeup is treated exactly like every other source this KB absorbs (D21) — **you learn the technique from it and write an original entry, crediting the source by URL in `references`; you never reproduce its prose, structure, screenshots or phrasing.** The reversal is the decision; the honest record is that 0xdf was declined first and that position did not survive contact with the rule already governing every other source. Two things were changed and one was deliberately left alone. The docstring that forbade what the pipeline now does was rewritten to state distil-not-parrot and the attribution requirement, because a comment contradicting practice is worse than either policy. The link-index skip at `consolidate.py:2324` — which drops a 0xdf *link-list* file as "no technique" — was **left untouched**, because a list of links carries no technique regardless of sourcing policy, the same reason `awesome-oscp` yields nothing. *Applied (0xdf fingerprint batch, 2026-08-01).*

- **D24 — a shared version predicate names its boundary convention EXPLICITLY, because two subsystems need opposite ones.** `_version_verdict` in the CVE→exploit index is the single comparison both the exploit lookup and the fingerprint corpus (2.7 retrieval) key on. They disagree on what the stored endpoint *means*: the CVE index stores `versions[-1]` as the **fix** version (first patched → exclusive `<`), while the fingerprint corpus stores the **last vulnerable** version (→ inclusive `<=`, which is what the `lte` kind name literally says). One predicate, an unstated convention, and nothing asserting the obvious — so **35 of the 38 versioned fingerprints silently failed to match the very version they were written about**, the most precise hit there is. The decision, and the reason it is a D-entry: the fix is the predicate, not either caller (per `backend/AGENTS.md`). `_version_verdict` took an explicit `inclusive` argument, and **every** caller now states its convention — `search_service` and `critic` pass `inclusive=False`, the fingerprint matcher passes `inclusive=True` — so neither relies on a default. Option (b) (rewrite the 35 corpus entries to store fix versions) was rejected: it is smaller but leaves the convention implicit, and the next entry author gets it wrong again. The CVE→exploit index is provably untouched — all **110,695 verdicts across 47,108 entries byte-identical** before/after, `test_exploits.py` green. A second defect rode alongside: `fingerprint()`'s first-token heuristic collapsed `Apache Tomcat` and `Apache httpd` onto the same `apache/<ver>` key, so two unrelated products were told apart only by version — a *confidently wrong* hit, not a missing one; it now resolves the product (`tomcat`) not the vendor. Both are regression-locked by tests that iterate the real corpus and carry a positive control (`test_fingerprint_versions.py`, `test_fingerprint_norm.py`), lifting the covered-banner hit rate 70%→93% at 96% precision with near-miss false-fire held at 0%. **This is the THIRD shared-predicate defect in the project's history** — build #5's WinRM `argv[0]` classification and D22's proxychains red-confirm laundering were the first two — and the pattern is now worth watching for by name: a guard or verdict shared by two callers with an unstated convention, sitting behind no test that pins the obvious. *Built (fingerprint fix, 2026-08-01).*

- **D25 — `fingerprint_match` is reserved for a STRUCTURED match; a bare product-name hit is a distinct, weaker signal.** The last measured gap in 2.7 retrieval was a **20% false-fire on services the corpus does not cover** ( `pure` →Pure-FTPd, `node.js` →Node.js, `minio` →MinIO), stable across the corpus's growth. Its cause was a *second* code path from the structured matcher: the version-less substring fallback in `rerank()` set `fingerprint_match=True` whenever the scanned product merely appeared as a substring anywhere in a plain entry — so an entry that had nothing to do with the service was surfaced with the same confidence as a real fingerprint, and the grounding line claimed "this exact stack was solved by X" unearned. The decision was made from measurement, not taste: the eval was first instrumented to split covered fires into structured vs fallback, and it showed the fallback contributes **0 of the 28 covered fires** while being the **whole** of the false-fire — so this was never a stricter-matcher-vs-hit-rate trade-off, and the fallback could be demoted at no cost. `fingerprint_match` now means a structured `meta.fingerprint` match only; an unstructured product-name hit becomes a labelled `fallback_match` that still outranks a pure token match (a version-less scan keeps some signal, which is legitimate) but never claims the exact stack, and the substring was tightened to a word boundary. The fallback was **not deleted** — the instruction and the measurement agreed to keep and label it rather than drive a number to zero by removing real behaviour. Result: UNCOVERED false-fire **20%→0%** with covered hit rate, precision, near-miss and self-match all unchanged. Regression-locked by `test_fingerprint_fallback.py` (real corpus + 15 real uncovered services + positive control), the last of the retrieval-path correctness gaps to close. *Built (residual fix, 2026-08-01).*
- **D26 — a corpus CI needs is DERIVED from the real one and proven equivalent; it is never authored.** Set while wiring the first CI. Three fingerprint locks iterate the live KB deliberately ( `backend/AGENTS.md` §1: draw from the real population, so a fingerprint added tomorrow is covered without anyone editing a test) — but `/data/` is gitignored, so on a clean checkout they did not skip, they **crashed**. The obvious fix, a small hand-written sample KB, is precisely what §1 forbids and for a documented reason: the gate audit's worst finding was a test asserting on a synthetic value the real system never produces, which stayed green while eight dangerous tools passed the danger gate. The decision is the third option. One measurement made it available: `retrieval._entry_blob` reads `title, text, body, summary, tags, product`, and the live KB carries **no `text`, `body` or `product` field at all** (it stores `body_md` and `steps`, which that function never consults) — so the fingerprint path only ever sees title + summary + tags. The committed fixture is therefore the **complete** corpus — all 2,743 entries, all 105 structured fingerprints — projected onto the fields the matcher provably reads: 22 MB becomes 1.1 MB with **zero** information loss for these tests. It is not a sample and answers to nobody's judgement about what was "representative". The claim is checked rather than asserted: `test_kb_fixture.py` re-derives the fixture from the live KB and requires byte-identity (staleness), compares **20 probes × 2,743 entries** verdict-for-verdict between the two corpora (equivalence), reads the matcher's field tuple **out of `retrieval.py` by AST** so teaching `_entry_blob` a new field fails the test instead of silently blinding the fixture (faithfulness), and carries a positive control that a truncated fixture is caught. The two checks that need the live KB report **NOT-RUN**, loudly, in both the test output and the CI job summary — the `test_proof_honesty` discipline applied to CI itself, because a green badge that quietly means "and four checks were skipped" is the same defect as an unfilled proof slot scoring as a pass. *Built (build #12, 2026-08-03).*

## Phase 1 — reach a real target

*Commits `e4e8de5`, `80a18d5`, `2d0928f`.*

- **New sandbox image** ( `docker/Dockerfile.sandbox` ) on `kalilinux/kali-rolling` + `kali-linux-headless`, with SecLists, the Kali `wordlists` set (rockyou ungzipped) and **~13,400 nuclei templates** baked in (the lab has no egress, so nothing can be fetched at runtime). Thematic tool layers for the specific tools the assessment found missing — impacket, netexec, evil-winrm, bloodhound.py, certipy-ad, bloodyad, responder, mitm6, smbmap, enum4linux-ng, gobuster/nikto/feroxbuster/subfinder/httpx/amass, metasploit, exploitdb, hashcat/john, chisel/ligolo-ng/proxychains/socat/ncat, openvpn — plus katana/kerbrute/waybackurls/dalfox/gau/rustscan/jwt_tool as pinned release binaries (no Kali package). **Result: ~63 of 73 catalogued tools present.** Kali's setcap'd `nmap` / `fping` are stripped at build so `no-new-privileges` doesn't refuse to exec them.

- **Privilege split** ( `docker-compose.yml` ) — the **engage** sandbox runs `user: root` with Docker's default capability set + `NET_ADMIN` and `/dev/net/tun` , so `-sS` / `-sU` / `-O` , masscan, tcpdump, privileged-port binds (responder/ntlmrelayx) and VPN all work. **Lab and `:kali` are untouched** — uid 1000, `cap_drop: ALL` , no devices. Verified: engage does SYN scans and binds :445; lab/ `:kali` refuse `-sS` .
- **Per-request timeouts** (default 180s, ceiling 3600s) on `/cockpit/exec` and `:kali` ; **background/detached jobs** returning a run_id and a reconnectable replay stream ( `/cockpit/runs/{id}/stream` ), with a reaper. `:kali` 's old hardcoded 60s is gone.
- **Loot volume** — `backend/data/engagements` → `/loot` on the engage and `:kali` sandboxes (deliberately **not** the isolated lab); each engagement works in `/loot/<id>` via `docker exec -w` , so `nmap -oA` output survives `docker compose down` .
- **Startup reconciliation** ( `GET /tools` ) — probes the running sandbox with one `command -v` sweep over every catalogued name+alias, filters the planner's prompt to installed tools, and surfaces the gaps in the arsenal UI. Windows-only tools (rubeus/powerview/mimikatz/winpeas) are marked `platform: windows` , kept for planning/write-ups, and never proposed.
- **Arsenal catalog** reconciled against the shipped image; the orchestrator prompt no longer hardcodes `gobuster` / `nikto` .

## Phase 2 — make it remember

*Commits `c606f6d` (backend), `a8f40ae` (UI).*

- `backend/state/` **package** — the structured engagement state the assessment identified as the deepest gap: `hosts → services` , `endpoints` , `credentials` , `findings` as dataclasses over a shared SQLite store where **every write is an upsert** (re-running a scan corrects, never duplicates). Output **parsers** (nmap XML, httpx/ffuf/nuclei JSON, secretsdump) as pure functions, and a post-run **ingest** that reads both a run's stdout *and* any file it wrote into its loot directory (that is how `nmap -oA` populates state). The package **executes nothing** — AST-asserted, because ingest runs automatically after every command.
- **Pentest Task Tree** ( `state/tasks.py` ) — PentestGPT's central idea (§4.3): layered `1 / 1.1 / 1.1.1` tasks with `todo | done | n/a` , persistent per session, updated as results arrive. The model returns **validated operations** ( `add_subtask` / `mark_done` / `mark_na` / …) that code applies against the stored tree; one bad op is rejected on its own and the rest still apply.
- **Orchestrator grounding** — the proposer prompt now leads with the accumulated state + live task tree, with raw output demoted to a short tail. Measured: **460 chars of state vs 6,600 of stdout tails** for 12 runs of one nmap, and each fact appears once instead of once per run. A tail window forgets; state does not.
- **Credentials in the prompt: real values** (Zaid's explicit decision) so the planner writes fully-formed commands — one constant ( `render.INCLUDE_CREDENTIAL_SECRETS` ) with a test proving the flip works, so the choice stays reversible.
- **UI** — the `CockpitState` panel: counters, the task tree (with "seed from plan"), hosts+services grouped, endpoints with status colours, credentials masked with per-row reveal + validated/untested, findings by

severity.

- **Bonus fix found by the browser pass** — the arsenal `/arsenal` response model was silently dropping `platform` / `runs_here`, so every tool badged "windows only". Fixed + regression-tested.

## Phase 3 — make it fast to drive

*Commit `b316b74`.*

- **Step 11 — one-keystroke approve.** The guided loop takes `Enter` = approve, `S` = skip, `Esc` = stop, hints shown on the buttons. A dangerous proposal never fires on `Enter` (the explicit danger confirm is still required); shortcuts are ignored while a text field is focused.
- **Step 12 — `_looks_like_host()` fixed.** Flipped from "assume host unless the last segment is a known file extension" to a **positive test**: a dotted token is a host only if its last label is a plausible alphabetic TLD and not a file extension. Stops the false rejections (`directory-list-2.3-medium`, `Mozilla/5.0`, `-oA scan.1.2`, version strings) while still catching real hosts; a genuine off-target host is still refused. This was the prerequisite for any future auto-run tier.
- **Step 13 — persistent `:kali` shell.** One long-lived `docker exec -i sh` instead of a fresh exec per command; each command is delimited over the pipe with a per-command sentinel so output and exit code read back cleanly. `cd`/env/background jobs persist (verified in-browser: `cd /tmp` → `export FOO=…` → `echo cwd=$(pwd) foo=$FOO` returned `cwd=/tmp foo=…` across three separate API calls). Every `:kali` containment invariant intact. No PTY, by design (§1.1). (Caught the Windows `\n` → `\r\n` stdin-translation bug in the process — the Linux shell was receiving `pwd\r`; fixed with bare-LF stdin.)
- **Step 14 — credential vault.** Captured credentials fill the `<user>` / `<password>` / `<ntlm-hash>` / `<domain>` placeholders the AD templates carry, one click instead of retyping a hash. The placeholder→field mapping lives server-side (`state/credvault.py`) so front and back can't drift; a password fills `<password>` not `<hash>` and vice-versa; only credential placeholders are touched. Verified in-browser: "use svc_sql" turned `nmap -u <user> -p <password> …` into `nmap -u svc_sql -p S3cr3t! …`.

## Phase 4 — content & goal-specific

*Commits `90fe260`, `885d776`, `bf46695`, `8486c56`, `937c420`.*

- **Item 1 — KB Route B additive ingest (D11/D12).** `pipeline/ingest_corpora.py` — a targeted ingester over only the missing files, run *additively* on the built KB (never through the real pipeline, which would revert downstream enrichment and rewrite a 15 MB gitignored, Defender-sensitive file). Two regression-locked guarantees: existing entries pass through as **bytes** (no key/escape drift on the 1,601 it never looked at), and every line it owns carries `meta.corpus_ingest` so a re-run is byte-identical. Shapes (§4.4): **64** **`payload-set`** + **2** **`dork-list`** entries, all `no_merge` so consolidation can't collapse a corpus into a technique page; each holds a capped excerpt while the full list is a **sidecar** under `data/kb/payloads/`, mounted read-only at `/payloads` in all three sandboxes so ffuf/wfuzz point straight at it. **56** **`oscp_tools`** files went to the Scripts Arsenal as file-backed rows (a path to copy, not 20,000 lines of PowerView), 38 Windows-only ones kept and marked `runs_here=false` (D9). Two env traps handled: rglob's OneDrive omission (→ union with `git ls-files`) and AV-locked files (→ git-blob recovery). KB 1,601 → 1,667; arsenal 1,028 → 1,137. The Defender exclusion on `HackPit\data` was added first.
- **Item 2 — evasion/OPSEC as an additive second channel (D10).** The blue-team detection view is **byte-for-byte unchanged** and still passes `assert_describes_not_prescribes` (a test proves every blue field is identical with

and without the offensive half attached, and that the default `footprint()` carries no `opsec` key — so tagging/reports/run-annotation are untouched). `detection/catalog.py` gains a curated `OPSEC` table (10 notes keyed by spec key: what makes it loud, the quieter tradecraft, the tradeoff, and — mandatory — `still_recorded`, the honesty marker). `resolver.py` gains an opt-in `include_opsec` channel with its **own** guard (`assert_opsec_is_separate`): a note that fabricates the honesty marker or advises disabling/clearing/tampering with a sensor is rejected. `report.py` gains an off-by-default red-team OPSEC roll-up. The panel renders it as a distinct amber section. LLM may extend to uncatalogued commands, `ai_suggested`.

- **Item 3 — HTTP repeater.** `cockpit/repeater.py` mirrors `:kali` containment (hardcoded open container, human-only + source-scan locked, audited) and adds a scope check `:kali` lacks. **Argv-only curl**, never a shell — method/headers/URL are discrete tokens, the body rides on stdin, so a header of `a; rm -rf /` is one literal `-H`. A human clicking Send *is* the approval (no per-send prompt); the orchestrator/agent have zero path to `send`. When the send names an active engagement the URL host is scope-checked (out-of-scope → 403, nothing sent). Frontend `/repeater`: compose/send, response view, per-session history with edit-to-resend and a line-level body diff. The VPS-for-callbacks piece stays its own decision (D2).
- **Item 4 — pivot/tunnel routing.** `cockpit/tunnels.py` — chisel/ligolo-ng lifecycle (start the listener in the engage sandbox, hand back the agent one-liner to paste on the compromised host; human-only start/stop, source-scan locked). `route_for` / `wrap_command` are **pure** — they compute a **visible** `proxychains -q` prefix (chisel/SOCKS) or leave the command unwrapped with a route note (ligolo/interface), applied *before* the approval screen so the human approves the exact argv (silent routing was rejected for exactly this reason). A tunnel's subnet enters scope only via `engagement.add_pivot_subnet` — the one deliberate, audited widening path; recon expansion still cannot widen. Frontend `/tunnels`: start form, live tunnels with the copy-ready one-liner + per-subnet "add to scope (by hand)", and a pure route preview.
- **Item 5 — exam-mode report templates + proof.txt as per-host state.** `state` `Host` gains `local_txt` / `proof_txt` + `ownership()`; captured automatically when a command reads a flag file (`proof.txt` / `root.txt` → proof; `local.txt` / `user.txt` → local — a stray 32-hex hash is never mistaken for a flag) or pasted via `POST /sessions/{id}/state/proof`; the state panel badges foothold vs owned. `report.py` gains four templates — standard, **OSCP** (per-host walkthrough + a proof.txt table spliced from state at `{{PROOF_TABLE}}`, computed not model-written), **CPTS** (exec-summary + findings register), **H1/Bugcrowd** (impact-first + a **CVSS 3.1** base score computed by `cvss31_base`, matching the official calculator). Every template keeps the shared grounding rules + the `{{EVIDENCE}}` splice. Frontend: a template picker on the report screen.

## Phase 5 — the PTY gap, and finishing the KB

*Commits `e8eeeef`, `8f0b91b` (terminal), `84e31bf` (CVE index), `f6fb6a4` (KB batches).*

- **Item 1 — a real PTY, as a SECOND surface.** §1.1 used to call this a trade: readable transcripts *or* full-screen tooling. It is not, if they are separate surfaces. `:kali` is untouched — still sentinel-delimited over a plain pipe, still the clean per-command transcripts reports are built from — and `cockpit/terminal.py` adds `:terminal` beside it. `docker exec -t` refuses without a client TTY and a WebSocket handler has none, so the pty is allocated **inside** the container by a constant driver that `pty.fork()`s bash and multiplexes it over the pipes. Its stdin is **framed** (`0x00` = keystrokes, `0x01` = `COLSxROWS` → `ioctl TIOCSWINSZ`), which is what makes resize possible without an in-band escape a keystroke could forge; its stdout is the raw pty stream, straight to xterm.js. Containment mirrors `:kali` point for point: hardcoded container (the request carries `session_id` / `cols` / `rows` and nothing else), driver a raw-string constant passed as one argv element, no isolation gate, **human-only and source-scan locked** like `run_kali`, and the raw stream audited to the run store — capped, and checkpointed every 30s so a crash still leaves a transcript. The WS route pins `Origin` by hand, because CORS middleware does not cover WebSockets. `test_terminal.py` (16 checks) locks all of it *plus the additive property itself*: `kali.py` must still carry its sentinel and must never grow a pty, so the clean transcript cannot quietly die. Verified live — `tty` reports `/dev/pts/0`, `top` and `vim` render, resize propagates 30×100 → 43×132 → 60×200.

- **Item 2 — CVE → exploit index (§3, finding 6).** The OSCP inner loop, built as a **keyed lookup rather than another KB batch**: "find the exploit for *this* version" wants an exact table, not prose retrieval, so it gets its own data shape, search path and surface. `pipeline/ingest_exploitdb.py` reads `/usr/share/exploitdb/files_exploits.csv` out of the sandbox image — 47,108 exploits, 27,384 with a CVE, 25,041 distinct — nothing fetched from the internet. exploit-db titles follow `<Product> <Version> - <Vuln>` on ~100% of rows, so parsing the left side turns a flat catalogue into `(product, version) → exploits` with the constraint typed (exact / lte / range / wildcard / none); 32,807 rows yield a version. `backend/exploits/` then does what `searchsploit`'s substring match cannot: `2.4.49` satisfies `< 2.4.50`, sits inside a range, and matches the `2.4.x` line — each with a *different stated verdict*, and the verdict is the ranking's primary key, with token similarity only breaking ties inside a tier. (Blending them let `Apache 2.4.x` outrank the entry naming 2.4.49 exactly — caught on real data, now regression-locked.) Surfaces: `/exploits`, plus a per-service `exploits →` link in the state panel, so a fingerprinted service reaches its exploits in one click. Every hit names the file **already inside the sandbox**. It executes nothing, and `test_exploits.py` asserts the package contains no subprocess path and never reaches the execution layer — finding is not running.
- **Item 3 — PortSwigger Web Security Academy (+372).** `pipeline/fetch_portswigger.py` turns the site into the same shape every other source has, so nothing downstream is special-cased. URLs come from the publisher's own `sitemap.xml` rather than from crawling — it is complete (the all-topics page renders client-side, so scraping it yields 19 of ~140) and nothing is ever guessed; `robots.txt` restricts only `/bappstore/bapps/download/`. 398 pages, 273 of them labs, whose *Solution* steps survive. Two rules written by what dry runs actually did: **labs and sub-topic pages are** `no_merge` (the first run folded ten Academy pages — Reflected/Stored/DOM XSS, contexts, CSP, dangling markup — into one pre-existing grab-bag called "xss resource", so "Reflected XSS" stopped being retrievable as itself; only a topic *root* may consolidate, which the Academy expresses as URL depth, taking 90 merges down to 26); and the two cheat sheets use `<section>` not `<main>`, so without a fallback they were 2 of 18 pages silently extracting zero bytes.
- **Item 4 — HackTricks Cloud (+578), and the cloud gap closed.** Same GitBook layout and adapter as the HackTricks book already ingested. Two corrections, both from dry runs: it is **not class-grouped** (grouping collapsed 44 distinct CI/CD pages — Jenkins RCE, GH Actions cache poisoning, Okta, Cloudflare — into one candidate), and `CANON` gains only **technique-level** cloud classes. "kubernetes", "ci-cd" and "supply-chain" name a *platform*, not a technique, and `CANON` is the authority on what consolidates; including them collapsed every "Kubernetes X" page into one entry. Their absence is now documented in `CANON` so they are not re-added. **cloud 2 → 535, supply-chain 1 → 47.**
- **Item 5 — the HTB Academy slice, audited (§3, finding 4).** Auditing what the local folder holds against what had been ingested found two files silently lost, both to one root cause — bare `rglob` over `*.md`. `File_Inclusion/README.md` (4.5 KB of LFI module content) is tracked in git but **gone from disk**, the dehydration/quarantine pattern this repo has hit before — and that module is full of web-shell examples, which is exactly what trips the signature. `File_Upload_Attacks/1.txt` (8 KB of module answers and webshell walkthroughs) was lost purely for its extension: §4.1's headline finding in miniature. Now `_all_md` (rglob ∪ `git ls-files`, with git recovery) plus `.txt`.

**KB 1,667 → 2,617**, 0 malformed rows, embeddings rebuilt incrementally, scripts index rebuilt, `entries.jsonl` verified present and counted after every pass (the Defender-quarantine trap).

## Windows execution backend — the WinRM driver (shipped 2026-07-26)

*Commits `72b0959` (transport + profiles + `windows` mode), `cf64b8f` (profile CRUD/picker + per-target `/tools`), `d8854dc` (native Windows abuse variants + AD live), `1ce53ce` (frontend), the VM guide, and this doc.* Closes D9 the honest way. See `docs/WINDOWS-EXECUTION.md` + `docs/WINDOWS-TARGET-SETUP.md`.

- **A new execution transport, behind the same gates.** `docker exec` is swapped for a WinRM call; nothing about the safety model changes. A third mode, `windows` (alongside `lab` and `engagement`), is selected by `ExecRequest.windows_profile_id`. The target is the profile's **host — hardcoded server-side, never a request field** (the same containment shape `:kali` gets from its hardcoded container: a command physically cannot reach a box you did not pick). Gates: `windows` (profile exists) → `target` (host in the engagement scope, if one is also named) → **never-auto-run** approval → danger red-confirm. No isolation gate — it is a real external box, like engagement.
- **Model A — HackPit drives, it does not own.** No Windows VM lives inside the project. The operator runs a Windows/AD VM in VMware Workstation; HackPit opens a WinRM session and runs one PowerShell command string on the box (Rubeus/PowerView/Mimikatz/.NET all run *there*). `pywinrm` is **lazy-imported** so the hermetic suite needs no dependency and no network; the AD-live path is unit-tested with a **mocked** transport.
- **Saved connection profiles** (`cockpit/winprofiles.py`) — a "Windows targets" store in the gitignored `sessions.db`: name, host, transport (WinRM; SSH a documented later seam), port, username, `password` **or** `ntlm-hash` (pass-the-hash, presented `LM:NT`), domain. The secret is **write-only** — masked to `has_secret` in every view, read only by the transport, never in a response, record or command line. A **captured vault credential can fill a profile**, resolved server-side so the secret never round-trips.
- **The AD graph executes live.** Every abusable edge gained a **native Windows variant** (PowerView/Rubeus/Mimikatz) beside the Linux one; the walk/orchestrator's proposed command runs over WinRM when a Windows target is picked, and `advance` still requires an approved + exit-0 run. The danger heuristic learned the native destructive cmdlets (`Set-DomainUserPassword`, `Add-DomainObjectAcl`, `Set-DomainObjectOwner`, `Add-DomainGroupMember`, `Set-DomainRBCD`, `Invoke-Mimikatz`, …), so a native DCSync/password-reset trips the **same** red confirm as its Linux cousin — the oracle test now checks both transports.
- **Per-target tool reconciliation.** Windows-only tools (Rubeus/PowerView/Mimikatz/winPEAS) report runnable when a Windows target is selected and N/A on a Linux run — reconciled per active target (`/tools?windows_profile_id=…`), never a global flip. Served from `main.py` so the cockpit stays arsenal-blind.
- **Frontend.** A **Windows targets** page (`/windows`) — profile CRUD, masked secrets, connectivity test — and a "run on" picker in the AD walk (Linux sandbox vs a WinRM target); the confirm/flags/command shown follow whichever command will actually run.
- **Safety regression-lock** (`test_winrm_safety.py`): host-locked to the profile / no gate bypass / secrets never leak / **the orchestrator cannot auto-run WinRM** (transport reachable only from the gated executor + the human router probe, source-scanned). Functional path (`test_winrm.py`) uses a mocked transport.

**Deferred to a live VM:** the live/browser verification against a real Windows box, until the operator's VM is up (build + unit tests are hermetic) — the same way tunnels and AD-live execution were deferred.

## Post-assessment refinements (2026-07-27)

Three items from a read-through of this assessment, each done the low-risk way. All query-time / config / ranking changes — **no KB re-ingest, no 15 MB rewrite, no Defender risk.**

- **KB consolidation audited (§4.2) — no re-ingest needed.** A read-only provenance audit confirmed the OSCP "3:1 collapse" is healthy dedup, not loss: 36 command-rich survivors, 12 folded into stronger canonical pages (still attributed), ~37 same-technique OS/lab splits merged, **0 dropped as low-value**. Also corrected the framing — `oscp-cpts-notes` is a tier-3 public GitBook repo, not the author's own writing. Decision: leave the KB untouched (re-adding with `no_merge` would recreate duplicate technique pages that hurt search).
- **Relevance-first KB ranking (§3, tiers).** The composer was already relevance-first (BM25 + cosine RRF; tier a small additive nudge), but a *thin* tier-1 stub still got a flat boost that could edge out richer, more relevant lower-tier content. Fixed in `pipeline/search.py`: the tier-1 boost is now **gated on substance** (a stub with no commands + little body gets nothing), and a **completeness nudge** (command-rich, any tier, saturating/capped) lets content decide close calls. Trust breaks ties among substantive entries; it no longer manufactures relevance. Locked by `test_search_ranking.py`.
- **SAST coverage widened (§ Code scan).** The offline Semgrep bundle grew from 19 rules (Python/JS/TS) to **34 across 8 languages** — Java/Go/PHP/Ruby/C# added (`rules/hackpit-languages.yaml`) — plus a **ruleset picker** (bundled / per-language, or a registry pack for the full online catalogue). Default scan loads the whole offline directory. Locked by `test_codescan_rules.py` (rules well-formed + 8-language coverage + `semgrep --validate`, resolved from the venv the runner uses so it validates rather than skips). The new rules were confirmed **valid** (semgrep `--validate`: 0 errors, 34 rules) and to **fire** on real vulnerable samples (Java/Go/Ruby/C#). One robustness fix fell out of that check: on some Windows builds semgrep itself *crashes* on a `.php` file, and the scan treated semgrep as fatal — so a crash now **degrades to a warning** (the scan still returns with whatever else ran, like bandit), instead of 502-ing the whole scan. *(Engagement export/import was considered and dropped — the existing report generator already covers a shareable dump.)*
- **UI cleanup (Frontend).** Small polish from a working session: the two shell tiles were merged to **one `:kali` tile that opens the real PTY** (`/terminal`) — full-screen tools render from the everyday shell, while the sentinel "transcript shell" (clean per-command records) is preserved off-nav at `/kali`; both backends and their containment tests stay intact. The **phishing** category is now surfaced on the front-page grid (its KB entries already existed; forensics/ics/supply-chain stay hidden). The **top nav** dropped the redundant `:library` tile (home is the wordmark), the search field was widened to a one-line field, the unused accent-swatch picker was removed (amber is the default and only accent), and the nav is **optically centered** (a small left nudge balances it against the heavier bordered search box, so it reads centered rather than geometrically dead-centre). All display-only; no backend touched.

## reconFTW review + incorporation (2026-07-27)

A reconnaissance engine — reconFTW (six2dez, MIT) — was reviewed end to end for anything worth mining. The short version, and what was folded in:

**What it is.** An autonomous external-recon / bug-bounty runner: one Bash orchestrator plus eight modules driven by a config file, chaining ~100 tools end to end (optionally across a cloud fleet) and emitting a consolidated report. Its pipeline is OSINT → subdomain enumeration (passive → certificate transparency → ASN/CIDR → permutations → mass-resolve + wildcard-filter → NOERROR → web-metadata scraping → recursive → reverse-IP) → resolution/scope → host/port scan → web probe + screenshots → URL discovery + `gf` triage + JS mining + param discovery + fuzzing → vuln checks → report.

**Bottom line: mine the knowledge, never the runner.** reconFTW's mechanism — autonomous, fire-and-forget chaining — is the exact opposite of HackPit's one-gated-executor, one-approval-per-command model, so none of the runner, config engine, axiom fleet or monitor/notify machinery is adopted (see Part II's skip list). Its *value* is a battle-tested set of recon tools, proven flag patterns, and a recon ordering — and that lands squarely on HackPit's thinnest area, external attack-surface discovery. So the worthwhile half was folded in as **data and grounding, not a new execution path**:

- **Arsenal (`backend/arsenal/tools.json`) — +28 tools, 73 → 101**, in a new `osint` category (6) plus `recon` (10), `web` (10) and `cloud` (2): the URL-triage set, permutation engines, mass-resolution, JS mining, ASN expansion, external OSINT, bucket enum and the injection completers, each with `<target>`-based invocation templates. `executes_nothing` is unchanged, and the arsenal safety suite (schema + inertness + no template hardcodes a host) passes at **101 tools / 257 templates** — a rendered invocation is still a string until a human approves it through the same executor.
- **KB methodology — two `checklist` entries** (`recon-methodology-*`): the subdomain-enum ordering and the URL→gf-triage→fuzz pipeline, folded in by a new additive ingester (`pipeline/ingest_recon_methodology.py`) that mirrors the corpora discipline — byte-preserving pass-through, its own idempotency marker, re-run byte-identical, and it segregates the corpus block to the tail so `ingest_corpora`'s byte-identity invariant still holds (verified — `test_corpora` green). KB **2,617 → 2,619**.
- **Composer grounding** — an advisory recon-ordering note in the real-target proposer prompt: the sequence as guidance, still one gated command at a time, never a chain.
- **Sandbox image (`docker/Dockerfile.sandbox`)** — the 28 tools installed the way each ships (`go install` for the Go set, venv + PATH wrapper for the Python set, apt for `massdns` / `commix`), with `gf` pattern packs and a static resolver list baked for the no-egress lab. `subwiz` is catalogued but **not baked** — it fetches an ML model at runtime, so it runs only in the egress-enabled sandboxes (engagement / `:kali`), never the airgapped lab; `gotator` / `regulator` cover permutations offline. The ~8–10 GB image was rebuilt and smoke-tested — **all 26 baked tools resolve** in the fresh `hackpit/kali-sandbox:m1` (trufflehog moved to its release-binary installer after a recent release began requiring Go ≥ 1.25; `subwiz` intentionally absent), with the `gf` pattern packs (37) and a 12,956-line resolver list baked in.

The methodology, the arsenal entries, the composer note and the image rebuild are all complete and verified — arsenal suites green at **101 tools / 257 templates**, `test_corpora` byte-identity intact, the full hermetic safety suite green, and the fresh image resolving every baked recon tool.

## Persistence / backdoors (TA0003) enrich (2026-07-27)

The post-exploitation **persistence** layer was reviewed the same way — audit first, add only what is genuinely missing — because HackPit already *executes* persistence on scoped targets through the one gated executor (engagement + WinRM + `:kali`) and `msfconsole` is installed. This is an enrich: **knowledge, catalog and describe layers only — no new execution capability, no persistence engine or autorunner.**

**What the audit found.** The earlier "~60 persistence entries" figure conflated **cloud** persistence with host persistence: of the ~72 persistence-relevant KB entries, **52 are cloud** (IAM / AAD / GCP backdoors) and only **2** were dedicated host-persistence entries. The TA0003 *mechanisms* were present, but only at the command level, scattered through the OSCP/HackTricks corpus (scheduled-task/cron 64 hits, services 59, backdoor accounts 70, web shells 46) — what was missing was the **organised, per-mechanism methodology** tying them together, exactly the reconFTW shape (the raw material existed; the map did not). On the detection side, `attck.py` carried almost none of the persistence technique rows.

- **KB methodology — two `persistence` checklist entries** (`persistence-methodology-windows`, `persistence-methodology-linux`): a TA0003 mechanism map per OS — Registry Run keys, Startup folder, scheduled tasks, services, WMI subscriptions, accessibility/IFEO, backdoor accounts and web shells on Windows; cron, systemd units/timers, SSH `authorized_keys`, shell-init profiles and backdoor accounts on Linux — each mechanism a single gated command plus the footprint it leaves, cross-referenced to the detection panel. Folded in by a new additive ingester (`pipeline/ingest_persistence_methodology.py`) mirroring the recon-methodology discipline (own idempotency marker, byte-preserving pass-through, corpus block segregated to the tail, re-run byte-identical). KB **2,619 → 2,621**; `test_corpora` byte-identity intact. Rootkits and bootkits are named as knowledge only, never tooling.

- **Arsenal (`backend/arsenal/tools.json`) — +4 tools, 101 → 105**, in a new `persistence` category: **SharPersist** (`platform:windows`, add/remove templates, mirroring `rubeus`) plus **Sliver**, **Empire** and **weevely** as **reference-only** — catalogued but not installed, so reconcile reports them not-present and the planner will not propose them (the same treatment as `subwiz`). Their install and C2 wiring are **deferred to build #4**. `executes_nothing` unchanged; arsenal suites green at **105 tools / 262 templates**.
- **Detection footprint (`backend/detection/`) — 13 `FootprintSpec`s + 11 ATT&CK rows.** `attck.py` gained the persistence technique rows (T1547.001, T1543.003/.002, T1053.003/.006, T1546.003/.004/.008, T1136.001, T1505.003, T1098.004) — additive, transcribed from ATT&CK v19.1 and trimmed to the Windows/Linux/network log-source subset, verified against live upstream. `catalog.py` gained a describe-side footprint per mechanism (Event IDs / Sysmon / Autoruns / auditd, loudness, and a `why_rating` that carries the honesty marker — *quiet = a defender coverage gap, not an operator advantage*). This is the **blue/describe half only**: no OPSEC/evasion notes were added (that channel is build #4), so the `assert_opsec_is_separate` and never-prescribe guards are untouched. Aliases were added for unambiguous binaries only (`schtasks`, `crontab`, `useradd`, `sharpersist`, `weevely`); multi-purpose `reg` / `sc` / `net` / `systemctl` stay on the argv path so a `reg query` is never mislabelled persistence.

All three landed additively and verified — the full hermetic safety suite green (`test_detection`, `test_detection_safety`, `test_arsenal`, `test_arsenal_safety`, `test_corpora`), and `pipeline/detection_sources.py --verify` clean against live ATT&CK v19.1 + SigmaHQ. No image rebuild (OS built-ins and the installed `msfconsole` cover the mechanisms), and no new execution path — the gated executor already ran these commands, one human approval at a time.

## AV/EDR evasion + traffic obfuscation (2026-07-27)

Build #4 closes the C2/evasion channel the persistence enrich deferred. Unlike that enrich, this one **does add execution capability** — a Sliver C2 surface, DNS-tunnel listeners, and a generate-only evasion engine — so the whole design is about making the new surfaces inherit the existing containment rather than sit beside it. It also makes a **deliberate policy change**, recorded as D16 below.

**D16 — the OPSEC channel may now prescribe evasion.** Until this build, `assert_opsec_is_separate` refused any sensor-blinding phrasing outright: the channel could say "rate-limit and add jitter" but not "patch AMSI". That ban is **lifted entirely and deliberately**. The reason is coherence: this build ships an engine that *emits* AMSI-patch and ETW-blind artifacts, and a tool that produces an artifact it is forbidden to describe is worse than one that describes it honestly. **Two invariants survive and are now the whole contract:** every OPSEC note must carry a substantive `still_recorded` naming what catches the technique anyway, and the blue-view footprint is always produced alongside and is never suppressible. What did **not** change: the blue-side describe-never-prescribe guard (`_evasion_prescription`) is untouched, so the defender's copy is byte-identical to before; and the four execution gates — human approval per command, scope lock, red-confirm, audit — are untouched. This is a **posture shift on a public repo** and is stated plainly rather than buried: HackPit now documents in-process sensor tradecraft, always paired with its detections.

- **Sliver C2 ( `backend/cockpit/sliver.py` ) — a split-gated surface.** Server lifecycle (start/stop/list) is **human-only** with no red-confirm, mirroring `tunnels.py` : it is operator infrastructure on the operator's own sandbox, so clicking start *is* the approval. Implant **generation** is a **gated command**, mirroring `session.py` : it builds an `ExecRequest` and runs the real `executor.validate_request` — approval, scope, red-confirm — before anything is produced. It only generates; there is no delivery or execution route, and live beacon catch is deferred. `<listener>` is the operator's callback address and passes through verbatim, never target-substituted. A defect found while building it: `allowlist._FRAMEWORKS` held `"sliver"` , which never matches `sliver-client` , so an implant build would have passed the danger gate with **no red-confirm at all**; the real binary names were added.
- **DNS-tunnel obfuscation ( `backend/cockpit/obfuscation.py` ) — dnscat2 / iodine listeners**, entirely human-only. The far-side client half is returned as a **string for the operator to carry across by hand**; the module has no delivery primitive and must never gain one. The tunnel's pre-shared key is masked **at source** — `client_command` is built from a masked copy, so the real key never occupies a field that crosses the HTTP boundary. That mattered: the obvious fix ( `model_dump(exclude={"secret"})` ) still leaks, because the key is embedded verbatim inside the one-liner.
- **Bespoke evasion engine ( `backend/evasion/` ) — generate-only, with forced honesty.** A top-level package (peer to `detection/` and `state/` , per the cockpit/arsenal decoupling rule). It runs donut/ScareCrow as argv-only `docker exec` into a container resolved from the execution mode — never a request field — and **never runs or deploys what it builds**: the produced artifact is never `argv[0]` , and no deploy primitive exists in the package. **The honest half is computed *before* anything is built**: if the engine cannot resolve the blue-view footprint for a technique it refuses outright rather than emit a footprint-less artifact. Both the footprint and the `still_recorded` note are required fields on the result model, so no route can return one without the other, and the two PowerShell stubs carry the same honesty header *inside the artifact* so it survives being copied out of HackPit.
- **Detection ( `backend/detection/` ) — 7 new footprint specs and 20 new OPSEC notes.** TA0011 gained `c2_dns_tunnel` , `c2_malleable_profile` , `c2_jitter_beacon` and `c2_domain_fronting` (describe-only; domain fronting is documented as largely dead); TA0005/TA0112 gained `evasion_packed_loader` , `evasion_amsi_patch` and `evasion_etw_blind` , which are what the engine maps onto. The 13 persistence specs from build #3 were backfilled with per-mechanism OPSEC notes — 7 new notes alongside the new specs plus those 13 is where the 20 comes from. **SPECS 46 → 53, OPSEC notes 10 → 30** (both re-counted against the live catalog; an earlier draft of this bullet said "4 OPSEC notes", which contradicted its own 10 → 30). Two accuracy corrections were made under review: T1029 had been filed under TA0011 when *Scheduled Transfer* is TA0010 Exfiltration (it was dropped rather than retagged, since an Exfiltration label on a beaconing footprint reads as wrong), and the telemetry strings on five new rows had been written rather than transcribed — replaced with real ATT&CK v19.1 values, `--verify` clean against live upstream. No new ATT&CK rows were needed for the evasion specs: under v19 naming T1562.001 **and** T1562.006 (Indicator Blocking) *both* revoke into **T1685**, which was already present. An earlier version of this claim paired T1562.006 with T1690 and mapped the ETW footprint onto it — T1690 is the successor of T1562.003 (*Impair Command History Logging*) and covers shell history, not sensors. The Tasks 8–13 review caught it; the spec now cites T1685. `--verify` did **not** catch it and could not have: it checks that a cited id's name, tactic and log sources match upstream, not that the technique is the right one for the activity.
- **Arsenal — 105 → 110 tools**, new `c2` and `evasion` categories; Sliver moved out of `persistence` into `c2` . `<tunnel-zone>` and `<tunnel-net>` are new placeholders rather than reuses of `<domain>` , because the composer's target substitution rewrites any dotted token — spelling them `<domain>` would have pointed an operator's tunnel at the system under test. A latent data bug was found and fixed here: `placeholders`

declared `"<payload>"` **twice**, so every `json.load` silently discarded the first description; a duplicate-key check now guards it.

- **Image (`docker/Dockerfile.sandbox`) — Sliver, dnscat2, donut and ScareCrow installed; iodine was already present.** Three of the plan's assumptions did not survive contact with the base image and were caught by the layer's own smoke tests: `dnscat2` and `ScareCrow` are **not packaged** in kali-rolling, and there is **no Go toolchain** in the image, so release binaries and a from-source dnscat2 build were the only options. Two further faults surfaced only because the smoke test *invokes* each tool rather than running `command -v`: dnscat2's server dies on `require 'ecdsa'` (four gems missing), and `donut-shellcode` is a C extension with no console script, so `python -m donut` cannot work and a real CLI had to be written.

**Containment, restated.** No orchestrator, agent or loop module can reach any of the three surfaces — asserted by whole-tree source scans, not by convention. Nothing here is autonomous, no gate was weakened, and the only capability the agent gained is none: every new action is human-initiated and audited.

## Gate-integrity audit (2026-07-27)

After build #4 shipped, the whole project's **guards** were audited with one question: *would this actually fire?* Not "is a guard present" — the build had already shown that a guard can be present, look correct, pass its tests, and never trigger. **24 guards were probed by constructing a deliberate violation for each. Seven silently failed to fire.** Full detail in `docs/GATE-AUDIT-FINDINGS.md`; the three criticals were:

1. **Eight catalogued tools passed the danger gate with no red-confirm** — demonstrated end to end through `executor.validate_request`. `weevely` (which both generates a webshell *and* drives it), dnscat2's client and server, `iodine` / `iodined`, `commix`, `SSTImap`, `SharPersist` and `Invoke-Obfuscation` all returned zero reasons, as did the argument-driven `sqlmap --os-shell`, `commix --os-shell` and `netexec -x`. **FIXED and pushed (`4492fec`).**
2. **The WinRM transport classifies only the first token of what is a whole PowerShell script.** `executor.py` joins command+args and `run_ps()` executes the lot, so `;` and `|` are live separators: `Write-Host go ; Invoke-Mimikatz` is silent, while the same cmdlet as `argv[0]` fires. **FIXED — build #5 (see "Gate integrity — build #5" below).** This was the most consequential finding of the whole review cycle, because it executes on a real domain-joined host under real credentials.
3. **The `:kali` human-only lock never opens 39 of 69 backend modules** — its scan globs only `backend/*.py` and `cockpit/*.py`. A planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` passes. The same narrow glob was copied into the tunnels, repeater, terminal and WinRM locks, and the cockpit→arsenal lock covers 5 of 22 modules. **FIXED — build #5 (see "Gate integrity — build #5" below).**

**What Critical 1's fix actually changed**, because the shape matters more than the list. The `.exe` / `.py` / `.ps1` normalisation that the AD sets already did was **collapsed into one shared normaliser used by every set** — that asymmetry was the root cause, not a symptom, and it was why `powershell.exe`, `nc.exe` and even `sliver-client.exe` produced no reason at all on the transport where `.exe` is the ordinary spelling. Two new sets were added rather than padding `_FRAMEWORKS`, so the reason the operator reads is true: `_TUNNEL_TOOLS` (a tunnel is a C2 path and an exfil path, not a payload generator) and `_RCE_TOOLS`. `_FRAMEWORKS`' `ligolo` entry was corrected to `ligolo-proxy` — the binary the repo actually runs — an entry that could never match, added *after* the `sliver` / `sliver-client` bug was fixed, and worse than no entry because it reads as coverage.

**The finding underneath all three.** `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm — while testing `python3`, **a command that is not in the catalog at all.** That single choice is why eight tools shipped ungated. It now pins all **176** catalogued invocation names (every name, alias and template `argv[0]`) into exactly one bucket, so an unclassified tool fails the suite; five planted regressions were run against it and all five fail correctly. Every critical in this audit, and two of the five findings in the Tasks 8–13 review, reduce to the same root cause: **a test that could not fail, or that tested a value the real system never produces.** That is what build #5's third task turns into a written repo convention.

## Gate integrity — build #5 (2026-07-27)

The audit's two remaining criticals, and the testing pattern that hid them. Plan: `docs/superpowers/plans/2026-07-27-build5-gate-integrity.md`.

**Critical 2 — the WinRM danger gate read the first token of a whole PowerShell script.** `cockpit/executor.py` joins `command` + `args` into one string and the Windows transport hands that string to PowerShell, where `;`, `|` and newlines are live statement separators. The heuristic classified `basename(argv[0])` and scanned only the *args* for markers. So the gate was defeated by moving a cmdlet one token to the right, on the one path that executes against a real domain-joined host under real credentials:

| Script | Before |
|---|---|
| `Write-Host go ; Invoke-Mimikatz -DumpCreds` | no reason, allowed |
| `Get-DomainUser \| Set-DomainUserPassword` | no reason, allowed |
| `IEX (New-Object Net.WebClient).DownloadString(...)` | no reason, allowed |
| `Write-Host go` + newline + `Invoke-Mimikatz` | no reason, allowed |
| `powershell -enc <base64>` | fired **by accident** — `-enc` splits into `-e`, `-n`, `-c` |

All five became tests first, verbatim, and were confirmed failing before anything was written.

**The fix shape, and why it is not a parser.** Four options were weighed; the choice was to **scan the whole joined string for every marker, wherever it appears**, and explicitly *not* attempt statement splitting. Correct PowerShell tokenising has to handle quoting, `$( )` subexpressions, the `&` call operator, backticks and line continuations — and **any parser written here becomes a new bypass surface, which is precisely the bug being fixed: the classifier's model of the input was narrower than the input.** Whole-string scanning has no "position" to exploit, and that is the property the first-token design lacked. The cost asymmetry settles it: this gate raises a **red-confirm, not a block**, so a false positive costs one click while a false negative runs Mimikatz on a domain controller.

Two things a name-based scan cannot see were closed alongside it. **Download cradles** — `IEX (New-Object Net.WebClient).DownloadString(...)` never spells the payload's name anywhere, so the *cradle* is the marker. **Encoded commands** — `-enc` defeats every text scan by construction, so the payload is decoded and re-scanned, matching every PowerShell prefix of the flag rather than only the spellings a human types; a blob that will not decode is itself reported rather than passed.

**The root-cause lock.** The bug was never only "the heuristic is too narrow". The string the gate classified and the string the transport executed were **built in two places**, and scanning a different string than the one that runs would have reproduced the bug somewhere new. Both now derive from `executor.join_ps_command()`, and a test asserts on the source — transitively, with a negative control — that they still do. The AD orchestrator's advisory pre-check was moved onto the same union, so the panel can no longer promise a confirm the gate would not require, or stay quiet where it would.

**The false-positive cost, measured rather than promised** (plan Task 1 Step 6). Run across every AD/Windows template in `tools.json` and every command in the AD graph's edge definitions: **78 of 145 invocations in the full population (53%), and 27 of 43 in the WinRM-reachable subset (62%), now demand a confirm.** The second number is the one that matters — the catalog's Linux impacket invocations go through the docker path, which is untouched. Of those 27, **none is a false positive**: every one is a genuine destructive abuse (Set-DomainUserPassword, Add-DomainObjectAcl, Add-DomainGroupMember, Invoke-Mimikatz, SharPersist, Invoke-Obfuscation, Rubeus ptt, Set-DomainObjectOwner). Read-only enumeration stays silent — Rubeus kerberoast and asreproast, `Get-DomainUser -SPN`, `Find-InterestingDomainAcl`, `Find-LocalAdminAccess`, winPEAS, `Get-ADServiceAccount`, `Get-DomainObject` — and that half is pinned as a control, because a banner on ordinary enumeration is how an operator learns to click through the one that matters. The measurement also found **two real false negatives** the sets had missed: `Invoke-SQLOSCmd` (OS command execution on a SQL host) and `Enter-PSSession` / `New-PSSession` (a further hop out of an existing session). Both were catalogued; both were silent.

Because whole-string scanning over-flags by design, **every reason names the marker it matched and the offset it matched at** — `powershell script: 'invoke-mimikatz' at offset 14 — dumps/ replicates domain credentials`. A reason the operator can evaluate is a gate; a banner they cannot is decoration, and trading a silent bypass for a decorative one is not a fix.

**Critical 3 — eleven source-scan locks, nine of them narrower than their own docstrings.** The `:kali` human-only lock is the entire reason the one deliberately-unbounded arbitrary-shell surface is considered safe. Its docstring said *scan the whole (non-venv) source tree*; it globbed `backend/*.py` plus `backend/cockpit/*.py`, which is **30 of 69 modules**, so all of `adgraph/`, `arsenal/`, `codescan/`, `detection/`, `evasion/`, `exploits/` and `state/` were never opened and a planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` — a literal orchestrator module — shipped green. Three distinct defects, now fixed structurally in one shared `backend/test_support/scans.py` rather than by asking each suite to remember:

1. **Narrow file selection** — an `rglob` over the whole backend. Coverage per lock goes 30 to 67 modules; there is no per-suite glob left to get wrong.
2. **Basename allow-lists** — `{"router.py"}` matched against `f.name` exempted `adgraph/router.py`, `detection/router.py`, `arsenal/router.py` and every other `router.py` in the tree by accident. Allow-lists are now keyed on repo-relative POSIX paths only.
3. **Four literal substrings** — an AST pass alongside them catches what the audit planted and the old predicate missed: aliased imports, imports opened inside a function body, `import_module("cockpit." + "kali")`, `getattr(m, "run_" + "kali")`, and f-strings with constant parts.

Prose is stripped so a module that *documents* a rule does not violate it — this is not hypothetical, documenting the Critical 2 fix tripped the WinRM lock — but ordinary string literals are deliberately **kept**, because a scanner that blanked every string would go blind to `import_module("cockpit.kali")`, which is the indirection the lock exists to catch.

Two more locks were closed with it: the **cockpit→arsenal** lock checked a hardcoded list of five filenames while printing "the cockpit package has zero references to the arsenal" (the package has 22 modules, 17 of which post-date the invariant), and the **evasion agent-path** scan still filtered by *filename*, so `from evasion import engine` in `cockpit/executor.py`, `chat.py` or `adgraph/techniques.py` was never looked at. A module named something nobody predicted is exactly what a name filter cannot cover.

**Widening the scans produced two false positives, and both were fixed in the predicate.** `cockpit/reconcile.py` "referenced the arsenal" — as a *parameter name*; it takes the loaded catalog as an opaque injected object precisely so it has no import-time dependency. `detection/{catalog,resolver,router}.py` and `report.py` "reached the evasion engine" — they are the *anti*-evasion guard, the code that refuses prescriptive evasion copy. Both locks now assert the claim the invariant actually makes (no import, in any form the AST can see). **Narrowing the file set to silence a false positive is how these guards got broken in the first place**, so that is written into the convention as a rule.

**Task 3 — the convention, which is the part that outlasts both fixes.** Every critical in this audit, and two of build #4's five review findings, reduce to one root cause: a test that could not fail, or that tested a value the real system never produces. `test_arsenal_safety` claimed to verify that a catalogued dangerous invocation demands the red-confirm while testing `python3`, a command absent from the catalog it was guarding — one choice that hid eight tools. `backend/AGENTS.md` now carries the rule, with the five failures that produced it in a table: **a safety test must iterate real data, assert on what it actually checked, and prove it can fail.** Concretely — draw inputs from the real source of truth so a tool or module added tomorrow is covered by nobody remembering; assert on the *filtered* count, never files opened; and carry a planted-violation positive control in the same test. `test_scans.py` runs **first** in the suite for that reason: ten locks rest on the shared scanner, and it must demonstrate it catches a planted violation before any of them means anything.

Sweeping for the *pattern* rather than the files the plan named found three more, none of which was in the task list. The **live-session stdin lock** — the load-bearing one, since anything able to type into an already-approved running session is executing un-gated commands — said "the whole source tree" and globbed three directories. `test_sliver_safety` and `test_obfuscation_safety`, the two suites the audit called correct, incremented their `scanned` counter *above* the `test_` skip, so their `>= 40` controls counted files opened rather than files judged. And re-running the audit's own probe list surfaced a real heuristic gap: `sekurlsa::pth ... /run:cmd.exe` as `argv[0]` fired only because `cmd.exe` happens to sit in its args (change it to `/run:powershell.exe` and it went silent), while `kerberos::list /export` — which writes every ticket in the session to disk — matched nothing at all.

**A follow-on fix, after the plan's three tasks: the tunnel listener start is now gated ( `I2` ).** It was outside build #5's plan and was recorded here as open, then closed immediately after. `cockpit/tunnels.start_tunnel` reached `subprocess.Popen` directly with no `validate_request` call, and `POST /cockpit/tunnels` carried no `approved` / `dangerous_ack` field — so a pivot listener, which is a C2 path and an exfil path in one, was raised on a plain POST with human-only as its *sole* bound, strictly less than every other execution surface gets. The docstring's claim that "the rewritten pivoted command still runs through the gated executor" was true and beside the point: the listener start *is* the tunnel primitive. Fixed test-first (the unapproved-start test was confirmed failing on the old code): `start_tunnel` now runs the real `executor.validate_request` before anything spawns, `TunnelStartRequest` carries `approved` + `dangerous_ack` defaulting **false** (an omitting client is refused, never silently granted), and the route maps a safety refusal to 403 naming the gate while an availability problem stays 409. The danger verdict comes from the server binary — `chisel` and `ligolo-proxy` are both in `_TUNNEL_TOOLS`, pinned by a positive control so the leg cannot pass vacuously — and a start now requires an engagement, because without one it would resolve to lab mode and fire the lab's isolation gate about a container the listener does not run in. The gated argv and the spawned argv derive from one `server_argv_for`, the same drift lock Critical 2 uses. This is deliberately **not** the D17 shape: for Sliver, C2 lifecycle is human-only while artifact generation is gated, because a listener nothing has connected to is inert; a pivot listener's whole purpose is to become a route into an unseen network, so it is gated like an execution.

**What still remains open.** `iter_run`'s prevalidated path re-checks approval but not danger (nothing is exposed today, because the single `prevalidated=True` caller validates first, but the asymmetry is the kind of thing a future caller trips over), and there is still no route-level authorisation on any of the routes, which remains the known localhost-only posture rather than a new finding.

## Live fire, image pinning, and the prevalidated danger re-check — build #7 (2026-07-27)

Three items from Part II's deferred list, none of which adds offensive capability. One makes the image byte-reproducible, one closes a latent asymmetry in a belt-and-suspenders gate, and the largest — live-fire verification — went looking for evidence that the C2 and tunnel surfaces work and came back with three defects and a lot of confirmed containment.

### Live fire: what ran, and what it found

`docker/proof/live_fire_proof.sh` builds a lab that is ours end to end — four containers on four `internal: true` networks (`docker/proof/live-fire-lab.yml`), no third-party target, no public callback — and drives the surfaces through their shipped entry points via `docker/proof/live_fire_driver.py`. Every gated step **proves the refusal first**: the same request with the acknowledgement missing must be refused with nothing spawned, because a proof that only shows the happy path cannot tell a working gate from an absent one. Results are reported as PASS / FAIL / **NOTRUN**, and a step that could not run is never folded into a pass. The run stands at **26 passed, 3 failed, 5 not-run**.

The gates and the containment held, live and without exception:

- **The `I2` pivot gate fires on all three limbs** against a real `ligolo-proxy` start — no engagement refuses at `gate=engagement`, unapproved at `gate=approval`, un-red-confirmed at `gate=danger` — and the approved start is accepted. This is the first time that gate has been exercised outside a stub.
- **The Sliver implant gates fire** — `gate=approval` unapproved, `gate=danger` without the red-confirm.
- **The pivot subnet enters scope only through the explicit amendment**, verified against a before/after snapshot of the engagement record, with the control taken first: the deep target is confirmed *unreachable* from the operator box before any tunnel exists, so "we reached it through the tunnel" would have meant something.
- **Containment holds live, measured from inside the contained host.** The implant host cannot reach the internet (`1.1.1.1`) or the host (`host.docker.internal`), and *can* reach the C2 — so the beacon has nowhere else it could go, which is what makes demonstrating a callback safe in the first place.
- **The DNS tunnel key never enters the pasteable one-liner.**
- **A real Sliver server runs.** `sliver-server daemon` starts through `cockpit/sliver.py`, unpacks its embedded assets and holds `:31337`.

Three things **failed**, and they are defects in the surfaces rather than in the gates. All three are the same shape: a claim that unit tests asserted structurally and nothing had ever executed.

1. **The Sliver implant build cannot succeed as shipped.** `_implant_argv()` builds `sliver-client generate --os … --mtls <listener>`, and `sliver-client` has no `generate` subcommand — it has `import`, `version`, `completion`, `help`. In Sliver 1.5 `generate` is an *interactive console* command. The existing tests assert the argv's **shape** (that `<listener>` is emitted verbatim, that the target is never emitted) and never that Sliver accepts it, so a build that could not run looked correct for as long as nobody ran one. The console route was verified working during this build with the identical flags over stdin — `echo "generate --os linux --arch amd64 --format exe --mtls H:P --save …" | sliver-client` produced a 14 MB implant in 27 s — so the fix is known and small. **It was deliberately not applied here**: making a non-functional offensive path functional is a capability change, and this build adds none. It is the operator's call.
2. **`tunnels.start_tunnel` reports `status="listening"` for a process that has already exited.** `ligolo-proxy` is an interactive console binary; spawned with no usable stdin it binds the port, prints `Listening on`

`0.0.0.0:11601`, reads EOF and exits 0. The status is assigned at `Popen` time and never observed, so the panel shows a live pivot over a dead process. Confirmed by `proc.poll()` returning `0` four seconds after the model claimed it was up, with nothing bound in the container.

3. `obfuscation.start_listener` **has the same defect** for `dnscat2-server`, for the same reason.

Because the implant never built and the listeners never survived, the three end-to-end claims those steps feed are recorded as **not-run, not as passed**: no beacon callback was caught, no proxychained traffic reached the deep subnet, and no dnscat2 session was established. The harness now asserts process liveness directly, so these three become regressions the moment they are fixed.

Two further items are **not-run because they need operator infrastructure**, and no amount of local wiring substitutes for either:

- **A delegated DNS zone.** Only the direct-to-server dnscat2 path can be exercised locally. A genuine delegated tunnel needs a domain the operator controls with an NS record pointed at the listener. "Demonstrated in lab" and "needs a real delegated zone" are different claims and are kept apart.
- **iodine**, which needs a TUN device at *both* ends plus that same zone; the engage sandbox has `/dev/net/tun`, the lab client container does not.

A side finding, recorded rather than acted on: `obfuscation.start_listener` justifies being human-only-and-ungated by citing "the identical reasoning the pivot-listener lifecycle uses" — and the pivot listener was tightened in build #5 (`I2`) to require engagement plus approval plus the red-confirm. The cited precedent no longer holds. Whether the DNS listener and the Sliver server start should follow it is a design decision, not a defect, and it is left open.

One environmental fact surfaced immediately and is worth writing down: the running stack was still on `hackpit/kali-sandbox:m1` built *before* build #4, so none of the C2 tooling existed in the sandbox the gated code execs into. The compose file rebuilds that tag from the current Dockerfile; nobody had re-run it with `--build`. The stack was realigned and both network proofs re-verified afterwards — `isolation_proof.sh` 4/4, `engage_open_proof.sh` 4/4.

### The image is now byte-reproducible — and its smoke test had never passed

`docker/pin-build4-versions.sh` resolved the real upstream values and rewrote the layer: dnscat2 to commit `42f8d783…` (it has no release tags, so a SHA is the only thing there is to pin to), the four Ruby gems to `trollop:2.9.10 salsa20:0.1.3 sha3:2.2.4 ecdsa:1.2.0`, and `donut-shellcode==1.1`. Two bugs in that script had to be fixed before it could run: it required `python3` on `PATH` (only the backend venv exists here) and its verification step used `grep … && { exit 1; }`, which under `set -e` exits 1 on the *success* path. `test_evasion.py` now asserts the stronger claim — all three installs pinned, three `ARG` version pins, the SHA a real 40-char object id, no floating form left behind — instead of tolerating a documented gap.

The fresh build then failed, and not because of the pins. The consolidated smoke test asserted `sliver-server version 2>&1 | grep -qi sliver`, and `sliver-server version` prints exactly `v1.5.42 - <sha>` — no substring "sliver". **That assertion could never pass, so this layer had never built since the check was added**; the image in the daemon predates it. The static guard that watches this block (`test_evasion.py`) reads the *shape* of these lines — `|| true`, `| head` — and by construction cannot know whether a grep pattern matches anything, so only a real build could surface it. Both sliver checks now match `v${SLIVER_VERSION}`, which is strictly stronger: it fails when the binary is missing, when it dies, and when it is not the version pinned above. `hackpit-kali:build7` builds clean, and all three pinned tools resolve and smoke-test by their real invoked names, with the shipped versions confirmed inside the image against the pins.

### The prevalidated path now re-checks danger

`iter_run(prevalidated=True)` skips `validate_request` because the router already ran it to decide the HTTP status. It has re-checked **approval** for engagement and windows since build #4 — never-auto-run being the sole floor on a real target — but had no equivalent **danger** re-check. Nothing was exposed: the one `prevalidated=True` caller validates first. But "no caller does this today" is a coincidence, not an invariant, and the asymmetry was a trap set for the second caller.

Test-first per `backend/AGENTS.md`: the failing test showed a dangerous, un-acked, prevalidated request reaching `start` and spawning. The re-check mirrors the approval one and uses the **same per-mode classifier** `validate_request` uses — `windows_danger_reasons` on the joined script for windows, `dangerous_command_heuristic` on the argv for lab and engagement. The windows probe is deliberately a command only the script classifier catches (`Write-Host go ; Invoke-Mimikatz`, the Critical-2 shape), so a re-check wired to the generic heuristic fails the suite rather than passing quietly.

The refactor that made this work is worth recording because the project's own guard caught it. Routing all three validators through one shared `danger_reasons_for_mode()` dispatcher looked strictly better — divergence becomes structurally impossible — but `test_winrm_safety` asserts over the **static call graph** that `_validate_lab` can never reach `join_ps_command`, and that assertion is the positive control proving the reachability check can fail at all. The dispatcher reaches the PowerShell derivation on its windows branch, so wiring the docker-path validators through it emptied the control while every test still passed. The validators call their classifier directly and deliberately; only `iter_run`, which knows a `mode` string rather than a branch, uses the dispatcher. The classifier each side calls is identical either way — only the call graph differs, and the call graph is what the control reads.

One existing test needed updating rather than the guard being loosened: `test_winrm_safety`'s host-lock case drove `Invoke-Command` prevalidated with no ack. `Invoke-Command` *is* flagged, so the router would have demanded the red-confirm before ever setting `prevalidated=True`; the test had been relying on the gap it was sitting next to. It now carries the ack, which is what the real path sends.

## The three live-fire defects, fixed — and a stale precedent decided (2026-07-28)

Build #7 found three surfaces that did not work as shipped and deliberately left them alone, on the grounds that making a non-functional offensive path functional is a capability change and that call belongs to the operator. The call was made: fix all three, and decide the design question the run had left open. This section is what that took, including two further defects the fixing surfaced — both of the same family, and both found the same way.

### The Sliver implant build: a subcommand that does not exist

`_implant_argv()` built `sliver-client generate …`, and neither Sliver binary has a `generate` subcommand — `sliver-client --help` offers `completion`, `help`, `import`, `version`, and nothing else. In Sliver 1.5 `generate` belongs to the interactive **console**, which `sliver-server` hosts when run with no subcommand. A build now runs `docker exec -i <container> sliver-server` — still an argv **list** — and writes one `generate …` line to its stdin, followed by `exit`. Verified against the pinned 1.5.42 image: 16 s, a 14 MB implant on disk.

Two consequences follow, and both are handled rather than hoped about.

The console line is **text** reaching Sliver's own flag parser, so it is a parser this code did not previously feed. Everything interpolated into it is bounded before it arrives: `os` / `arch` / `format` / `transport` already came from fixed sets, `listener` is now pinned by a pattern that admits a host or IP with an optional port (or an angle-bracket placeholder) and rejects whitespace and a leading `-`, and the server-chosen artifact path is checked whitespace-free rather than trusted — it is derived from a caller-supplied engagement id, and a space in it would split into extra console tokens. Nothing here reaches a shell: the argv is still a list, so `/bin/sh` is not in the picture at all.

And **the console exits 0 whether or not the build inside it worked**, so an exit code cannot decide success. `generated` versus `failed` is now decided by reading the artifact back out of the container with `stat`. That is a stricter test than the one it replaces, in both directions: a console that exits 0 having written nothing is `failed`, and a console that complains while producing a real artifact is `generated`. The gate surface moved with the binary — `_gate_request` classifies `argv[0]` itself rather than a constant, so the audit row names the process that actually runs.

### Both listener lifecycles: a status that was assigned, never observed

`ligolo-proxy` and `dnscat2-server` are interactive consoles. `docker exec` **without** `-i` hands a container process a closed stdin no matter what the host process got, so both printed their banner, read EOF and exited — while `start_tunnel` and `start_listener` returned `status="listening"`, assigned at `Popen` time and never looked at again. The operator was then holding an agent one-liner for a port with nothing behind it.

Both halves are fixed in one place, `backend/cockpit/lifecycle.py`, now shared by all three lifecycle surfaces:

- **A console binary gets a stdin that does not end.** `-i` is per-binary and off by default — `ligolo-proxy` and `dnscat2-server` need it; `chisel server`, `iodined -f` and `sliver-server daemon` are real daemons and are spawned without one, verified rather than assumed.
- **The write end is a raw file descriptor, deliberately.** Passing an int fd rather than `subprocess.PIPE` leaves `proc.stdin` as `None`, so there is no writer object for any caller to reach for. Holding a listener's stdin open is a liveness requirement; being able to *type into* a live C2 console is a capability, and this grants the first without the second. A whole-tree source scan with a planted-violation control keeps the handle inside the four modules that own a listener.
- **The status is observed.** After a settle window: a process that exited is a **refusal** carrying the output it died on; a live process whose port `ss` confirms bound inside the container is `listening`; a live process whose bind could not be confirmed is `starting`. The probe is tri-state on purpose — "could not probe" is not "not bound", and neither is ever rendered as a confirmed listener. Every model now carries a `liveness` field holding the evidence, not just the verdict.
- **Output is drained** into a bounded ring buffer, so a chatty server cannot fill the 64 KB pipe buffer and wedge mid-engagement looking exactly like a hung tunnel.

### The stale precedent, decided: both listeners are now gated

`obfuscation.start_listener` justified being ungated by citing "the identical reasoning the pivot-listener lifecycle uses" — and build #5's `I2` finding had already tightened that pivot listener to require engagement plus approval plus the red-confirm. The citation was stale. The larger problem was that `I2`'s actual argument applies to a DNS tunnel with full force: *a listener whose purpose is to become a route into a network the scope gate has never seen is an execution, not a lifecycle no-op.* A DNS-tunnel listener is the server end of a covert exfil channel — carrying arbitrary traffic out of a network through that network's own resolvers is the thing egress controls exist to stop. "No target" was true and beside the point, exactly as it was for the pivot listener.

**The DNS listener is now gated the same way**: engagement, approval, red-confirm, through the real `executor.validate_request`. `dnscat2-server` and `iodined` were already in the covert-tunnel danger set, so the heuristic fires on the binary and the red-confirm is always required.

**The Sliver server start is gated too, on a weaker argument that is worth stating honestly.** A Sliver daemon is genuinely less exposed than either listener: starting it opens no channel toward anything under test, it binds a multiplayer port on the operator's own sandbox for the operator's own console, and the steps that *do* reach a target — creating a listener, generating an implant, delivering it — are separate and separately gated. That argument is real, and it is not quite sufficient, because Sliver's server config can **persist listener jobs** that come back up with the daemon. "Starting it is inert" is therefore a property of a config file, not a property of this code, and a gate should not rest on something the module cannot see.

Human-only remains the load-bearing property for all three surfaces and is untouched: the whole-tree source scans still prove no orchestrator, agent or loop path reaches any of them. The gates are belt to those suspenders. What still differs between the two halves is the **target**: a C2 server and a DNS listener have none, so neither carries a target-lock, while implant generation is scope-checked as before — and `SliverServerRequest` is asserted to have no `target` field, because one would mean a scope gate firing on the operator's own box.

### Two further defects, found the same way

Re-running the live fire against the fixed surfaces surfaced two more, both invisible to every unit test and both the same shape as the originals: a state reported rather than observed.

**A stop did not stop.** Killing a `docker exec` client does not kill the process it started inside the container. After stops that reported `status="down"`, `ss` inside the sandbox still showed `iodined` holding UDP/53 and two `chisel server` processes listening — and the next start failed with `EADDRINUSE`, a refusal about a listener the operator had been told was already stopped. A stop now closes stdin (which is enough for a console binary), kills the client, and then reaps the server inside the container by **its own argv**: `pkill -f` against the full command line, which carries the listener's port, so it stops that listener rather than every process of the same kind. A blanket `pkill -f chisel` would have stopped somebody else's tunnel.

**`proxychains` pointed at Tor.** Debian ships `/etc/proxychains4.conf` configured for `socks4 127.0.0.1 9050`, while HackPit's chisel plan brings up a reverse SOCKS5 at `127.0.0.1:1080` and `wrap_command` rewrites a command to `proxychains -q …` on the strength of it. With the stock config the rewritten command the operator approves would have quietly tried Tor and timed out. The Dockerfile now replaces that line rather than appending to it — proxychains reads the last `[ProxyList]` section, and leaving 9050 in place would make the chain depend on ordering. This is exactly the class of gap a proof harness exists to find: every unit test passed, and the shipped path could not work.

### Both fixes verified against a rebuilt image, not just against a diff

Two of the five fixes carried a claim that had not itself been executed, which is the exact failure mode this build exists to eliminate, so both were closed properly rather than left as edits.

**The `proxychains` line had never been built.** The fix was written into `Dockerfile.sandbox` while the running image predated it, and the harness applied the same one-line edit at runtime — correct for making the proof valid, but it left the Dockerfile layer itself unproven. `hackpit-kali:build7b` now builds from the current Dockerfile with exit 0, and the config inside the image reads `socks5 127.0.0.1 1080` with **zero** `socks4` lines remaining. The stack was rebuilt and recreated from that Dockerfile, and the live fire re-run against it takes the "*already points at the chisel SOCKS5 — baked into the image*" branch rather than the runtime-patch branch, with the proxychained request still routing. The runtime patch stays in the harness on purpose, so the proof remains valid on an older image, and it says which branch it took.

**The stop-reap was unit-tested but never demonstrated against the failure that motivated it.** Run live: a `chisel server` started through the lifecycle and confirmed bound; killing only the `docker exec` client left the process running and the port still bound — the defect reproducing exactly as described; the reap then cleared both. And it is targeted, which is the part that matters: reaping one tunnel left a second `chisel server` on a different port still bound, where a blanket `pkill -f chisel` would have stopped it too.

That second run also surfaced a sixth defect of the same family, now fixed. On a freshly recreated container `ligolo-proxy` took longer than the settle window to bind, so it was honestly reported `starting` — and `route_for` correctly refuses to route through an unconfirmed bind. But the status was **observed once and never re-observed upward**: the list functions only ever demoted to `down`, so a listener that bound a second after its window closed stayed `starting` for its whole life, and therefore permanently unroutable. The refresh now moves in both directions and only on evidence — a dead process becomes `down`, a `starting` listener whose port has since appeared becomes `listening`, and a probe that says "not bound" or "could not run" promotes nothing. Only `starting` listeners are re-probed, so a confirmed bind does not cost a `docker exec` on every poll of the panel. The negative controls are in the same test, because a refresh that promoted on an unknown would be laundering exactly the uncertainty the settle-window observation exists to preserve.

### What the live fire now shows

**66 passed, 0 failed, 3 not-run** (after the not-run-closing pass below). The defects are fixed and their assertions pass; the gates and the containment continue to hold, and iodine now demonstrates the DNS-tunnel class end to end.

- **A real implant builds** through the gated path — `status='generated'` with a 14 MB artifact read back out of the container by size — and **the beacon calls back**: the implant, run on a lab host whose only network is internal, registers a session with the operator's server. Containment is measured from inside that host: no internet, no Docker host, C2 only.
- **A proxychained request routes through the pivot** to a host on a subnet the operator box has no route to, returning the deep target's known body — and the argv that ran is the one `wrap_command` produced, so what was demonstrated is what a human would have approved. The same tunnel cannot reach `1.1.1.1`.
- **All three lifecycle gates refuse on all three limbs** — no engagement, unapproved, un-red-confirmed — with nothing spawned, for the Sliver server, both pivot kinds and the DNS listener. Each surface's status is separately asserted to have been *observed*, against the process handle and against the `liveness` evidence.
- **Every listener stays up** and its port stays bound for the whole phase.

Four things are **not-run, and they are not passes**:

- **ligolo's interface routing.** ligolo routes at the interface, and creating that route means typing `session` then `start` into the proxy's console. HackPit holds that console's stdin open and deliberately never types into it — an unapproved command on a live C2 console is precisely what the gates exist to prevent. Completing a ligolo session stays the operator's own step. The routed-traffic claim is carried by chisel, whose server needs no console at all, and that is why both kinds are exercised.
- **The dnscat2 client handshake** — now diagnosed decisively (see the next subsection): `tcpdump` shows the `dnscat2-server` process itself answering `NXDOMAIN` to correctly-formatted queries, reproduced with both ends in one container over loopback, which isolates it to the pinned dnscat2 build rather than to HackPit or the lab. The listener HackPit starts is gated, bound and stable throughout.
- **A delegated DNS zone.** Unchanged and unfakeable: a genuine delegated tunnel needs a domain the operator controls with an NS record pointed at the listener. "Demonstrated in lab" and "needs a real delegated zone" stay different claims.

- **A delegated DNS zone** — the public-hierarchy hop in front of either tunnel. iodine's IP-over-DNS channel itself is now **demonstrated** (below), carrying traffic and confirmed DNS-encapsulated on the wire; what stays not-run is only the delegation in front of it, which needs a domain and a public listener the operator does not have.

One harness lesson is worth recording, because the harness was briefly lying about the product. A console listener's lifetime is tied to the process holding its stdin — in the backend that is the long-lived server process, which is correct. But each driver invocation was its own short-lived Python process, so a listener came up, was confirmed bound by `ss`, and died the moment the driver exited, after which the shell's follow-up check reported "did not reach LISTEN" for something that had demonstrably been listening seconds earlier. Client connect-backs now happen inside the same process that holds the listener, so every assertion is measured while the thing it is about is alive. Three further false findings had the same character and are documented in place: a `pkill` sweep that killed its own wrapper shell on the first pattern and silently abandoned the rest, a `sed` whose absolute path Git Bash rewrote into a Windows path so the config edit never applied, and a `dnscat2` client invocation refused for passing both a `--dns` argument and a bare zone. A proof that reports its own bugs as product defects is worse than no proof.

### Closing the not-runs: iodine demonstrated, dnscat2 diagnosed (2026-07-28)

Four things were left not-run after the surfaces were fixed. Two are now resolved — one by demonstration, one by a decisive diagnosis — and the remaining two are stated for what they actually are rather than carried as vague gaps.

**iodine's IP-over-DNS tunnel is now demonstrated carrying traffic, through the gated surface.** A dedicated lab fixture (`iodine-client` in `live-fire-lab.yml`) carries the one thing iodine genuinely needs and the dnscat2 host must not have — a `/dev/net/tun` device, `NET_ADMIN`, and root, because iodine brings up a whole tunnel *interface* and a `cap_add` is not effective for a non-root process. The listener still comes up through HackPit's gated `start_listener(kind="iodine")`, refusing on all three limbs first; iodine is the other tool on the same gated path, not a bypass. The client is then forced onto the DNS-encapsulated path with `-r` — load-bearing, because on a flat lab bridge iodine will happily fall back to raw UDP and report a "tunnel" that never touched DNS, which would be a dishonest pass. The proof confirms the real thing on the wire: four ICMP echoes reach the server's tun endpoint through the tunnel with 0% loss, and a `tcpdump` on the server's UDP/53 shows the traffic arriving as DNS-encoded queries under the tunnel zone — eight of them for that ping. That is a genuine interface-level DNS tunnel, gated, carrying traffic, confirmed DNS-mode. (One harness bug surfaced and was fixed in the same pass, and it is the honesty guard earning its place: `tcpdump` line-buffers, so a first draft grepped the capture file before the packets had flushed and reported "no DNS packets" for a tunnel that was demonstrably DNS-encapsulated. The check now waits out the capture window.)

**dnscat2's failure is now diagnosed decisively, and it is not HackPit's.** A `tcpdump` on the server shows the client's encoded queries arriving correctly formatted — `<data>.lab.hackpit.internal`, alternating MX/TXT/CNAME, which is dnscat2's own protocol — and the `dnscat2-server` process itself answering `NXDOMAIN` to every one. Put both ends in the *same container* talking over `127.0.0.1` — no network, no resolver, no HackPit code, the server's DNS driver confirmed started (`New window created: dns1`) — and it still `NXDOMAIN`s, in every client shape. That isolates it fully to the pinned dnscat2 build (commit `42f8d783…`): a defect in the tool's own client/server handshake, sitting entirely above the listener HackPit starts, which is proven gated, bound and stable throughout. It is not chased further — fixing upstream dnscat2 is out of scope, and iodine now demonstrates the DNS-tunnel class end to end regardless. The proof records this as a precise finding rather than a bare "did not work".

**The two that remain are genuinely operator-side, and cannot be faked into a pass.** A *delegated* DNS zone — the public-hierarchy hop in front of either tunnel — needs a domain the operator controls with an NS record pointed at the listener, plus a publicly reachable listener, i.e. the same VPS gap as D2. The operator has neither, so it stays recorded as a prerequisite. And ligolo's *interface* route is a deliberate **won't-do**, not a not-yet: completing a ligolo session means typing `session` then `start` into a live C2 console, and the only way HackPit could do that is a surface that writes to a listener's stdin — precisely the capability `test_lifecycle_safety` exists to forbid (the held stdin is a raw fd specifically so no writer object exists). chisel already carries the routed-traffic claim end to end, so nothing is lost by leaving ligolo's console step to the operator.

## Build #8 — the reasoning copilot, and a measured substrate (2026-07-28)

Two things this build set out to establish, one about the agent and one about the ground it stands on: make the orchestrator a genuinely *deeper* proposer without giving it one inch more autonomy, and turn "110 tools" from a claim into a measured number.

### The substrate actually runs, and here is the number

`reconcile.py` has long answered *installed?* with a `command -v` sweep. That is a weaker claim than it sounds: a binary can resolve on PATH and then die the moment the dropped capability is needed. So `backend/substrate_probe.py` asks the harder question directly — it invokes every catalogued tool in the **live sandbox image, under its real profile** ( `no-new-privileges` + `CapDrop: ALL` ) with a trivial call ( `--version` / `--help` ) and records three tiers: *catalogued → installed → actually-runs*.

**97 of 104 catalogued Linux tools actually execute (93.3%)**, measured, not asserted. The breakdown is reported honestly, because a tool that does not run is never counted as covered:

- **2 installed-but-do-not-run** — `amass` and Empire ( `powershell-empire` ). Both are the documented landmine class, though not the one expected: their packaged wrappers shell out to `sudo`, which `no-new-privileges` refuses, so the binary resolves and then fails. (nmap, the *expected* casualty, runs fine — the image was hardened since that trap was recorded.)
- **5 not-installed** — `prowler`, `scoutsuite`, `kube-hunter` (cloud-audit tooling), `ghidra`, `subwiz`. Genuinely absent from the image; reported as gaps, not covered.
- **6 windows-only** — Rubeus/PowerView/Mimikatz/winPEAS-class entries, not applicable to a Linux sandbox by construction (D9).

The probe earned its place immediately: it surfaced two real catalog defects of exactly the kind Task 1 warns about — **a package that installs under a different binary name than the catalog uses.** Sliver resolves as `sliver-server` / `sliver-client` and Empire as `powershell-empire`; the catalog listed neither alias, so both read as "not installed" until the aliases were added — and adding them then forced those binaries through the arsenal danger gate under their *real* invoked names ( `sliver-server`, `sliver-client`, `powershell-empire` now each demand the red-confirm, the same `sliver` -vs- `sliver-client` bug the gate audit fixed once already, caught again by the same test). The full per-tool report is committed at `docs/substrate-coverage.md` (+ `.json` ); the static half (catalogued names vs. Dockerfile install lines) is unit-tested and needs no stack.

### The reasoning proposer (2.1–2.8) — deeper proposals, not autonomy

The orchestrator used to pattern-match: it saw a port and reached for the tool it always reached for. `backend/reasoning/` makes it reason, and every component is additive, independently tested, and produces **proposals + rationale only**:

- **2.1 working memory — a tried/failed ledger.** The single biggest anti-looping lever. The proposer is fed the full state plus a ledger of what has been tried and how it turned out (read from the exit code *and* the output — a tool can exit 0 having failed), and a lead ruled out by the critic is persisted as a dead end. A failed lead does not get re-proposed. Validated live: with a `curl` to a closed port recorded failed, the model proposed the SQLi step against the real finding instead of repeating the dead lead.
- **2.2 hypothesis-first proposals.** The schema now leads with `hypothesis` + `expected_signal`, then the command — enumerate-before-exploit as a shape, and a wrong guess made visible. A proposal missing hypothesis, expected_signal, or a citation **fails a validation gate** (invariant 3, expressed as code with a control that shows it can fail).
- **2.3 a candidate frontier, not a greedy single step.** The proposer surfaces N leads, each scored by *evidence strength* × *expected payoff*; the top is pursued and the rest are **persisted as untried leads**, so a dead-end recovers to the next-best instead of looping. This generalizes the AD graph's edge-frontier to the whole engagement. It is a set of leads, **never a run queue** — nothing in it fires.
- **2.4 failure diagnosis.** A connection-refused output yields a reachability check *before* another attempt, not a blind repeat.
- **2.5 a skeptic/critic pass.** A second look tries to *refute* the proposal against the evidence, reusing the CVE→exploit index's rule that **the version verdict outranks token similarity**: a proposal citing a CVE whose exploit targets Apache 2.4.49 is caught and downranked against an observed 2.4.58, no matter how well the words match. This is the check that kills the top hard-box failure mode — confident hallucination. The critic is **advisory only — it downranks and flags, it never suppresses**: a refuted proposal is surfaced with its concerns and a lowered confidence so the human sees it is shaky, but the proposer stays free to raise it again, and nothing is ever blocked from being run by hand. (The first live run against Ollama found a real bug in this pass — it read an IP octet as a port — which was fixed and regression-locked; the honesty guard earning its place.)
- **2.6 domain-specialist routing.** A web finding routes to a web lens, an AD state to an AD lens, a foothold to a privesc lens — instead of one generalist voice.
- **2.7 fingerprint-keyed retrieval.** An exact service+version fingerprint (case-based: "this stack was solved by X") ranks ahead of generic token matches. The plumbing is built now; **growing the exploitation-write-up corpus it keys into is a follow-on** — noted, not faked: on today's KB the ranker degrades to the base order.
- **2.8 model-tier routing.** A config lever, default OFF and inert: with no `reasoning_tiers` configured, every step uses the base config byte-for-byte; configured, a hard step can be routed to a more capable model.

**The honest ceiling.** Reasoning depth is mostly the base model. A small local model caps it no matter how good the scaffolding is — the same live run that produced a fully-cited, correctly-routed proposal produced, on a re-run, one with no hypothesis at all (which the schema gate then flagged, correctly). This build moves the loop *up the curve* on easy and known-pattern-hard boxes; it does not turn a 9B local model into something that "solves anything hard." What it reliably changes is the loop's behaviour: it stops re-proposing dead leads, it states what it is testing, and it refuses to cite a CVE that does not fit the version in front of it.

### Web-exploitation and privesc depth (Tasks 3 & 4, still human-fires)

Two drafting surfaces, both propose/ground/generate only:

- **Web (`reasoning/webexploit.py`, `POST /sessions/{id}/webexploit/draft`).** Given a web finding in state, HackPit drafts the actual exploit — the sqlmap invocation, the SSRF metadata probe, the IDOR replay, the parameter-pollution request, the LFI traversal — with an explanation grounded in the bug-bounty KB and citations back to the finding, handed to the human to fire through the **existing** repeater/executor. Nothing in the path sends without the human.

- **Privesc ( `reasoning/privesc.py` , `POST /sessions/{id}/privesc/ingest` ).** Paste linpeas/winpeas output; it identifies the vectors (NOPASSWD sudo, SUID GTFOBins, PwnKit, capabilities, SeImpersonate, AlwaysInstallElevated), drafts the escalation for the strongest one grounded in KB + state, and hands it over. Execution stays human-approved.

### What is re-locked, unchanged

The whole point is that the proposer got smarter and nothing else moved. `test_loop.py` 's source-scan lock is **extended onto the shared scanner across the entire `reasoning/` package**: orchestrator.py plus all eleven reasoning modules are proven to have no execution path — no subprocess, no executor run method, no `:kali` /sandbox — by AST call-analysis (so a *drafted payload string* that contains `os.system(...)` as exploit text is correctly **not** a violation, while a real call is), with positive controls. A companion scan proves there is **no auto-approve / batch-approve / run-the-chain** anywhere in the proposer path or the frontier. The four execution gates (scope, approval, danger red-confirm, isolation) are untouched and still fire.

### KB exploitation-writeup corpus — the case-based material 2.7 keys into (follow-on, now landed)

Build #8 shipped 2.7's fingerprint retrieval as plumbing and flagged the corpus it queries into as a follow-on; that corpus now exists. `pipeline/ingest_exploitation_writeups.py` seeds **78 distilled exploitation writeups** keyed by service+version fingerprint — the initial 16 (vsftpd 2.3.4, ProFTPD mod_copy, Samba usermap, SMBv1/EternalBlue, Apache 2.4.49/.50 traversal→RCE, Shellshock, Jenkins, Tomcat, Drupalgeddon2, GitLab ExifTool, Log4Shell, unauth Redis, BlueKeep, Kerberoasting, AD CS ESC1, Zerologon) plus a second batch of 19 (Confluence OGNL, Struts2, Citrix, F5 BIG-IP, PrintNightmare, ProxyShell, ProxyLogon, Spring4Shell, FortiOS, PHP-CGI, Grafana LFI, WebLogic, phpMyAdmin, MSSQL xp_cmdshell, SNMP, PostgreSQL, Webmin, Joomla, Elasticsearch), and a third of 43 across four themes — **AD depth** (AS-REP roasting, DCSync, noPac, unconstrained/RBCD delegation, PetitPotam+ESC8, GPP cPassword, LAPS, pass-the-hash, golden ticket), **unauth network services** (MongoDB, Memcached, CouchDB, MySQL UDF, JMX/RMI, IPMI, VNC, SSH user-enum, anon FTP/LDAP, SMTP enum), **Linux privesc** (DirtyPipe, Dirty COW, Baron Samedit, Docker/LXD group, NFS no_root_squash, PATH hijack, LD_PRELOAD, wildcard injection) and **modern enterprise CVEs** (Spring Boot Actuator, Laravel Ignition, Text4Shell, ColdFusion, PaperCut, TeamCity, OFBiz, vCenter, Cacti, ThinkPHP, WordPress, GraphQL, Zimbra) — each carrying a structured `meta.fingerprint` ( `service` + `version_kind` + `versions` + `cve` + `solved_via` + a one-line detection footprint), whose `version_kind` / `versions` shape **mirrors the CVE→exploit index exactly** so the version verdict stays authoritative.

Three properties matter and each is verified:

- **Distilled, not verbatim.** Every entry is a general technique note — the approach, the command pattern, what it leaves in logs — distilled from the offensive-*/ bug-bounty methodology and public CVE facts. Not one line is a copyrighted third-party walkthrough. Operator-supplied writeup notes are handled like the phishing lures: imported at runtime with `--operator` from a gitignored engagement directory, marked `operator`, and never committed (entries.jsonl is itself gitignored, so the seed ships only as ingester* code* that regenerates it).
- **Additive, idempotent, consolidation-safe.** The ingester mirrors the recon-methodology template: existing lines pass through as raw bytes, its own lines carry `meta.exploitation_writeup`, and a re-run is **byte-identical** (verified by hash). Each entry is `category="writeup"` + `meta.no_merge`, which `consolidate.py` treats as standalone — so no duplicate is ever created, without re-running the full pipeline (which would risk the downstream-enrichment revert and a Defender quarantine). The file was confirmed present and countable after every write. KB 2,621 → 2,699.

- **Reachable by 2.7, with the version verdict intact.** A fingerprint query for a discovered service returns the seeded writeup **ranked ahead of generic token matches** — verified end-to-end through the real KB search (vsftpd 2.3.4, log4j 2.14.0, gitlab 13.9.0, drupal 7.50, apache 2.4.49 all range-match their writeup at rank 1) — and an **out-of-range** version does not wrongly match (log4j 2.17.0, apache 2.4.58, gitlab 14.0.0 correctly do not). `retrieve()` gained an optional entry-hydrator so the structured `meta.fingerprint` (which the lightweight search index does not carry) is available to the range match; the exact-version substring path remains the fallback.
- **Now wired into the live loop.** 2.7 is no longer only plumbing: `main.py` injects the KB retriever into the orchestrator at startup, and the proposer prompt carries a **FINGERPRINT-MATCHED WRITEUPS** block for the exact service+version fingerprints in state — "this stack is commonly solved via X", with the `solved_via` lead and a citable entry id. Verified live: a `vsftpd 2.3.4` service in state pulls its backdoor writeup into the prompt. Additive and injected: with no retriever wired (the hermetic tests), the block is empty and the prompt is unchanged; it retrieves, and like the rest of the proposer path it runs nothing. The remaining growth is content — more distilled fingerprints — not code.

## Verification

- **Hermetic safety suite** ( `sh backend/run_safety_tests.sh` ) — green after every phase, expanded across all five with `test_phase1_runtime` , `test_state` , `test_scope_hostcheck` , `test_credvault` , `test_corpora` , `test_detection` / `test_detection_safety` (OPSEC channel + blue-view-unchanged), `test_repeater` , `test_tunnels` , `test_report_templates` , persistent-shell containment tests in `test_kali` , Phase 5's `test_terminal` (PTY containment + the sentinel shell provably untouched) and `test_exploits` (version comparison, tiered ranking, executes-nothing), and the Windows backend's `test_winrm` + `test_winrm_safety` (host-locked / no gate bypass / secret never leaks / orchestrator can't auto-run WinRM) with the AD oracle extended to the native Windows variants, plus the post-assessment `test_search_ranking` (substance-gated tier boost + completeness nudge) and `test_codescan_rules` (8-language rule bundle + resolver), plus build #4's six new suites — `test_sliver` / `test_sliver_safety` , `test_obfuscation` / `test_obfuscation_safety` and `test_evasion` / `test_evasion_safety` (no agent path, gated-vs-human-only split preserved, `<listener>` never substituted, the tunnel key never crossing the HTTP boundary, the artifact never executed, and a negative control proving that stripping the OPSEC note makes the engine **refuse** rather than degrade), plus build #5's `test_scans` — the shared source scanner's own regression file, which runs **first** in the suite and plants a violation of each shape (unscanned package, basename-exempted path, aliased import, in-function import, `import_module` string concatenation, f-string) into a temp tree to prove the scanner catches it before ten locks rest on it, and the `I2` follow-on's four new `test_tunnels` cases (a start needs approval + the red-confirm, both server binaries actually trip the heuristic so the danger leg is live, the gated argv is the spawned argv, and the request carries the gate fields defaulting false), plus build #7's `test_prevalidated_gates` — the belt-and-suspenders re-checks on the prevalidated path, proving danger is re-checked in **all three** modes with a windows probe ( `Write-Host go ; Invoke-Mimikatz` ) that only the whole-script classifier catches, so a re-check wired to the generic heuristic fails rather than passing quietly, carrying positive controls that the same request **with** the ack still runs in all three modes and that a benign request is untouched. plus build #7's continuation — `test_lifecycle_safety` , the lock on the shared listener lifecycle (a watched listener exposes **no stdin writer**, because the child is handed a raw pipe fd rather than `subprocess.PIPE` so `proc.stdin` is `None` ; a whole-tree scan with a planted-violation control keeps the handle inside the four modules that own a listener; every branch of the status derivation is exercised including the one that must never claim a listener, an unprobed port; a dead process is reported dead **with the output it died on**; and a stop is asserted to reap the container-side server targeted at its own argv rather than the tool in general), the new per-surface cases for the two gated lifecycles (engagement + approval + red-confirm each refusing with nothing spawned, both server binaries in each pair actually tripping the heuristic so the danger leg is live, `-i` present for the console binary and absent for the daemon, and a dead listener refusing rather than reporting up), and the Sliver build's new contract ( `generated` vs `failed` decided by the artifact read-back, not the console's exit code, asserted in **both** directions), plus build #8's two new suites — `test_reasoning` (the reasoning copilot's plumbing: the ledger records a failed lead and blocks its re-proposal; the hypothesis/expected_signal/citation schema **fails** an uncited proposal; the frontier persists untried leads and recovers a dead-end without resurrecting a killed one; a connection-refused output yields a reachability diagnostic; the **positive control** that a version-mismatched CVE is caught and downranked while the matching version passes; web/AD/privesc specialist routing; fingerprint retrieval outranking a token match; the model-tier lever inert when unset; and the web-exploit + privesc drafts grounded and cited) and `test_substrate` (the Task-1 verdict function separating a landmine from a version banner from an absent binary with a control, the whole pipeline classified over the **real** catalog with an injected container, and the static Dockerfile coverage) — and the extension of `test_loop` itself onto the whole `reasoning/` package (no execution path anywhere in orchestrator + eleven reasoning modules, by AST call-analysis with a control that a payload *string* is not a false positive, and no auto/batch-approve in the proposer path or frontier), and the exploitation-writeup corpus's retrieval-alignment case ( `test_reasoning` ): a structured-fingerprint writeup range-matches an in-range version and floats above a token match, while an **out-of-range** version does not match — the version verdict holding through the corpus. Build #9 then added three regression files/cases for the defects its live fire surfaced — `test_secretargs` (credential values redacted out of the persisted record, per-tool, with nmap-ports and password-with- `@` controls), the `impacket-` -prefix parser-name case in `test_state` , the inherited-rights-never-armed case in `test_adorch_safety` , and the `-dc` -needs-FQDN ordering case in `test_adgraph_collector` . **48 test files, 529 checks, 0 failures.**

- **Live-fire proof** ( `sh docker/proof/live_fire_proof.sh` ) — **66 passed, 0 failed, 3 not-run** against the rebuilt image, after the defects the earlier runs found were fixed and iodine was demonstrated end to end. Passing: an implant that builds through the gated path and a **beacon that calls back**; a **proxychained request routed through the pivot** into an otherwise unreachable subnet, using the argv `wrap_command` produced, with the same tunnel unable to reach `1.1.1.1` ; all three lifecycle gates refusing on all three limbs with nothing spawned; every listener observed bound by `ss` inside the container and still bound at the end of its phase; the pivot subnet entering scope only via the explicit amendment, with the deep target confirmed unreachable first; and containment measured **from inside** the implant host — no internet, no host, C2 only. Also passing: iodine's IP-over-DNS tunnel, through the same gated surface, carrying ICMP with 0% loss and confirmed DNS-encapsulated by a server-side `tcpdump` . Not-run and reported as such: ligolo's interface route (a deliberate won't-do — it needs commands typed into a live C2 console, which HackPit will not do), the dnscat2 client handshake (decisively an upstream defect in the pinned build, above HackPit's proven-bound listener), and a delegated DNS zone (a domain and public listener the operator does not have).
- **The audit's "probed and holds" list, re-run after build #5** — all 41 items still hold, including the `.exe` spellings from `I1` , the four gate orders, the no-silent-downgrade-to-lab path, and Critical 1's eight catalogued tools. The five demonstrated Critical 2 bypasses are all refused. The planted `from cockpit.kali import run_kali` in `adgraph/orchestrator.py` now **fails** the scan (verified against a temp mirror; the repo was never modified). Read-only enumeration — `nmap` , `sqlmap --dbs` , `netexec --shares` , `ldapsearch -x` , `hashcat` , `nuclei` — stays clean on both paths.
- **Docker proofs** — `isolation_proof.sh` 4/4 (lab still cannot reach internet or host), `engage_open_proof.sh` 4/4 (engage has full reach). Both re-verified in build #7 **after** the sandbox stack was rebuilt and recreated from the current Dockerfile — twice, the second time after the `proxychains` fix was added to that Dockerfile — which is a real change to the running containers rather than a no-op re-run.

- **Image build** — `hackpit-kali:build7b` builds from the current `Dockerfile.sandbox` with exit 0 and carries the `proxychains` fix baked in (`socks5 127.0.0.1 1080`, zero `socks4` lines); the stack was rebuilt and recreated from it, and the live fire re-run against that image reports the config as coming from the image rather than from its own runtime patch. `hackpit-kali:build7` builds from the pinned `Dockerfile.sandbox` with exit 0, and every pinned tool resolves and smoke-tests by its real invoked name (`dnscat2-client`, `dnscat2-server` via its four gems, `donut`), with the shipped dnscat2 `HEAD`, gem versions and `donut-shellcode` version confirmed **inside** the image to equal the pins.
- **Browser** — every UI surface exercised against a live Ollama backend per the testing rule: the Phase-4 surfaces (payload-set arsenal rows, the OPSEC red-team channel, repeater send/replay/diff, tunnels route preview, exam report templates with the proof table) and the Phase-5 ones — `:terminal` running `top` and `vim` with live resize, `:exploits` resolving `vsftpd 2.3.4` to the backdoor exploit and its CVE, the state panel's per-service jump into it, and `/category/cloud` at 535 entries. **Windows targets** (`/windows`): profile create/list/test/delete and the AD-walk "run on" picker verified against the backend. As of build #9 the connectivity **test** and a live WinRM round-trip are no longer deferred — they are **demonstrated live** against a real DC through the same HTTP API the UI drives (see the build #9 live-fire below); the browser render of `/windows` and `/cockpit` was confirmed serving 200 while that run executed.
- **Live fire against a real Windows/AD domain** (`backend/livefire_windows.py` → `backend/livefire.log`, build #9) — **45 passed, 0 failed, 3 not-run** against the operator's VMware Server-2022 `corp.local` DC, driving the live backend over its real HTTP API. Demonstrated live: a WinRM round-trip landing on the profile host with the credential absent from every record, the whole-script danger classifier firing on real input, real `bloodhound-python` collection parsing into the typed graph (84 nodes / 625 edges) with the DA path computed on it, a real DCSync returning `krbtgt` through the red-confirm with loot ingesting into state, and the walk advancing only on the approved exit-0 run. It also found and fixed four defects invisible to the hermetic suite (credential-in-record leak, impacket parser-name mismatch, KB grounder arming a free edge, collector FQDN classifier), and truth-checked the detection catalog against the DC's own Event Log — confirming DCSync's 4662/DS-Replication signal (12 of 12) and **correcting two catalog claims the real telemetry contradicted** (secretsdump's mode-specific artefacts, and `ad_collect` raising no 4662). Not-run and reported as such: a Windows-target C2 callback (needs a routable listener), the native `Invoke-Mimikatz` DCSync variant (no offensive tooling on a bare DC), and a model-spontaneous DCSync pick.
- **Frontend** — `tsc` clean, lint at the pre-existing baseline (11 errors + 1 warning, unchanged — verified by stashing the changes and re-running), `next build` exit 0 (routes include `/terminal`, `/exploits`, `/windows`).
- **The launcher, verified against real state rather than a mock** (build #11) — the status rail was exercised in both directions on the same page with no code change: with Docker down it reported `sandbox stack down` with **7 tiles dimmed and 7 `stack down` markers**; after `docker compose up -d` it reported `up · engage up` with **0 dimmed and 7 `needs the stack`**. The freshly created networks were re-checked against the isolation floor — `assert_isolation_proven()` passes and `hackpit-isolated` is `internal=true` while `hackpit-open` and `hackpit-engage` are deliberately not. 4 bands, 15 surface tiles and 30 category cards render live.
- **Operator identity, checked against git itself** (build #11) — `test_operator.py` asserts `backend/operator.json` is ignored by asking `git check-ignore` rather than by reading the ignore file and trusting the pattern, and carries a control proving the predicate can answer "no" for a tracked file. The staged diff was grepped for the real name and identifiers before committing; only synthetic fixtures remained.
- **KB enrichment, measured rather than asserted** (2026-07-31) — the first external-corpus batch is verified four ways instead of by row count: **per-source counts diffed before and after** (2,699 → 2,712, every other source byte-for-byte unchanged), the KB file confirmed to still exist afterwards (Defender has deleted it before), **6 of 7 natural-language retrieval probes returning the new entry at rank 1**, and the full safety suite at 52 files. A first suite run aborted at `test_corpora.py` and passed on re-run and standalone; the cause is environmental and

is recorded rather than hidden — `pipeline/ingest_corpora.py:139` shells to `git show HEAD:<file>` expressly "to recover AV-locked / dehydrated files", and rewriting the 22 MB `entries.jsonl` triggers a Defender sweep that briefly locks corpus sidecars.

- **KB enrichment batch 2 — 24 cert/CTF/pentest repos** (2026-08-01) — verified the same four ways, and the numbers are small on purpose. **Per-source counts diffed before and after**: 2,712 → 2,714, all 22 sources still present and every one of the other 21 unchanged to the row; `pivoting` 6 → 8. The KB file, `embeddings.npy` and `ids.json` all confirmed present afterwards (22,487,315 bytes; Defender has deleted the KB before, so it is backed up first and re-checked after). **Retrieval: 6 of 6 natural-language probes surfaced a new entry in the top 5, four of them at rank 1** — including the ones a command reference cannot answer ("the second pivot host cannot reach my attacking machine, how do I get a shell back", "why does nmap find nothing through proxychains"). Full hermetic suite green at **52 files, every one exit 0**; `test_corpora.py` aborted the first run and passed standalone and on re-run, the documented Defender/sidecar flake, recorded rather than hidden. Saturation was measured rather than asserted: 308 terms appear in ≥3 repo files and never in the KB, and all 308 are URL slugs, hostnames, playlist IDs, filenames or typos. Arsenal: 110 → 115 tools, all 188 catalogued invocation names carrying a pinned danger verdict, all 286 templates across 115 tools rendering target-faithfully with no foreign host. All five new binaries were confirmed **present and runnable inside the live sandbox image** before being catalogued, rather than assumed from the Dockerfile. All 1.6 GB of clones deleted afterwards, with `git status` confirming the manifest is the only thing in `sources/` that git can see.
- **KB enrichment batch 4 — the 0xdf fingerprint corpus** (2026-08-01) — unlike batches 1–3 this one **added** rows, so the verification is that it added exactly what it claimed and touched nothing else. Per-source counts diffed before/after: **only `hackpit-distilled` moved, 78 → 97 (+19); no other source lost a single row**, and a total alone was not accepted as proof. `data/kb/entries.jsonl` still exists (22.5 MB, 2,733 rows) after both the ingest and the embed — the Defender-quarantine check that has bitten before. `embed.py` reports **19 new, 2,714 cached**, so the vector space grew by exactly the new entries. Retrieval was run through `search.search(entries, query, top)` (positional, no `k=`) on realistic scan strings — `OpenSMTPD smtpd 6.6.1 25/tcp`, `Apache Tomcat 9.0.27 8080`, `Apache Solr 8.3.1 wt velocity`, `jetdirect printer 9100 snmp`, `IPMI 623 BMC RAKP`, `gitlab 11.4.7 redis ssrf`, `netdata ndsudo setuid`, `xwiki solr search groovy`, `pgadmin query tool 9.1`, `mariadb wsrep 10.3.25` — and each new fingerprint ranked first or top-three for its own banner while the controls `vsftpd 2.3.4` and `Apache 2.4.49` still won theirs. `sh backend/run_safety_tests.sh` → **52 test files, every one exited 0**, `test_corpora` included and with no flake this run.
- **KB enrichment batch 3 — 7 GitBook certification spaces** (2026-08-01) — the verification here is of a **zero**, which is a different claim and needs different evidence: not "the ingest did no damage" but "no entry was warranted." The KB is byte-unchanged at **2,714 rows**, and no ingester was run. Saturation was established at three independent levels. **Index level, before any page was requested**: the 545 page URLs the six reachable spaces publish were tokenised against all 2,714 rows, and of 109/88/59/37/56/166 slug words per space, the words absent from the KB number **6/5/3/1/2/2** — every one an author name, a certification acronym, or a typo (`foundamentals`, `accross`, `uncostrained`, `gnereral`). **Page level**, for the one space fetched in full: 175 distinct leading commands across its 214 code blocks, of which 10 appear nowhere in the KB and all 10 resolve to covered material. **Concept level**, for the two survivors that a token diff called new — and this is the leg that mattered, because `seshutdownprivilege` (11 occurrences, 0 in the KB) is a real technique that the KB **already carries under a different spelling**, `ex-windows-checking-services` step 4: "*reboot if you hold SeShutdown.*" A token miss is not a coverage miss. Full hermetic suite green at **52 test files, every one exit 0**, run after the batch; `test_corpora.py` did not flake this time, which is consistent with its documented cause — no 22 MB KB rewrite happened to trigger the Defender sweep. Politeness is verifiable rather than claimed: `robots.txt` is parsed and honoured **before the first request**, and it refused a space during this batch rather than in theory.

- **Fingerprint retrieval — measured, then fixed, then re-measured** (2026-08-01, `docs/FINGERPRINT-EVAL.md`) — a measurement session first drove the real 2.7 path (`orchestrator._fingerprint_reference` → `reasoning.retrieval.retrieve`) against a test set that was **not** drawn from the corpus alone: 30 covered service+version strings in real `nmap -sV` formatting, 15 near-miss versions outside the stated range, 15 uncovered services. It found the corpus firing at only **70%** on covered banners with two structural defects behind the misses (D24), and — the number that mattered — **0% false-fire on near-miss**, the safe direction. The fix session then lifted covered hit rate to **93%** at **96%** precision, took the exact-boundary case from **0/4 to 4/4** and the corpus-wide self-match from **62/97 to 97/97**, **held near-miss false-fire at 0%** (a version one step above the boundary still does not fire, 0/3 — the change is inclusive `<=` at the boundary and nothing looser), and left the CVE→exploit index **provably byte-identical** (110,695/110,695 verdicts unchanged, `test_exploits.py` green). Two residuals were left deliberately and named rather than folded in: a 20%-unchanged uncovered false-fire from the version-less substring fallback (a different code path), and 2 covered misses that are a base-retriever top-5 recall limit, not a fingerprint one. Locked by two new suites that iterate the real corpus with positive controls — `test_fingerprint_versions.py` (every fingerprint matches its own stored version) and `test_fingerprint_norm.py` (`Apache Tomcat` ≠ `Apache httpd`, no collision). **54 test files, every one exit 0.**
- **Fingerprint retrieval — the last residual closed** (2026-08-01, D25) — the one gap the fix session named but left, a **20% false-fire on UNCOVERED services**, is now fixed and re-measured on the same three groups. It was the version-less substring fallback in `rerank()` setting `fingerprint_match=True` on a bare product-name substring; the eval was instrumented before the change to prove the fallback contributed **0 of the 28 covered fires** and was the **whole** of the false-fire, so demoting it (reserve `fingerprint_match` for structured hits; a product-name hit becomes a labelled, lower-ranked `fallback_match`; word-boundary match) cost nothing on hit rate. Measured before/after on all six: **UNCOVERED false-fire 20% → 0%**, COVERED hit rate **93% (unchanged)**, precision **96% (unchanged)**, NEAR-MISS **0% (held)**, corpus self-match **105/105 (held)**, fire-above-boundary **0/45 (held)** — a clean removal, not a stricter-matcher trade. Locked by `test_fingerprint_fallback.py` (live corpus + 15 real uncovered services + positive control, can-fail proven by monkeypatch). **55 test files, every one exit 0.** KB untouched — the fix is entirely in `reasoning/retrieval.py`.
- **KB enrichment batch 3 — PDFs, pages, and the CPENT repo: the series close-out** (2026-08-01) — five sources, **2 entries added**, verified the four required ways. **Per-source counts diffed before/after**: only `hackpit-authored` moved, 27 → 29; every other source unchanged to the row, and the category deltas are exactly `ics 1 → 2` and `recon 69 → 70`. `data/kb/entries.jsonl` confirmed present after both ingest and embed (the Defender-quarantine check; the KB was backed up first, then the backup deleted once integrity was confirmed). `embed.py` reports **2 new, 2,741 cached** — the vector space grew by exactly the two entries. **Retrieval through `search.search(entries, query, top)` (positional, no `k=`)**: both new entries rank **first** on all four probes a command list cannot answer — "nmap scan an OT network with fragile embedded PLCs safely", "scan an ICS SCADA network without crashing controllers", "map firewall rules and enumerate an ACL with nmap", "bypass a packet filter with fragmentation decoys and spoofed source port". Saturation of the four zero-yield sources was measured, not assumed: every candidate technique (printer pass-back, IKE aggressive mode, open mail relay, ligolo/chisel/sshuttle pivoting, employee OSINT, o365 spray) grepped as already covered, several better than the source covers them — the two entries that landed came from the CPENT repo's two thinnest modules naming a scan-safety discipline the KB had never written down. Full hermetic suite green at **55 test files, every one exit 0**; `test_corpora.py` aborted the first run on the documented Defender/sidecar flake (the 22 MB KB rewrite) and passed on re-run. The 185-page PNPT space, both ebook PDFs and the parzival blog post all yielded **zero**; raw trees deleted, `sources/pdfs-pages-manifest.md` the only residue.

## Status

Build #4 (AV/EDR evasion + traffic obfuscation) is complete and verified: image `hackpit-kali:build4` builds with all eight new binaries smoke-tested by invocation, the full hermetic suite was green at 42 files / 446 checks at that point, and the frontend builds clean. Tasks 8–13 have now had their independent review (see Part II) — no containment hole, five substantive defects fixed, each with a regression test. **Two caveats are carried forward as open items in Part II rather than papered over**, and build #4 should not be called field-ready until they close: the C2 and tunnel surfaces have never been run against a live beacon, a real delegated DNS zone, or an instrumented Windows host, so their *efficacy* claims are documented rather than demonstrated (the *containment* claims are what the suite locks, and those hold without live infrastructure); and three installs in the image layer are still unpinned, so that layer is not yet byte-reproducible.

A note the review sharpened, because it cuts against the instinct to trust the tooling: `pipeline/detection_sources.py --verify` reported **0 problems** throughout, including while `evasion_etw_blind` carried a technique id about shell history. It verifies that every id, name, tactic, log source and Sigma reference matches live upstream — it cannot verify that the technique chosen is the right one for the activity being described. A clean `--verify` is a check on transcription, not on judgement.

**Build #5 (gate integrity) is complete, and the audit's one remaining important item is closed with it.** All three gate-audit criticals are now closed: Critical 1 in build #4's fix wave (`4492fec`), and Criticals 2 and 3 here. `I2` — the ungated tunnel listener start, outside the plan's scope — was recorded open, then fixed as a follow-on in the same build. The suite stands at **43 files / 477 checks / 0 failures**, the frontend builds clean, and the audit's full "probed and holds" list was re-run with nothing regressed. What changed is not only three guards but the conditions under which a guard is allowed to be believed: `backend/AGENTS.md` now requires a safety test to iterate real data, assert on what it actually checked, and carry a demonstration that it can fail — and `test_scans.py` runs first in the suite to hold the shared scanner to that standard before ten locks depend on it. The remaining work is the deferred list in Part II — chiefly live-fire verification of the C2 and tunnel surfaces, and the three unpinned installs in the image layer.

**Build #7 (live fire, image pinning, prevalidated danger re-check) is complete, and it is the first build whose headline result was a set of defects rather than a set of features.** Two of its three tasks closed cleanly: the image layer is byte-reproducible and `hackpit-kali:build7` builds green, and the prevalidated path now re-checks danger in all three modes. The third — live-fire verification, Part II's largest outstanding item — did what it was supposed to do. It proved the gates fire against real binaries and proved containment holds live from inside the contained host. It also found that **three of the surfaces did not work as shipped**: the Sliver implant argv named a subcommand that does not exist, and both listener lifecycles reported `status="listening"` for a process that had already exited. None was a containment failure — nothing escaped and no gate was skipped — but they meant the efficacy claims those surfaces carried were never true. Two of the three were found only because a real build and a real process replaced a structural assertion; the pinning task independently surfaced a smoke test that had never passed for the same reason.

**All three are now fixed, and the design question the run left open is decided (2026-07-28).** The implant builds through the console `sliver-server` hosts and its success is decided by reading the artifact back rather than by a console exit code; both listeners hold a console binary's stdin open — as a raw fd with no writer object, so the surface gains liveness without gaining the ability to type into a live C2 console — and report a status they observed, with a dead process now a refusal rather than a fiction. The live fire stands at **66 passed, 0 failed, 3 not-run**: a beacon calls back, a proxychained request routes into an otherwise unreachable subnet, iodine's IP-over-DNS tunnel carries traffic confirmed DNS-encapsulated on the wire, and every gate still refuses on every limb. The stale precedent is resolved by tightening rather than by deleting the citation: the DNS-tunnel listener is gated exactly as `I2` gated the pivot listener, because a covert channel *is* an exfil route; the Sliver server is gated too, on the narrower ground that its config can persist listener jobs that come up with the daemon, so "starting it is inert" is a property of a config file rather than of this code. Human-only is untouched and remains the load-bearing property; the gates are belt to those suspenders.

Fixing them surfaced two more defects of the same family, both also fixed: a stop that killed the `docker exec` client while the server kept running and holding its port inside the container, and a `proxychains` config shipped pointing at Tor rather than at the SOCKS5 port HackPit's own rewrite targets — a shipped path that every unit test passed and that could not have worked. A second pass then verified both of those against a rebuilt image rather than against a diff — the `proxychains` layer now builds and the config is baked in, and the stop-reap was demonstrated live against the failure that motivated it, including that reaping one tunnel leaves a neighbouring one bound. That pass surfaced a sixth defect of the same family: a status observed once and never re-observed upward, which left a listener that bound just after its settle window permanently `starting` and therefore permanently unroutable. It now refreshes in both directions, and only on evidence. That is the durable lesson of this build, and it is worth stating plainly: **six defects in these surfaces were invisible to a green test suite, and every one of them was a claim asserted structurally that nothing had ever executed.** The suite now stands at **45 files, 502 checks, 0 failures**, with `test_lifecycle_safety` locking the new shared lifecycle — including the positive controls that a dead process is reported dead and that an unprobed port is never called listening. Two of the four not-runs then closed: iodine's DNS tunnel is demonstrated end to end, and dnscat2's failure is pinned decisively to the upstream build rather than to anything HackPit owns. What remains genuinely undemonstrated is stated as such and not folded into a pass: a delegated DNS zone (an operator prerequisite), ligolo's console-driven interface route (a deliberate won't-do), and detonation on an instrumented Windows host.

**Build #8 (the reasoning copilot + substrate proof) is complete.** It did the two things it set out to do without moving the safety boundary an inch. The substrate claim is now a *measured* one — **97 of 104 catalogued Linux tools actually execute** in the live image under its real profile (93.3%), with the seven that do not reported honestly as two `sudo`-wrapper landmines and five genuine absences, and the probe surfaced two real catalog defects (Sliver/Empire installing under a different binary name) that are now fixed and forced through the danger gate under their real invoked names. The orchestrator is a genuinely deeper proposer — working memory that stops it re-proposing dead leads, hypothesis-first proposals with an enforced citation gate, a scored candidate frontier with dead-end recovery, failure diagnosis, a refute-first critic that kills version-mismatched CVE hallucination, specialist routing, fingerprint retrieval, and a model-tier config lever — and **the honest ceiling is stated plainly: reasoning depth is mostly the base model, so this moves the loop up the curve on easy and known-pattern-hard, not to "solves anything."** The one line that did not move is the one that matters: **the loop reasons but never executes, and approval stays per-command** (D18), re-locked by extending the source-scan onto the whole `reasoning/` package and by a scan proving no auto/batch-approve exists in the proposer path or the frontier. The suite stands at **47 files, 519 checks, 0 failures**; the loop was validated live against Ollama (proposals cited, reacting to prior output, not re-looping, every command still stopping for human approval).

The follow-on the build flagged is now **landed**: the **KB exploitation-writeup corpus** that 2.7's fingerprint retrieval keys into. `pipeline/ingest_exploitation_writeups.py` seeds 78 distilled, non-verbatim technique writeups keyed by service+version fingerprint (the CVE-index `version_kind` / `versions` shape, so the version verdict stays authoritative), additively and idempotently (byte-identical re-run, `category="writeup"` / `no_merge` so consolidate never duplicates them, file confirmed un-quarantined; KB 2,621 → 2,699). Operator writeups import at runtime into gitignored engagement data and never reach the repo. Retrieval alignment is verified end-to-end through the real KB search: an in-range version returns the seeded writeup ranked ahead of token matches, and an out-of-range version does not wrongly match. The corpus is intentionally a seed — growing it further stays an ongoing content task, not a code one.

**Build #9 (the Windows/AD path, driven live against a real domain) is complete.** It is the build that moved the Windows/AD path from "built and locked hermetically" to "demonstrated live," and — like build #7 before it — its lasting value was as much in what a real target exposed as in what it confirmed. It confirmed a great deal: a WinRM round-trip locked to the profile host, the whole-script danger classifier firing on real input, real BloodHound collection parsing into the typed graph, and a real DCSync returning `krbtgt` through the red-confirm with loot ingesting into state — the gates holding at every step, on real input, 45 passed / 0 failed / 3 not-run. It also found **four defects the green hermetic suite could not see, every one a place where two halves of the codebase agreed in a test and disagreed against a real domain**: a domain credential persisted into the run record (and thence into the LLM proposer context), impacket loot ingesting as nothing because the catalog and the parser spelled the tool differently, the KB grounder arming an inherited-rights edge with a destructive command, and a failure classifier that misdiagnosed the collector's FQDN error. All four are fixed and regression-locked; the suite stands at **48 files / 529 checks / 0 failures**. Task 5 was the first time the detection describe-side was checked against real telemetry rather than documentation — it confirmed DCSync's high-fidelity 4662 signal and corrected two catalog claims the DC's own Event Log contradicted. What stays not-run is stated plainly and not folded into the pass: a full Windows-target C2 callback (a routable listener the sandbox does not expose by choice), and the native-tooling abuse variants (offensive tooling on a bare DC, out of scope). The durable lesson repeats build #7's in a new domain: **a live target is the only thing that finds the bugs that live between two components that were each tested alone.**

**Build #10 (the opt-in C2 exposure override, and an honest NOT-RUN) — complete.** Build #9's Task 4 recorded a Windows-target C2 callback as NOT-RUN for one concrete reason: the Sliver/tunnel listener lives in `hackpit-engage-sandbox`, which by standing invariant publishes no ports, so the lab DC had no route to it. Build #10 supplies exactly that missing route **without widening the default posture**: `docker/proof/c2-lab.yml`, an opt-in compose override that publishes a single port — `192.168.13.1:53/udp`, the iodine tunnel server's DNS port bound to the one host interface (VMnet8) the lab DC can reach — and nothing else, never a wildcard. It is never applied by the documented bring-up; `backend/test_exposure_safety.py` locks three invariants (default compose publishes nothing, every override port names a literal host IP, nothing composes it in implicitly) and plants a violation to prove the scan can fail. The override is not only built but **demonstrated live** — the exposure came up on `192.168.13.1:53/udp` and tore down cleanly. On it sits a gated C2/AD proof harness (`docker/proof/c2_0{1..4}_*.sh` + `c2_lab_proof.sh` orchestrator + in-process driver) built on build #7's discipline: offensive command strings are never inlined — they are operator-supplied paste values read by file path, and an empty value reports **NOT-RUN**, never a fake pass. The four offensive proofs it drives — staging the tunnel client on the DC, the iodine DNS tunnel, a Sliver beacon over it, and a native DCSync behind a scoped Defender **exclusion** (not a real-time-protection toggle) removed in a `trap` guard — are **NOT-RUN**: harness-ready but requiring operator-supplied commands that were deliberately not filled, recorded as such rather than folded into a pass. A final hardening pass closed the build's engineering items and corrected one of its own claims: the suspected `run_safety_tests.sh` gating hole did **not** exist (`set -e` has been in the committed runner all along, and a planted failure was verified to abort it), while the failing `nc` danger-gate test turned out to be an environment leak rather than an over-block — the one test in `test_session.py` that called the gates outside the hermetic fixture, so it reached a live Docker probe. Both are now fixed and the runner additionally checks every test's exit code explicitly. The build also added a read-only DC prerequisite probe and a harness-honesty regression test, and that test immediately caught a real defect: three of the four proof scripts' `[[PASTE]]` slots were not actually empty, so an unfilled proof was one WinRM round-trip away from being scored as a genuine attempt. Suite: **538 checks across 50 files, green**. The four offensive proofs remain **NOT-RUN**.

**KB enrichment batch 2 (24 cert/CTF/pentest repos) — complete, and its honest result is two entries.** 24 repositories cloned, **22 yielded nothing**, one produced both entries. Under D21 that is the process working: the KB was already saturated in what these repos cover, and 308 candidate "new" terms turned out to be URL slugs and filenames rather than techniques. The batch's scoping prediction was **wrong in a way worth recording** — CPENT was expected to carry IoT and ICS/SCADA and carries neither; across all 24 repos exactly one file mentions ICS vocabulary and it is a CTF challenge named "Scada" that is really Jinja2 SSTI. `iot` (2) and `phishing` (1) remain the thinnest KB categories and now have a *reason* attached: exam notes are the wrong source class for them. The two entries fill the one genuine gap the batch did expose — every existing `pivoting` row is a command reference, and none teaches multi-hop chain discipline. The unplanned find was worth more than the planned one: the arsenal had **zero** pivoting tools despite `cockpit/tunnels.py` driving chisel, ligolo and proxychains since Phase 4, and cataloguing `proxychains` surfaced a **pre-existing gate hole** — the danger heuristic classified `argv[0]` only, so wrapping a dangerous command in a tunnel wrapper stripped its red-confirm, making the gate weaker the deeper you reached. Fixed in the predicate and regression-locked through `validate_request` (D22). Suite: **52 files, all green**; KB 2,712 → 2,714; arsenal 110 → 115. All 1.6 GB of clones deleted; `sources/repos-manifest.md` is the reproducibility record and required a deliberate, single-file gitignore exception to survive.

**KB enrichment batch 4 (the 0xdf fingerprint corpus) — complete, and written at the time as the series close-out** (superseded: batch 3 was restored as the actual closer and has since run — see the batch-3 records above, which are authoritative for the series' completion). This was the first non-cert-note source, and the first batch to add rows: **19 entries from 12 posts, `hackpit-distilled` 78 → 97, KB 2,714 → 2,733**, all in the thin categories a fingerprint corpus is for (services, pivoting, credentials, persistence, reversing/privesc). The standing ban on 0xdf was reviewed and reversed to the project-wide distil-not-parrot rule (D23), the ingester docstring was corrected to match, and the `consolidate.py` link-index skip was deliberately left alone. Phase 1 cost two requests and mapped 614 posts / 276 CVE tags, **175 of them never before in the KB** — the measurement that told us this source was unlike the cert notes before a page was read. 55 posts were selected by KB gap and fetched (Defender deleted 5 mid-triage; triage tolerated it and re-fetched), every candidate was concept-grepped before writing, and that grep killed ~43 shortlisted posts as already-covered and caught two pure slug collisions ( `schallenge` → `XSSChallengeWiki` , `wsrep` → `wsrepl` ) that a token count would have miscalled — the same `SeShutdown` trap batch 2 found. **The series, whole: five batches, of which the PDF/pages batch has NOT run** (it is written but never executed, and is not counted as complete). The four that ran added 34 entries (13 + 2 + 0 + 19). **The headline is the cert-note yield — 31 sources produced 2 entries** — a real measurement of the KB's maturity on exam-syllabus material, set against 19 from one narrative source that supplied the scan→root shape nothing else did. Carried forward: the token diff **nominates, never confirms** (grep the concept); themastermindnotes **declined** as a commercial product; `mqt.gitbook.io` **refused itself** by robots.txt; 0xdf **declined-then-reversed**. KB now at 2,733; suite 52 files all green.

**0xdf pass 2 (2026-08-01) — the one amendment to that close-out.** Batch 4 read as the series end, but the retrieval eval it prompted found the corpus half-firing and the fix ( `6c3ba42` ) made new entries fire, so a final novel-CVE pass ran: **8 fingerprints from 51 posts** (HFS 2.3, IIS 6.0 WebDAV, Icinga Web 2, OpenTSDB, SQLPad, ES File Explorer, Next.js middleware bypass, Strapi), `hackpit-distilled` 97 → 105, KB 2,733 → 2,741, 0xdf total **27**. The yield is a declining tail (19/55 → 8/51) into a residue of web-CMS boxes the KB already covers, so **0xdf is now recommended closed as a fingerprint source**. Measured, not assumed: all 8 landed in `category="writeup"` (the ingester forces it), **not** the thin `services` / `pivoting` categories — correcting pass-1's claim about where its rows went; and the eval's UNCOVERED false-fire residual **held at 20% (3/15), zero delta**. Suite **54 files, all green** (the two fix-session locks now validate all 105 fingerprints).

**KB enrichment batch 3 (7 GitBook certification spaces) — complete, and its honest result is zero entries.** Batch 2 ended by predicting that four of these seven spaces were near-certain duplicates; the prediction held, and the two it named as genuine unknowns resolved against it too. **Five of the seven were never fetched at all** — four because an index-level gate settled them from their published page lists alone, and one because its operator has opted out of machine collection. Of the two fetched, one was taken only as its 15-page delta over a space already mined, and that delta turned out to be an Active Directory chapter that has been outlined but not written (1,058 words, **zero code blocks**, 24-word pages). The remaining space was fetched in full, deliberately, because its hypothesis was *methodology structure* rather than technique novelty and no index gate can test shape — 20,248 words and 214 code blocks, and it yielded nothing either: the KB already carries **118 `checklist-*` entries** in exactly that shape. The build's carry-forward is a method, not a technique. The batch's one plausible find, `seshutdownprivilege`, was absent from the KB as a *string* and present as a *technique* under the spelling `SeShutdown` — so the token diff that batch 2 established as proof-of-saturation is now bounded by an explicit rule: **it can only ever nominate candidates, never confirm a gap; the concept has to be grepped before a zero or an entry is claimed.** Under D21 this is the process working: 21,306 words were read and nothing was written, because writing anything would have meant duplicating rows already present. KB unchanged at 2,714; suite **52 files, all green**; `sources/gitbooks-manifest.md` is the reproducibility record.

**Fingerprint retrieval (2.7) is measured, fixed, and locked (2026-08-01).** The eval that closed the 0xdf series asked the obvious unasked question — does the fingerprint corpus actually fire on real scanner output — and the answer was "half the time," behind two defects (D24): a shared version predicate with an unstated boundary convention that made **35 of 38 versioned fingerprints miss the exact version they were written about**, and a normaliser that collapsed `Apache Tomcat` and `Apache httpd` onto one key. Both are now fixed at the **predicate**, not the caller: `_version_verdict` names its boundary ( `inclusive=` ) so the CVE index (exclusive, fix-version) and the fingerprint corpus (inclusive, last-vulnerable) each state their own, and the CVE→exploit index is provably untouched (**110,695/110,695 verdicts identical**). Covered-banner hit rate went **70% → 93%** at **96%** precision; the exact-boundary case **0/4 → 4/4**; corpus-wide self-match **62/97 → 97/97**; and the safety number held — **near-miss false-fire stayed 0%**, with a version one notch above the boundary still refusing to fire. This is the **third shared-predicate defect** in the project (WinRM `argv[0]`, D22's proxychains laundering, now this), and it is called out as a pattern to watch. Two new regression suites iterate the real corpus with positive controls; **54 test files, all green**. Two residuals are named not hidden — a version-less substring-fallback false-fire and a base-retriever top-5 recall limit — neither blocking. No KB entry was touched: this was a matcher fix, and a second 0xdf pass is now worth running at the corrected rate.

**The last retrieval residual is now closed too (D25, 2026-08-01).** A follow-on session fixed the version-less substring fallback that was the whole of the 20% UNCOVERED false-fire: `fingerprint_match` is now reserved for a structured `meta.fingerprint` match, and an unstructured product-name hit is a distinct, lower-ranked `fallback_match` that never claims the exact stack (word-boundary matched). The decision was measurement-led — the fallback was shown to contribute **0 of the 28 covered fires** before it was demoted, so **UNCOVERED false-fire went 20% → 0% with covered hit rate, precision, near-miss and self-match all unchanged**. `test_fingerprint_fallback.py` locks it; the suite is now **55 files, all green**. That closes the last known correctness gap in the 2.7 path. Separately, the enrichment series' last batch (batch 3 — PDF/pages/CPENT) has since **run and closed the series** — see the batch-3 Status paragraph below and the *Batch 3 — the close-out* record.

**KB enrichment batch 3 (PDFs, pages, CPENT) — complete, and it closes the series.** The last batch ran its five sources — two redistributed cert-notes PDFs (OSCP, eCPPT), a 185-page PNPT GitBook space (mis-scoped in its own prompt as a "single web page"; its sitemap resolved to 185 pages, so it went through `fetch_gitbook.py` ), an OSCP-notes blog post, and the CPENT cheat-sheet repo — and added **2 entries**, both distilled (D21) from the CPENT repo's two thinnest modules: `authored-perimeter-filter-mapping` [recon] (reading a packet filter as an object of enumeration — ACK/window scans, firewalking, scan-time evasion with the decoy caveat) and `authored-ot-safe-scanning` [ics] (scanning an ICS/OT segment without faulting a controller — never `-sV` / `-A` , `-sT` over `-sS` , `--max-parallelism 1` , abort path in the ROE). Both were **genuine gaps confirmed by retrieval**, not by a token count: `--spoof-mac` / `--data-length` / `firewalk` / `--mtu` / `nmap -f` and `--max-parallelism` were all 0 hits pre-ingest, and the `ics` category held one entry that was an operator persona. The four cert-*notes* sources — the two PDFs, the blog, the 185-page PNPT space — yielded **zero**, every candidate already covered (printer pass-back, IKE aggressive mode, open mail relay, pivoting, spraying, employee OSINT), several better than the source. **The prediction held:** batch 3 was five more cert-note sources, predicted ~0–1 entries before the run; it yielded 2, close enough that the lesson stands. The final scoreboard: **44 entries across the whole series; cert notes 36 sources → 4 entries vs narrative writeups 106 posts → 27** — the ~20× per-source difference that is the series' reusable lesson. The thin categories the series set out to fill still barely moved ( `ics 1→2` , `recon 69→70` , `pivoting 6→8` ; `services` / `credentials` / `persistence` / `phishing` / `iot` unchanged), and this record says so. KB 2,741 → 2,743; suite **55 files, all green** ( `test_corpora` flaked on the 22 MB rewrite and passed on re-run, as documented). Records carried forward: the token diff **nominates but never confirms** (its top-two batch-3 nominations were both already covered); a prompt's description of a source's **scope is a claim to verify** (PNPT was 185 pages, not one); and **Defender deleted a file mid-run a fourth time** ( `bypassing-amsi.md` , errno 22 while still on disk) — now a standing repo hazard, not an anecdote. `sources/pdfs-pages-manifest.md` is the reproducibility record; the raw trees are deleted.

## Build #9 — the Windows/AD path, driven live against a real domain (2026-07-31)

Everything in HackPit's Windows/AD path was built and locked **hermetically**: `test_winrm` / `test_winrm_safety` monkeypatch the WinRM transport, and the AD graph, orchestrator and technique catalog were all exercised against `adgraph/sample_data.py` — a GOAD-shaped synthetic domain. Nothing had ever touched a real domain. Build #9 stands up the operator's own lab (VMware Windows Server 2022, promoted to a `corp.local` forest, WinRM over HTTP:5985, `Administrator@CORP` ) and drives the **live** backend over its real HTTP API against it. It adds no new capability and no autonomy: every live command clears the existing gates and the human approves each one; the orchestrator proposes, it never fires. The evidence is `backend/livefire.log` , produced by `backend/livefire_windows.py` — a by-hand harness that is deliberately **not** part of the hermetic suite (which stays runnable with no VM, no network and no `pywinrm` ). It records every check as PASS / FAIL / NOT-RUN, and a check that could not run is NOT-RUN, never a silent pass — the whole point being that "demonstrated live" and "still needs X" are different claims. **45 passed, 0 failed, 3 not-run.**

**Task 1 — the WinRM round-trip and the gates on real input (14/14).** A real PowerShell command runs on the DC over WinRM and comes back with the box's own identity ( `WIN-990RALNGERV` , `corp\administrator` , `corp.local` ) — the proof the host-lock held, since the request carries no host field and the destination is resolved server-side from the profile id. An unapproved command is refused live at the approval gate; a foreign host named in the args cannot redirect the run (it is echoed *by* the DC, which is where the run still landed); and build #5's whole-script danger classifier fires **on real input** against `Write-Host go ; Invoke-Mimikatz` — the exact argv[0]-vs-whole-script case that motivated it — then clears once the human acks. The credential appears in none of the run record, the event stream, or the recorded command line; the run is audited and retrievable by id; and real WinRM stdout ingests into state (an OSCP-style proof flag attributed to the profile host). This is the live half of the WinRM deferral, closed.

**Task 2 — the AD graph off real collection (10/10), not `sample_data` .** The gated, approved, scope-locked `bloodhound-python` collects the **real** `corp.local` over the engagement executor — 84 nodes, 625 edges, the live DC's own computer object present — parses into the typed graph, and the route to Domain Admin is computed on that real graph. The collector preview redacts the credential; an unapproved collection is refused live at the approval gate; and an **off-scope DC is refused live at the target gate**. This closes both "AD off synthetic data" and "touch a real AD domain."

**Task 3 — the live abuse walk (12/12, 2 not-run).** The orchestrator proposes edges off the real graph; the human approves; the executor runs them. A real **DCSync** ( `impacket-secretsdump -just-dc` ) executes against the domain and returns real credential material — four NTLM hash lines including `krbtgt` — after the danger red-confirm is demanded and supplied; the dumped credentials ingest into state ( `administrator` , `krbtgt` , `win-990ralngerv$` ); and the walk advances **only** on that approved, exit-0 run (advancing a commanded edge with no run is refused). A native Windows AD action ( `Get-ADGroupMember 'Domain Admins'` ) runs on the DC over WinRM. Two edges are recorded NOT-RUN, honestly: the model happened to pick `GenericWrite` over DCSync on the run, so the destructive step was operator-directed rather than model-selected; and the native `Invoke-Mimikatz` DCSync variant cannot run because a freshly promoted Server 2022 carries no offensive tooling, and staging some onto the DC was out of scope — the Linux/impacket variant is what executed.

**Four defects the live run found that the hermetic suite could not — all fixed, each now regression-locked.** Every one was a claim that two halves of the codebase agreed on in a test but disagreed on against a real domain:

- **The domain credential leaked into the persisted record.** The WinRM path never puts a secret on a command line — the profile resolves it server-side — but every credentialed Linux tool ( `bloodhound-python -p …` , `impacket-secretsdump DOM/user:pass@host` ) carries it as an argv token, which was stored verbatim in the run record, and from there into rendered reports **and the LLM proposer context**. So collecting a domain shipped the DA password to the model. `cockpit/secretargs.py` now redacts credential *values* out of the argv at the record boundary (per-tool, never a blanket flag list — `-p` is a password to netexec and a *port list* to nmap), wired into both `RunRecord` constructions. A first cut split the impacket positional on the *first* `@` and leaked a password fragment out the "host" side; the fix anchors the host to the last `@` . `test_secretargs.py` locks both the positive cases and the negative controls (nmap ports survive untouched), including a password-with- `@` fragment check.
- **Real DCSync loot ingested nothing.** The AD technique catalog proposes Kali's `impacket-secretsdump` ; the state parser registry was keyed only on upstream's `secretsdump` / `secretsdump.py` . Two halves disagreeing about one tool's name meant four dumped hashes ingested as zero credentials. `state/ingest.py::program_name` now strips the `impacket-` prefix; `test_state.py` locks that every spelling reaches the same parser (and that `impacket-wmiexec` does *not* become a secretsdump parser).
- **The KB grounder armed a "free" edge with a destructive command.** `MemberOf` and `HasSIDHistory` are inherited rights — nothing to run — but their KB seeds still matched a real ACL-abuse entry, whose example commands were adopted wholesale. On the real graph the orchestrator therefore proposed a runnable `net rpc password` (a destructive reset) for `ADMINISTRATOR -MemberOf-> DOMAIN ADMINS` , an edge with no action, rendered `resolution="ready"` . The gates all held — nothing fired — so this is a correctness lock, not a containment one, but a panel that asks a human to approve a domain change for a free step is how a hurried operator makes an unintended one. A `no_command` flag on the spec keeps the KB *citation* while refusing to let it manufacture an *action*; `test_adorch_safety.py` locks it with an adversarial grounder that always offers a destructive command, plus a positive control that an actionable edge is still groundable.
- **The collector's failure classifier had nothing to say about the FQDN error.** `bloodhound-python` refuses an IP for `-dc` ("looks like an IP address, but requires a hostname (FQDN)"), and that same message also mentions a "DNS server IP", so the generic DNS signature matched it and told the operator to fix their nameserver when `-dc` was the real problem. A signature for it now runs **first**; `test_adgraph_collector.py` locks the ordering.

**Task 4 — Windows-target C2 (feasibility, measured; not run).** Build #7 demonstrated Sliver + iodine against Linux. Completing that against this Windows target needs a callback path from the DC to the listener, and the harness **measures** rather than assumes one: the DC reaches the host's VMnet8 gateway and has DNS egress, but the Sliver listener lives in `hackpit-engage-sandbox` , which publishes no ports and sits on a Docker bridge the DC has no route to. Recorded NOT-RUN with the exact unblock (publish the listener port from the sandbox to the host, point the implant at the gateway) and the reason it was not done here: publishing a port changes the sandbox's network posture, and this build's standing invariant is that it adds no new exposure.

**Task 5 — the detection footprint, truth-checked against real telemetry (8/8) — the purple-team validation the describe-side had never had.** The catalog's footprint claims were written from documentation; this reads the DC's own Event Log back after techniques that really ran. The audit policy is read first (so "not observed" is distinguished from "never audited"), and events are counted as **before/after deltas keyed on `EventRecordID`** — the VM's clock drifts and w32time hauls it back, so a timestamp window silently lands in the future and reads as "the box logged nothing" (an earlier iteration of the harness "disproved" claims that were in fact correct, exactly this way; the log also flushes asynchronously, so the query settles on the DC first). Confirmed against real telemetry: DCSync raises Security **4662 with the DS-Replication GUIDs** — 12 of 12 events carried them — under the DC's **default** audit policy, so "loud" is the right rating; and T1003.006 is the right id. **Two catalog claims the real box contradicted, now corrected (atomic with `attck.py`, per the detection trap):** (1) the `secretsdump` telemetry list lumped SAM/LSA-mode artefacts (5145 admin-share access, 7045 short-lived service, hive saves) together with the `-just-dc` path — a live `-just-dc` run added **zero** of them, because it replicates over DRSUAPI and touches no share, service or hive; the entry now says which mode each artefact belongs to. (2) the `ad_collect` entry promised "Security 4662 … if object auditing is enabled" — but on a DC that auditing is on by default, and a full `-c All` collection added **zero** 4662 (4662 needs a SACL on the objects read; LDAP enumeration produces network logons, not directory-access events); the entry now points at LDAP diagnostics (1644) as the dependable host signal, while the "loud" rating stands on query volume. `test_detection` / `test_detection_safety` stay green after both edits.

The honest bottom line: the WinRM round-trip, real AD collection, the real DA path and the executed DCSync abuse are **demonstrated live**, with the gates holding on real input; a full Windows-target C2 callback and the native-tooling abuse variants still need more lab infrastructure (a routable listener, offensive tooling staged on the DC), and are recorded as not-run, not as passed.

## Build #10 — the opt-in C2 exposure, and an honest NOT-RUN (2026-07-31)

Build #9 closed the Windows/AD path live but left one item explicitly NOT-RUN: a full Windows-target C2 callback. The reason was precise and worth not papering over — the Sliver/tunnel listener runs inside `hackpit-engage-sandbox`, which by standing invariant publishes no ports, so the lab DC (on VMware's NAT subnet) had no route to it. Closing that needs a change to the lab's exposure posture, and build #9's rule was "no new exposure by default." Build #10 supplies the missing route as an **opt-in** and does the surrounding engineering; it does **not** fire the offensive proofs, and says so plainly.

**The exposure override — built, locked, and demonstrated live.** `docker/proof/c2-lab.yml` is the one place HackPit publishes a host port, and it is never applied by the documented bring-up — reaching it takes a second, explicit `-f docker/proof/c2-lab.yml`. It publishes exactly one port, `192.168.13.1:53/udp`: the iodine tunnel server's DNS port, bound to the single host interface the lab DC can see (the VMnet8 address), never a wildcard that would also expose it to whatever network the laptop is on. `backend/test_exposure_safety.py` locks three invariants hermetically — (1) the default `docker/docker-compose.yml` publishes zero ports; (2) if the override exists, every published port names a literal host IP and never `0.0.0.0` / `::` / `*`; (3) nothing else in the repo composes the override in implicitly — and it plants a wildcard-bound port and proves the scan catches it, so a silently-broken scanner cannot make the checks vacuous. The mechanism is not only tested but **demonstrated live**: the exposure came up on `192.168.13.1:53/udp` and tore down cleanly, leaving no published port behind.

**The proof harness — the same honesty discipline as build #7's live fire.** On top of the override sits a gated C2/AD proof harness: four scripts (`c2_01_dc_prereq_stage.sh` staging the tunnel client + TAP driver on the DC, `c2_02_iodine_tunnel.sh` the DNS tunnel, `c2_03_sliver_beacon.sh` a beacon over it, and `c2_04_dcsync_defender_excl.sh` a native DCSync behind a **scoped Defender exclusion** — `Add-MpPreference -ExclusionPath` for the tool path, removed in a `trap` guard, *not* a real-time-protection toggle), plus an orchestrator (`c2_lab_proof.sh`, the only script permitted to compose the override up and down) and an in-process driver that reuses build #9's credential redaction. The load-bearing property carried over from build #4/#7:

**offensive command strings are never inlined in the harness.** Each is an operator-supplied paste value handed to the driver by file path; an empty or unfilled value makes the proof report **NOT-RUN**, never a fake pass. Every offensive step stays behind the existing engagement + approve-each + danger red-confirm gates; no new autonomous path is added.

**What is NOT-RUN, and why it is recorded as such.** The four offensive proofs were **not fired**. They are harness-ready but need operator-supplied commands (the iodine client invocation, the Sliver implant generation and launch, the DCSync one-liner), and those were deliberately not filled — so the run reports NOT-RUN across all four, which is the honest state, not a failure. "The harness is built and safe" and "the offensive path was demonstrated end to end" are different claims, and only the first is made here.

**The safety harness, investigated — and one of this build's own claims withdrawn.** Wiring the new exposure test in raised a suspicion that the runner itself was not gating: `backend/run_safety_tests.sh` appeared to invoke every test file without checking any exit code, which would mean a failing safety test could let the suite exit 0 and print "passed." **That claim was wrong, and it is worth recording as wrong rather than quietly dropping.** `set -e` is present in the committed runner and has been since before this build; planting a deliberate failure and running the suite produced exit 1, a halt at the second test file, and no "passed" banner. The suite always gated. What *is* a real weakness is that `set -e` is silently suspended for any command inside a pipeline, an `if` / `while` condition, or an `&&` / `||` chain — so a later edit as innocent as `"$PY" test_x.py | tee log` would disarm the whole suite with no visible change in output. The runner now routes every test through a `run_test()` helper that checks the exit code explicitly, names the failing file and the invariants it guards, states how many files had passed before the abort, and reports the file count on success. A guard that can be switched off by accident is not one a safety suite should rest on, even when it happens to be working.

**The `nc` danger-gate failure: an environment leak, not an over-block.** `test_session.py`'s `test_start_trips_the_danger_heuristic` failed on unmodified code with "nc must start once explicitly acked" — `validate_start` refusing `nc -lvnp 4444` even *with* `dangerous_ack`. The danger/target/scope logic turned out to be correct and the engagement/Wall-A state clean (the request resolved to `lab` mode, no active engagement). The real cause is the LAB gate order — target → approval → danger → **isolation** — where that last gate calls the live `assert_isolation_proven()` Docker probe. The *un-acked* half of each assertion pair short-circuits at `danger` and never reaches it; the *acked* half falls all the way through, so on a host with the stack down it came back `gate="sandbox"` and the test failed for a reason with nothing to do with the heuristic it guards. It was the only test in the file that called the gates outside the `_Spy()` fixture that stubs that probe — contradicting the module's own "Hermetic: ... no Docker daemon" contract. Fixed by running it inside the fixture like every other test in the file, with the gate order written into the docstring so the next person does not re-learn it from a red suite.

**Harness honesty, locked — and a real defect caught doing it.** `backend/test_proof_honesty.py` locks the property the whole NOT-RUN story rests on: an unfilled offensive slot can never become a fake pass. It checks four independent angles — the reader treats empty/whitespace/comment-only as unfilled; driving the real driver subcommands with an empty slot emits NOTRUN, emits no PASS, and never reaches the WinRM transport (asserted with a tripwire, not assumed); an AST pass proves every `_read_paste` call site is followed by a NOTRUN empty-guard, so a *new* slot added later without one fails the suite; and the slots in the four committed scripts really are unfilled — plus a control that feeds it a filled slot to prove the checks can fail. Writing it caught a genuine defect: `c2_01`'s two slots and `c2_04`'s DCSync slot did **not** ship empty. They opened with a self-referential shell fragment ( `echo $(cat …| grep …| cut …)` ) left from an earlier placeholder sweep, sitting *outside* the first pair of quotes. That text is not a comment, so `_read_paste` returned it as live content, the empty-guard never fired, and the driver would have sent it to the domain controller as PowerShell and scored its exit code as a real attempt — an unfilled proof reporting FAIL, or conceivably PASS, instead of NOT-RUN. The slots are now comment-only and the regression is locked by a test verified to fail when it is re-planted.

**The DC prerequisite probe, finalised.** `docker/proof/dc_prereq_probe.py` answers whether the C2 path even has the pieces it needs — adapter inventory, staged tooling, DNS/HTTPS egress, route to the VMnet8 gateway, Defender posture, OS/PowerShell version — before anyone runs a proof. It is read-only and *structurally* so: every probe is checked against a read-only cmdlet allow-list that **fails closed**, so a probe added later with a mutating verb makes the file refuse instead of run. The stale hardcoded profile id was removed ( `--profile` is now required, so it can never fire at whatever host an old id happens to name) and a `--list` mode prints the probes while contacting nothing. It was **not** run against the DC in this session.

**Suite state.** `sh backend/run_safety_tests.sh` exits 0 with **538 checks across 50 test files** — now genuinely gated, verified by planting a failure and confirming the suite goes non-zero, names the offending test, and prints no "passed" banner.

## Build #11 — the launcher, and operator identity (2026-07-31)

Two small builds, both closing gaps that had been visible for a while and neither moving the safety boundary.

### A dozen surfaces were built and then invisible

The top nav carries five product sections and deliberately stays that size. Everything else — `:arsenal` , `:c2` , `:tunnels` , `:windows` , `:repeater` , `:scripts` , `:code-scan` , the AD graph — was reachable only by typing the URL. This was not a new discovery: the project's own working notes had flagged it twice (2026-07-26 and 2026-07-27) without it being fixed.

The launcher is 15 tiles in four bands — **plan**, **operate**, **infrastructure**, **reference** — merged into the home page per D19. The band label is not decoration: it states the posture of everything under it ("*proposes · never executes*", "*every command human-approved*", "*gated start · human-only stdin*"), so the page cannot show a shell tile without also saying who approves it.

`CategoryGrid` is now categories only. It had been carrying **five** product cards — attack-paths, Cockpit, the scripts arsenal, the tool arsenal and code scan — every one of which is now a launcher tile; keeping them would have shown each surface twice on one screen. `ScriptsCard` , `FEATURED` and `COCKPIT_FEATURE` are left in the tree unreferenced rather than deleted, so restoring the old layout is a revert of one file.

### The status rail answers a question the UI could not

Every execution surface shells out to `docker exec` and refuses with "*bring the stack up*" when the container is down. That state was only discoverable by clicking into a surface and reading an error. `GET /home-summary` now reports it up front — stack, LLM model, WinRM target, active engagement — and the tiles that need the stack dim when it is down.

Two details that are the point rather than polish. A **null** probe renders as "*unknown*", never as "*down*": an undeterminable probe must not send the operator chasing a container problem they do not have. And the endpoint is deliberately **separate from `/stats`**, because it runs docker probes and folding it in would gate the hero's counters behind a container inspect.

It lives in `main.py` because it is cross-cutting by construction — sandbox probe, LLM config, Windows profile store, engagement store, KB counters — and `cockpit` and `arsenal` may not reference each other.

### The report could not identify its own author

`backend/report.py` had **no author, candidate or OSID field anywhere** — every "author" match in it was the word "*authoritative*". An OSCP submission is not attributable to a candidate without a name and an OSID, so the report this tool generates could not be submitted without hand-editing identification in. For a project whose tagline is "*pass the cert*", that is a defect, not a missing nicety.

Fixed per D20, with the block spliced under the report title the same way evidence and the CVSS block are.

### A mistake worth recording

The identity module was first written as `backend/operator.py`. `backend/` is first on `sys.path`, so that filename **shadows the stdlib `operator` module for every import in the process**. It was caught immediately and renamed to `operator_identity.py`; `test_operator.py` now asserts `backend/operator.py` does not exist and that `import operator` still resolves outside the repo. It is recorded here because the failure mode is silent, process-wide, and would have been very hard to attribute later.

### What is locked

`test_home_summary.py` and `test_operator.py`, both in the suite. Between them: no secret reaches the browser (checked against the **real** stores — stored WinRM secrets, the configured LLM key, every credential in state — and reporting per-source counts so an empty leg is *visible* rather than absorbed into a reassuring total); the endpoints call only the masked accessors, asserted over the **AST** rather than a substring; a status endpoint cannot reach an execution path; the operator config is gitignored, asserted by asking git; and the OSID and email never reach the browser. Every check carries a positive control, and the leak test asserts it cannot go vacuous.

**Suite state.** `sh backend/run_safety_tests.sh` exits 0 across **52 test files** (50 → 51 → 52). Frontend lint holds at the pre-existing baseline of 11 errors + 1 warning; new tiles stagger via CSS `animation-delay` specifically so as not to add more `react-hooks/set-state-in-effect` errors, and `next build` exits 0.

**Build #11 (the launcher + operator identity) — complete.** Neither half moved the safety boundary; both closed a gap that had been visible and unaddressed. The launcher's value showed up immediately and by accident: bringing the Docker stack up during verification flipped the rail from `down` to `up · engage up` and un-dimmed seven tiles with no code change, which is the whole feature demonstrated end to end. The report identity block closes a real submission blocker — the generated OSCP report previously carried no candidate identification at all. One near-miss is recorded in full above (a module named `operator.py` shadowing the stdlib) because the failure would have been silent and process-wide.

## KB enrichment — the first external corpus, and what it cost to read it (2026-07-31)

A 687,830-character corpus of live bug-bounty stream transcripts was folded into the KB. It is recorded here because the *ratio* is the finding, and it sets the expectation for every batch that follows.

**11% of the file was exact duplication** — two sections were byte-identical copies of two others, caught by hashing before any reading effort was spent on them. **A further 44% (nine Hindi-language sections, 268,048 characters) yielded exactly one usable technique.** That was established by counting technique vocabulary on the Devanagari transliterations rather than by sampling: across all 268k characters the corpus contained 57 mentions of price tampering, 15 of SQLi, 2 of CSRF — and **zero** mentions of XSS, IDOR, open redirect, subdomain takeover, rate limiting, brute force, path traversal, Burp, nuclei or dorking. The remainder was stream management and narration.

So roughly a third of the corpus carried essentially all of its value. The lesson is not that the source was bad — the English third was excellent — but that **volume is not a proxy for content, and measuring density before reading is cheap.**

**Thirteen entries were written**, each checked against all 2,699 existing rows first. They cluster in three places the KB was genuinely empty:

- **Targeting** — hunting the most recently *added* wildcard (platforms publish scope changes as a dated diff, and a wildcard added weeks ago cannot have been fully tested); hunting the asset with the *fewest* resolved reports rather than the most; `origin-*` hosts that serve the same application with the edge WAF removed; and the cheap staleness signals that decide which of several hundred hosts is worth attention.
- **Method — STRIDE threat modelling**, which closes the gap between "recon is finished" and "what do I actually test": decompose into mechanisms, walk each through Spoofing / Tampering / Repudiation / Information disclosure / Denial of service / Elevation of privilege, and emit a ranked attack-vector list. The KB previously held two passing mentions of threat modelling and no method at all, which is a structural gap rather than a content one — HackPit's planner had no model of this phase.
- **Web** — repudiation and audit-trail attacks (the most overlooked STRIDE class: a role able to edit the logs it is audited by, with impact argued through the compliance regime); brute forcing through the change-password endpoint, where login is rate-limited and the change-password path usually is not; CSRF token *binding* failures rather than missing tokens; client-trusted entitlement flags, with the discipline that a UI which unhides is not an authorization bypass until the server honours it; and letting the framework fingerprint decide the test set — verb tampering is futile against Express and worth doing against Django/PHP.

`pipeline/authored/authored_entries.jsonl` is the one committable artifact; the raw corpus lives under the gitignored `sources/` tree and never enters this public repository.

**KB enrichment is now a measured process rather than an additive one (2026-07-31).** The governing rule is D21: distil, never parrot; check every candidate against the whole KB before writing; report zero when a source yields zero. The first batch under that rule took a corpus that was one-third signal and produced 13 entries aimed squarely at the categories that were thinnest, while deliberately adding nothing to the ones already carrying hundreds of rows. The next batch — 23 cert-study and CTF-writeup repositories — is scoped in `PROMPT-kb-repo-ingest.md` and is expected to yield most of its value in `pivoting`, `credentials`, `persistence` and `iot`, with several repositories predicted in advance to yield nothing at all.

## KB enrichment — 24 cert/CTF/pentest repos, and a prediction that was wrong (2026-08-01)

The batch scoped above ran. **24 repositories were cloned, 22 yielded nothing, one produced the batch's only two KB entries, and one contributed to them without earning an entry of its own.** That is the headline, and under D21 it is a correct outcome rather than a disappointing one.

**The prediction the batch was scoped on was wrong, and the way it was wrong is the useful part.** CPENT was expected to be the highest-yield group, because the KB is thinnest in exactly what CPENT claims to cover — IoT (2 entries), ICS, and pivoting. It covers none of it in practice. The study guide has only modules 05 and 06 written; modules 09 (wireless), 11 (IoT) and 12 (SCADA/ICS) are `[TBD]` stubs with no content behind them, and the companion repo is a six-file link list. Widening the check from that repo to the whole batch settled it: across **all 24 repositories**, exactly one file mentions ICS vocabulary (modbus, s7comm, dnp3, scada, plc, profinet, HMI, Purdue) four or more times, and that file is a CTF challenge *named* "Scada" which turns out to be Jinja2 SSTI. There is no

ICS, no IoT-hardware and no firmware/UART/JTAG material in this batch at all. `iot` (2) and `phishing` (1) are still the KB's thinnest categories, and they need a different kind of source — vendor and ICS-CERT advisories, teardown writeups, protocol specifications — not more exam notes.

**Saturation was measured, not assumed.** Beyond grepping each candidate technique against all 2,712 rows, every word in the batch was tokenised against the whole KB. **308 terms appear in three or more repository files and never once in the KB — and on inspection all 308 are URL slugs, lab hostnames, YouTube playlist IDs, filenames or typos.** Not one new tool name, not one new technique name. That is the quantitative form of "these repos are saturated," and it is a far stronger claim than reading a sample and forming an impression. Individually probed and confirmed already covered: DCSync (47 rows), mimikatz (90), LSASS dumping (34), password spraying (43, including lockout thresholds), Kerberoasting, chisel (19), proxychains (22), sshuttle (7), mitm6/DHCPv6/WPAD relay, IPsec/IKE, VoIP/SIP, Jinja2 SSTI, and the SEH / bad-character / `jmp esp` stack-overflow workflow.

**Where the two entries came from.** One repository — `eCPPTv2-PTP-Notes`, with the TryHackMe Wreath notes from `PNPT-study-guide` as a second source — carries a worked three-network, three-hop pivot lab. The KB's six existing `pivoting` entries and its pivot cheat sheet are all *command references*: here is `ssh -L` syntax, here is chisel syntax. None of them teaches the discipline that keeps a chain standing, which is a different kind of knowledge and the one thing in the batch the KB genuinely lacked. Two entries were written to fill it (`pivoting` 6 → 8, KB 2,712 → 2,714):

- **Multi-hop pivot chains.** The asymmetry every naive chain dies on: hop 1 can reach you, hop 2 cannot, so each hop needs an outbound path *and* a return path built by separate commands — a SOCKS proxy per hop on its own port, and a socat relay chain carrying callbacks back. Plus the parts that are only obvious in hindsight: stage tooling from hop N-1 rather than from your box (hop 2 cannot fetch from you), stage *static* binaries (a pivot host often has no interpreter and no package manager), verify each hop before adding the next, scope the subnet before routing into it, and tear down in reverse while recording what you left behind.
- **Choosing the pivot primitive.** Three properties decide it and none of them is preference: direction (egress decides forward vs reverse, not taste), shape (one port vs SOCKS vs a layer-3 TUN), and footprint. Including the SOCKS tax that costs people hours — a SOCKS proxy relays completed TCP only, so `-sS` and ICMP host discovery return a silence that reads exactly like "host is down" — and the quiet relay that moves both listeners onto the attacker's box so the compromised host has **nothing listening on it at all**.

**The arsenal gap this exposed was not in the plan, and was the more valuable find.** HackPit's `cockpit/tunnels.py` has driven chisel, ligolo and proxychains since Phase 4; all five pivot binaries were confirmed present and runnable in the live sandbox image — and the 110-tool catalog contained **zero** pivoting tools. Five rows were added in a new `pivoting` category (chisel, ligolo-ng, socat, sshuttle, proxychains), 110 → 115.

**Cataloguing proxychains then surfaced a real gate hole that predated it.** The danger heuristic classified `argv[0]` only. Bare `weevely` produced a red-confirm reason; `proxychains -q weevely …` produced **none** — and `wrap_command` builds precisely that argv for the human to approve. So routing a command through a tunnel *stripped its red-confirm*: the deeper into a network you went, the weaker the gate got, which is exactly backwards. The fix is in the predicate, not in the file set — wrappers are peeled off and the command that actually runs is classified, with the wrapper kept visible in the reason (`through proxychains: weevely: turns a vulnerability into …`). It is regression-locked by `test_a_wrapper_cannot_launder_the_red_confirm`, which asserts through `validate_request` rather than through the predicate, because the claim is about the *gate*; it carries the `-f <config>` case (skipping the flag but not its value would read the config path as the command) and the negative half, that a benign inner command must **not** raise a confirm.

`sshuttle` was classified into the same bucket as chisel and ligolo on what it does — it puts a whole subnet inside reach — rather than on how quiet it is; needing no agent and no listener makes it the stealthiest of the three, not the least dangerous.

**One gitignore exception was taken, deliberately.** `/sources/` was ignored wholesale, which meant `sources/repos-manifest.md` — the record of which commit of which repository produced which entry — could never be committed and would vanish with the clones. The pattern is now `/sources/*` with a single negation for that one file; it holds our own prose about third-party repositories and never their content, and `git status` was checked afterwards to confirm it is the only thing in the tree git can see.

**Also worth recording, because it was on disk and is not any more.** `ciwen3/PNPT` (307 MB, the largest clone in the batch) contains a `conti/` directory holding leaked Conti ransomware operator manuals in Russian, `rclone.exe`, and a Cobalt Strike 4.3 archive. Nothing from it was read into an entry and nothing from it could ever have been committed — `sources/` is gitignored — and all 1.6 GB of clones were deleted after verification, as the process requires. It is noted here because "we cloned 24 arbitrary repositories" has a supply-chain shape worth being explicit about.

**`0xdf.gitlab.io` is deferred with a concrete proposal, not skipped.** It is a website, not a repository, and was correctly excluded from this batch. It is also the highest-quality HTB writeup source in existence, and the KB already holds 41 `htb-writeups` + 173 `writeup` entries, so the first step is not fetching — it is **overlap measurement**: pull the sitemap, diff the box names against those 214 rows, and establish how many boxes are genuinely absent before writing an ingester. If the answer is meaningful, the shape is the existing sitemap-driven PortSwigger ingester (enumerate → fetch → distil → authored entries), never a crawler. Recorded as a follow-up.

## KB enrichment — 7 GitBook certification spaces, and a zero that took three probes to earn (2026-08-01)

Seven GitBook spaces of personal certification notes were supplied — two eCPPT, three CRTP, one OSCP, one pentesting checklist. **All seven yielded zero entries, five were never fetched, and the KB is byte-unchanged at 2,714 rows.** Under D21 that is the correct outcome and not a failed run: certification notes are the single most duplicated material in this KB, and the batch was scoped expecting it.

**What is new here is the gate, not the yield.** Batch 2 fetched 24 repositories in full and *then* discovered 22 were duplicates. This batch inverted that: the page lists were pulled from each space's own sitemap and tokenised against all 2,714 KB rows **before a single content page was requested**. Across the six reachable spaces that is 545 published pages, and the slug words absent from the KB number six, five, three, one, two and two respectively — `foundamentals`, `accross`, `uncostrained`, `gnereral`, plus author names and the acronyms `crtp` and `ecppt`. Four spaces were closed on that evidence and never fetched. The cost of settling three CRTP spaces was three sitemap requests.

**One space was resolved by path, not by inference.** `dev-angelist.gitbook.io/ecpptv2-ptp-notes` was suspected of being the GitBook publication of `github.com/dev-angelist/eCPPTv2-PTP-Notes`, mined in batch 2 at commit `a543e9167445`. Suspicion is not evidence, so it was checked: the space publishes `network-security/2.4-1/2.2-pivoting` and `…/2.2-pivoting-1`, byte-identical to the repo paths `repos-manifest.md` records as the source of both authored pivoting entries. Same content, same author, two surfaces. Not fetched.

**The two spaces batch 2 named as genuine unknowns both resolved against the prediction.**

- **`ecpptv3-ptp-notes` was the one expected to pay.** 75 of its 90 distinct page slugs are shared with the same author's already-mined v2 space, so only the 15-page delta was fetched — and the delta is the reason it yields nothing. It is an Active Directory chapter that has been outlined but not written: **1,058 words across 14 pages, zero code blocks**, median 23 words per page, with `6.1.4-ad-enumeration` at 25 words and `6.1.7-ad-persistence` at 24. Its one substantial page is a conceptual introduction to users, groups and OUs, against a KB holding 123 `active-directory` entries. eCPPTv3 is eCPPTv2 plus a stub.

- `pentesting-checklist` **was fetched in full on purpose**, 113 pages / 20,248 words / 214 code blocks, because its hypothesis was methodology *structure* rather than technique novelty and no index gate can test shape. Three probes, all negative. Word novelty: 10 words appear three or more times and never in the KB, eight of them the author's name and lab hostnames — batch 2's exact pattern. Tool novelty: 175 distinct leading commands, 10 unknown to the KB, all 10 resolving to covered material (PowerView cmdlets against 47 entries, `rpcclient` subcommands against 23, `vshadow.exe` as an alternate shadow-copy binary against `diskshadow` and `tool-sebackupprivilege`, a Linux capability string against `ht-linux-capabilities`). And the structure hypothesis itself failed: **the KB already carries 118 `checklist-*` entries** — 51 Active Directory, 21 privesc, 21 recon, 19 enumeration, 4 credentials, 2 persistence — against a space that is 74 Windows/AD pages, 17 Linux and 5 pivoting, the latter naming the same primitives batch 2 catalogued a day earlier.

**The finding worth carrying forward is a correction to batch 2's own method.** The token diff nominated `seshutdownprivilege` — 11 occurrences in the checklist, zero anywhere in 2,714 KB rows, and unlike the slugs and hostnames it is unmistakably a real privilege-escalation technique. It was the batch's one credible entry, and it is already in the KB. `ex-windows-checking-services` step 4 reads "*Stop the service, check its start mode, reboot if you hold SeShutdown*" and carries `shutdown /r /t 0`; `ht-service-triggers` covers the harder form of the same hinge, starting a service without `SERVICE_START` rights by firing its trigger rather than rebooting the host. The KB spells it without the `Privilege` suffix, and that one suffix was the entire "gap". So the rule batch 2 established is now bounded: **the token diff nominates candidates and can never confirm a gap.** A zero it produces is only trustworthy once the *concept* has been grepped, and an entry it motivates is only safe on the same condition. The second survivor, `setakeownership`, failed the same way against a dedicated `oscp-setakeownershipprivilege` entry.

**One space was refused rather than skipped, and the refusal is in code.** `mqt.gitbook.io` serves a `robots.txt` carrying `Content-Signal: ai-train=no` and a blanket `Disallow: /` for ClaudeBot, GPTBot, CCBot, Google-Extended, Applebot-Extended, Bytespider, Amazonbot and meta-externalagent. `pipeline/fetch_gitbook.py` parses that before its first request and skips the space; its sitemap was never requested. We are none of those user-agents, and the check is not one we could be compelled by — which is exactly why it belongs in the fetcher rather than in a person's judgement at fetch time. The space was independently a likely zero: all four OSCP repos in batch 2 yielded nothing.

`pipeline/fetch_gitbook.py` **follows** `fetch_portswigger.py` **and improves on it in one respect.** Sitemap-driven with the GitBook `sitemap.xml` → `sitemap-pages.xml` indirection resolved per space rather than assumed, serialised with a delay, honest User-Agent, fetch-once-to-disk. The improvement is that it does not scrape HTML at all by default: GitBook serves every page as markdown at `<url>.md` and advertises it in-page beside an `llms.txt` index, so the fetcher asks for the format the publisher chose to hand to machines — a third of the bytes, real fenced code blocks, no nav chrome. The `<main>`-to-markdown parser remains as the fallback. One sharp edge is handled explicitly: a missing page is served as HTTP 200 with a `# Page Not Found` body, so without a check a renamed page lands on disk as a stub and gets triaged as though it were content.

**An operational note, because it is the third time this has bitten.** Windows Defender deleted two fetched pages mid-triage — `enumeration/ports.md` (8.4 KB of port-enumeration commands) moved from `OSError 22` to `OSError 2` between two reads, and an LFI/RFI page followed. The same signature-on-our-own-examples behaviour has deleted `data/kb/entries.jsonl` before; it now reaches the fetched source tree as well. Triage was made tolerant of unreadable files rather than retried, 111 of 113 pages were analysed, and both lost pages sit in saturated categories (`network-services` 183, `web` 642).

The fetched tree is deleted. `sources/gitbooks-manifest.md` is what remains — URL, fetch date, pages published, pages taken, and the verdict for each of the seven, under the same single-file gitignore negation the repo manifest uses.

## KB enrichment — the 0xdf fingerprint corpus, and the series close-out (2026-08-01)

This is the last batch in the enrichment series, and the first that was **not** cert notes. The target was `hackpit-distilled`, the 78-entry corpus that keys a service+version fingerprint to the technique that solves it and that build #8's 2.7 retrieval ranks ahead of generic token matches — 78 entries is thin, and every `nmap` result the cockpit parses is a fingerprint that either hits that corpus or falls back to fuzzy prose. `https://0xdf.gitlab.io` is several hundred long-form HTB/CTF machine writeups, each starting from a scan result and ending at root: a service→technique mapping in narrative form, which is exactly the shape a fingerprint corpus needs and the one shape 31 sources of cert notes never supplied.

**The policy came first (D23).** The ingester's docstring named 0xdf as never-to-draw-from; that rule was reviewed by the operator and reversed to the project-wide distil-not-parrot rule, and the docstring was rewritten to match. The `consolidate.py:2324` link-index skip was deliberately left alone. None of the 55 fetched pages nor any 0xdf prose is committed — the committable artifacts are `fetch_0xdf.py`, the policy fix, the distilled entries and the manifest.

**Politeness, and why the index was cheap.** `robots.txt` is a 404 — no restrictions published — which the fetcher checks on every run and would refuse on. `fetch_0xdf.py` takes URLs only from the site's own `sitemap.xml` and its `/tags/` page, never from crawling, and serialises requests behind a 1.8s delay (slower than the PortSwigger fetcher on purpose — this is one person's blog on GitLab Pages). **Phase 1 cost exactly two requests**: the sitemap (614 dated posts) and the tags page, which is the whole archive's metadata in one 2.8 MB document — 614 posts across 3,981 tags, and **276 distinct CVE tags of which 175 had never appeared anywhere in the KB.** That 175 is the number that separated this source from the cert notes before a single writeup was read: the 31 cert-note sources re-covered a syllabus the KB had already absorbed, and this one did not.

**Phase 2 was selected by KB gap, not by recency.** Each post was scored `3×network-service tags + 2×thin-category tags + 4×KB-absent-CVEs - web-app tags`, and the top **55** were fetched — 55/55, no failures. Windows Defender then did exactly what batch 2 warned it now does: it deleted **5 of 58 files mid-triage** (`OSError 22` → `OSError 2`), on the KB's own web-shell-signature behaviour reaching the fetched tree. Triage was written to tolerate a file vanishing between listing and reading — it counted 53 analysed and named the 5 lost rather than crashing — and a re-fetch restored them.

**Yield: 19 entries from 12 posts, `hackpit-distilled` 78 → 97, KB 2,714 → 2,733.** Every candidate was grepped by **concept** — synonyms, stems, tool names, technique words, not the token — against the whole KB before it was written, per batch 2's correction. That grep did real work: it killed ~43 of the 55 shortlisted posts whose standout CVE was already covered (runc fd-leak escape was present via `cred-toctou`; Azure AD Connect decrypt, fail2ban, vm2 escape, Office/RTF CVE-2017-0199, polkit CVE-2021-3560, rsync 873, and the Craft/Openfire/Jenkins-CLI CVEs were all already in), and it caught two would-be false positives that were pure slug collisions — a `SeShutdown` -

style trap each: the KB's only `schallenge` hit was `XSSChallengeWiki`, and its only `wsrep` hit was `wsrepl`, a WebSocket tool. One candidate (CrushFTP / htb-soulmate) was **dropped** because Defender deleted its page twice — the rule is that if you cannot read it, you do not understand it well enough to write it.

The 19 landed in the thin categories a fingerprint corpus is for, not the saturated web tree: OpenSMTPD MAIL-FROM injection, SaltStack master auth-bypass, Tomcat FileStore deserialization, Solr Velocity RCE, OFBiz XML-RPC deser, Cacti graph_realtime SQLi, CUPS ErrorLog read, printer 9100/PJL+ SNMP credential leak, MariaDB wsrep RCE, IPMI RAKP hash leak, GitLab SSRF→Redis RCE, Vim modeline RCE, Rails cache-store deserialization, XWiki SolrSearch Groovy RCE, Netdata ndsudo PATH privesc, Firejail `--join` privesc, ManageEngine SAML RCE, pgAdmin Query-Tool RCE, and the Squid CONNECT pivot. Retrieval was confirmed against realistic scan strings (`OpenSMTPD smtpd 6.6.1 25/tcp`, `Apache Tomcat 9.0.27 8080`, `jetdirect printer 9100 snmp`, ...): each new fingerprint ranks first or top-three for its own banner, and the existing controls (`vsftpd 2.3.4`, `Apache 2.4.49`) still win theirs, so nothing was displaced.

### The series, whole (all batches)

Checking `git log` and the manifests in `sources/` for what actually landed rather than assuming an order, the enrichment series ran as six batches — batch 3 (this one) is the close-out:

| Batch | Source | Sources | Entries | Note |
|---|---|---|---|---|
| Transcript corpus | live-hunting transcripts | 1 corpus | **13** | set D21; first external corpus |
| Batch 1 — repos | 24 cert/CTF/pentest repos | 24 | **2** | both from one repo; exposed D22 |
| Batch 2 — GitBooks | 7 GitBook cert spaces | 7 | **0** | index-gate; bounded the token diff |
| Batch 4 — 0xdf | 0xdf machine writeups | 55 posts | **19** | first non-cert source |
| Batch 4b — 0xdf pass 2 | 0xdf, the novel-CVE tail | 51 posts | **8** | run after the retrieval fix; tail |
| Batch 3 — PDFs/pages/CPENT | `PROMPT-pdf-and-pages.md` | 5 | **2** | the close-out; ran 2026-08-01 |

**Batch 3 has now run and closes the series.** Its five sources — two redistributed cert-notes PDFs (OSCP, eCPPT), one OSCP-notes blog post, one 185-page PNPT GitBook, and the CPENT cheat-sheet repo — yielded **2 entries**, both distilled from the CPENT repo. The series added **44 entries total** (13 + 2 + 0 + 19 + 8 + 2).

**Amendment — 0xdf pass 2 (2026-08-01).** Batch 4 was written as the series close-out, but the retrieval eval it triggered found the fingerprint corpus half-firing, and the fix session that followed made new entries worth writing (`6c3ba42`: covered hit rate 70%→93%). So a second, final 0xdf pass ran against the remaining novel-CVE posts. It re-pulled the index (158 of 276 CVE tags still KB-absent, down from 175 as pass-1's 19 closed part of the gap), shortlisted 58 service- weighted posts, and — after Defender ate the same 7 pages on every fetch — concept-grepped 51. **Yield: 8 fingerprints** (HFS 2.3, IIS 6.0 WebDAV, Icinga Web 2, OpenTSDB, SQLPad, ES File Explorer, Next.js middleware bypass, Strapi), taking 0xdf to **27 fingerprints total** and `hackpit-distilled` to 105 (KB 2,741). The rate is a declining tail (19/55 → 8/51) and the remaining unfetched novel-CVE posts skew to web-CMS boxes the KB's 636 web entries already cover, so **0xdf is now recommended closed as a fingerprint source** — a pass 3 would likely yield <5, and only web-app entries. Two things measured rather than assumed: all 8 rows landed in `category="writeup"` (the ingester forces it), **not** the thin `services` / `pivoting` categories — correcting pass-1's report of where its rows went; and the eval's known **UNCOVERED false-fire residual held at 20% (3/15) with zero delta**, so the added substrings did not worsen the version-less fallback.

**The stark, honest number is the cert-note yield: 36 sources produced 4 entries.** Twenty-four repositories, seven GitBook spaces, two ebook PDFs, one blog post and the CPENT repo — the single most duplicated material in this KB — yielded four entries between them (two pivoting from batch 1, two scan-discipline from batch 3), each cluster from a single repo. That is not a failed series; it is a genuine measurement of the KB's maturity on exam-syllabus material, and it is why the series was worth running to its end: it told us, with evidence rather than assertion, that the KB is *saturated* on cert notes and *not* saturated on the service→technique fingerprint shape. The same effort spent on 0xdf — one source, narrative machine writeups — returned 27 entries against 4, because it supplied a shape nothing else in the KB did. The lesson for any future batch is to weigh the **source class**, not the source count: 36 derivative syllabus sources are worth less than one that maps scans to root.

**Two records the series is required to carry forward, both confirmed:**

- **The token-diff bounding (batch 2, methodological).** A token diff **nominates** candidates and can **never** confirm a gap. Batch 2's diff flagged `seshutdownprivilege` — 11 hits, zero across 2,714 rows — and it was already in the KB as `SeShutdown`; one missing suffix was the whole "gap." This batch applied the rule and it paid twice more: `schallenge` → `XSSChallengeWiki` and `wsrep` → `wsrepl` were both slug collisions a token count would have called gaps. Grep the concept, never the token, before claiming a zero *or* writing an entry.
- **The declines and refusals.** `themastermindnotes.com/products/ecppt-study-notes-guide-unofficial` **declined** as a commercial product for sale — not fetched; if the operator supplies a purchased copy it goes through the PDF path. `mqt.gitbook.io` **refused itself** via `robots.txt` (`Content-Signal: ai-train=no`, blanket `Disallow: /` for ClaudeBot and eight others) and was correctly not fetched. And **0xdf was initially declined and that position was reversed** (D23) — the reversal is part of the story, not a footnote to it.

The fetched tree is deleted. `sources/0xdf-manifest.md` remains — the 55 URLs, the fetch date, and which 12 produced an entry — under the same single-file gitignore negation the repo and GitBook manifests use.

### Batch 3 — PDFs, pages, and CPENT: the close-out (2026-08-01)

This is the close-out. `PROMPT-pdf-and-pages.md` ran against its five sources and the series is complete. Its two entries are **distilled** (D21) from the CPENT cheat sheet, written from scratch against a bare flag list — the entries are the mechanism and judgement, the source was the nomination:

- `authored-perimeter-filter-mapping` [recon] — reading a packet filter as an object of enumeration: the three-state model (`--reason`), ACK/window scans for ACL mapping, firewalking to locate the device, and scan-time evasion with the decoy caveat spelled out. **Gap confirmed by measurement:** `--spoof-mac`, `--data-length`, `firewalk`, `--mtu`, `nmap -f` and window-scan were all **0 hits** before ingest, and retrieval for "map firewall rules with nmap" returned *cloud* firewall enumeration.
- `authored-ot-safe-scanning` [ics] — scanning an ICS/OT segment without faulting a controller: never `-sV` / `-A` / `-O`, prefer `-sT` over `-sS`, passive-first, `--max-parallelism 1`, abort path in the ROE. **Gap confirmed:** `--max-parallelism` was **0 hits**, and the `ics` category held exactly one entry — an operator persona, not a technique.

**The prediction held, and is worth recording because it survived its test.** Batch 3 was five more cert-note sources; the prediction written down *before* the run was ~0–1 entries. It yielded 2, close enough that the lesson stands rather than needing revision — and the two it produced are the exception that proves the rule, since neither came from the cert *notes* (the PDFs, the blog, the 185-page PNPT space all yielded **zero**, every candidate already covered) but from a cheat-sheet repo's two thinnest modules, which happened to name a scan-safety discipline the KB had never written down. The earlier wrong prediction in this series was the mirror image: CPENT was called the highest-*yield* cluster on the strength of its IoT/SCADA coverage, and its study guide left those modules as

`[TBD]` stubs — yet the *repo* version of CPENT, a different author, is exactly where the 2 entries came from. So the corrected lesson: the CPENT syllabus is high-value, but which artifact of it you get matters more than the syllabus name.

**The final scoreboard, measured:**

```
transcript corpus (687k chars)      13 entries
batch 1 — 24 repos + gist            2 entries   (22 sources yielded zero)
batch 2 — 7 GitBook spaces           0 entries
batch 4 — 0xdf pass 1 (55 posts)    19 entries
          0xdf pass 2 (51 posts)     8 entries
batch 3 — PDFs/pages/CPENT           2 entries   (4 of 5 sources yielded zero)
                                    —— 44 entries total; KB 2,699 -> 2,743
```

**The finding worth stating plainly: cert notes 36 sources → 4 entries; narrative writeups 106 posts → 27 entries — roughly a 20× yield-per-source difference.** Exam notes are condensed, derivative, and cover a syllabus the KB had already absorbed; writeups that start at a scan result and end at root supply a shape nothing else did. That is the reusable lesson for choosing future sources, and batch 3 did not disturb it.

**The uncomfortable number that must also be stated: the thin categories the series set out to fill still barely moved.** Measured now against the start:

```
services 9 (unchanged) · credentials 5 (unchanged) · persistence 4 (unchanged)
phishing 1 (unchanged) · iot 2 (unchanged) · ics 1 -> 2 · pivoting 6 -> 8 · recon 69 -> 70
```

Batch 3's two entries nudged `ics` and `recon` by one each — the first movement in `ics` the whole series produced — but `services`, `credentials`, `persistence`, `phishing` and `iot` are exactly where they started. All 27 fingerprint entries land in `category="writeup"` (173 → 200) because `ingest_exploitation_writeups.py` forces it as part of its `no_merge` discipline. They **function** — 2.7 keys on `meta.fingerprint`, not category — but the series **did not achieve its stated goal** of filling those thin categories, and this record says so rather than reporting 44 entries as if it had. **Open question, recorded without acting on it:** is `category="writeup"` right for discoverability, or should a fingerprint entry carry the service category it describes? That is a contract change to the ingester and belongs in its own session.

**Records the series carries forward:**

- **D22 — proxychains laundering the red-confirm** was found by *cataloguing a tool*, not by looking for it: the danger heuristic classified `argv[0]`, so `proxychains … weevely` demanded no confirm. The single most valuable output of the whole series, and it came from an enrichment batch's side.
- **D24 — the shared-predicate boundary convention**, the **third** instance of that pattern (build #5's WinRM `argv[0]`, D22's proxychains, D24). Worth actively watching for a fourth.
- **D25 — the retrieval residual**, closed this session: `fingerprint_match` reserved for structured hits, UNCOVERED false-fire 20%→0% with no covered-hit-rate cost.
- **The token diff nominates, it never confirms** — `seshutdownprivilege` looked like a certain gap and was already present spelled `SeShutdown`; grep the concept, never the token. Batch 3 confirmed it a third way: its top two token-diff nominations, printer *pass-back* and IKE *aggressive mode*, were both already fully in the KB (the printer entry even carries the 2024/25 Xerox pass-back CVEs) — the two entries that *did* land came from concepts the diff's noise had buried, found by retrieval testing, not by the diff.

- **Sources declined and why:** themastermindnotes (commercial product, batch 3 honoured the exclusion — not fetched), `mqt.gitbook.io` (refused itself via `robots.txt`), 0xdf (declined then reversed, D23), and **0xdf now closed** as a fingerprint source on a declining tail (19/55 → 8/51).
- **A source's scope in a prompt is a claim to verify, not a fact.** Batch 3's prompt listed `pnpt.adot8.com` as one of "two web pages"; its sitemap resolved to **185 pages** — a GitBook space on a custom domain — and it was routed through `fetch_gitbook.py` accordingly. Resolve the sitemap before trusting a source's described shape.
- **Windows Defender deleted files mid-run in four separate sessions** now (batch 3 lost `bypassing-amsi.md` to an errno-22 lock while it still showed 3,453 bytes on disk), from fetched source trees as well as `data/kb/entries.jsonl` — it is a **standing operational hazard for this repo**, not an anecdote: back up the KB before any rewrite, and make every triage loop tolerant of a file vanishing between listing and reading.

---

## Build #12 — the capabilities nothing could reach, and the first CI (2026-08-03)

Two findings drove this build, and the first is D19 one level deeper.

### Ten endpoints that existed and could not be reached

D19 fixed surfaces that were *built and then invisible* — a dozen routes reachable only by typing the URL, cured by putting the index on the home page. This build found the same failure one layer further down: **capabilities with no route at all.** A frontend-to-backend coverage sweep (every one of the 112 source endpoints matched against every call site in `frontend/src`) found ten with no caller:

| Endpoint | Backing module | Consequence |
|---|---|---|
| `GET /exploits/cve/{cve}`, `/exploits/cves` | `exploits/index.py` | "I have CVE-2021-41773, show me exploits" was unanswerable in the UI over a 25k-CVE index — you could search by *service* only |
| `GET /arsenal/suggest`, `/arsenal/render/{name}` | `arsenal/router.py` | the planner's own tool shortlist, and server-side template rendering, were API-only |
| `GET /detection/catalog`, `/technique/{id}`, `POST /tag`, `GET /sources` | `detection/` | the whole curated map — 133 command families, 19 techniques — was unbrowsable |
| `POST /sessions/{id}/webexploit/draft` | `reasoning/webexploit.py` | **build #8 Task 3 shipped backend-only** |
| `POST /sessions/{id}/privesc/ingest` | `reasoning/privesc.py` | **build #8 Task 4 shipped backend-only** |

`detectionSources` is the sharpest instance: it had a typed client function in `api.ts` and no component ever called it, so the About box it was written for never existed. Two of the ten were the *depth* half of build #8 — the work was done, reviewed and locked, and no human could reach it.

All ten are now wired: CVE-id lookup takes the exact keyed path (labelled "unranked", distinct from ranked search) with a "N CVEs affect this service" band beside it; the arsenal renders any tool against a target with unfilled placeholders left **visible** and badged rather than guessed; a new `:detection` route browses the map, expands a technique with its Sigma rules, and offers a deterministic tag probe with no model in the loop; and the two build-#8 reasoning surfaces appear in the engagement-state panel — a per-finding *draft exploit* action and a linpeas/winpeas paste box. Both are styled deliberately as proposals: no approve affordance anywhere, and a `needs the red confirm` chip stating what the gate will demand. **D18 is untouched** — every drafted step is data the human fires through the already-gated surface.

`:detection` was registered on the home launcher rather than left as a bare route, because D19's own lesson is that a page you must navigate to fixes discoverability only if you remember to navigate to it.

### The first CI, and the corpus problem it exposed

Until this build the only thing running the safety suite was remembering to. That is the wrong resting place for a project whose value is that gates fire — the gate audit probed 24 guards and found **seven that never fired**; build #5 found a red-confirm defeated by moving a cmdlet one token right; build #10 found the runner disarmable by adding a pipe. A refactor that silently unlocks a gate should fail a build.

`.github/workflows/ci.yml` runs three jobs. Blocking on every push and PR: the **56-file hermetic suite** (gated per-file on explicit exit codes), `next build`, and an eslint **baseline** check — pinned at 11 rather than 0, because `frontend/AGENTS.md` documents those errors as accepted deliberate patterns, and because the baseline had already crept 10 to 11 unnoticed, which is the argument for pinning it. Non-blocking, weekly: `pipeline/detection_sources.py --verify`, the live ATT&CK/SigmaHQ drift check, kept off the merge path because a failure there means the world changed, not that a PR is wrong — and, since it had never once executed, now dispatched and made to report its verdict rather than fail silently into a log (below).

Wiring it surfaced the corpus problem D26 records: `/data/` is gitignored, three fingerprint locks iterate the live KB by design, and on a clean checkout they crashed rather than skipped. The fix is the derived, complete, equivalence-proven projection described in D26 — **not** a hand-written sample, which `backend/AGENTS.md` §1 forbids for exactly the reason that would have bitten here.

**Verified against the real CI condition, not an approximation.** A clean virtualenv carrying only the five dependencies CI installs — *deliberately without pywinrm*, so the suite's hermetic claim is genuinely under test rather than papered over — combined with `HACKPIT_FORCE_KB_FIXTURE=1` to hide the live KB: **56 test files, every one exited 0**, with the three fingerprint locks running for real on the fixture (105 fingerprints self-matched, 56 version checks, 15 uncovered services) rather than skipping. The same env var reproduces CI on any machine that has the KB, and is documented in `backend/AGENTS.md`.

### The first CI run failed, and that is the result worth recording

The workflow was verified locally before it was pushed — a clean virtualenv with only CI's five dependencies and `HACKPIT_FORCE_KB_FIXTURE=1` — and it still went red on its first real run. The local simulation had removed the *dependencies* and the *KB*; it had not removed **the machine**. Three tests were leaning on gitignored files that exist only on the author's laptop, and all three had been green for months:

- **`sqlite3.OperationalError: no such table: sessions`.** `test_home_summary._real_secrets()` iterates `sessions_db.list_sessions()` against a database that is gitignored. Fixed with `init_db()` (a `CREATE TABLE IF NOT EXISTS`, so a no-op where the DB exists) rather than a `try/except`: swallowing the error makes the leg silently empty, whereas an initialised empty DB makes it a real check that finds zero rows — which the caller then *reports* as having checked nothing.
- **The same file's invariant 1 was vacuous on a clean checkout.** Every store it draws from — stored WinRM secrets, `llm_config.json`, captured engagement credentials — is gitignored, so CI has nothing to leak and the check proves nothing. It now reports **NOT-RUN** and returns, instead of failing its own "no real secret found

in any store — this test proved nothing" assertion. Invariant 2, the AST no-execution scan, is structural and still runs in CI.

- `test_operator.py` **died in app *startup*.** `with TestClient(main.app)` enters the lifespan, which loads the KB and hard-fails when `data/kb/entries.jsonl` is absent. `/operator` reads `operator_identity` directly and needs no lifespan state, so the `with` was dropped — the construction `test_sliver_safety` and `test_obfuscation_safety` already use for their endpoint assertions. That keeps a real leak guard **running** in CI rather than skipped.

**One fix was tried and rejected, and the rejection is the more useful record.** Staging `kb_fixture.jsonl` at `data/kb/entries.jsonl` so the app could boot *works* — the app starts fine on the projection. It also **lies to every component that asks "is there a built KB"**: `test_corpora`'s skip guard saw one, proceeded, and failed on `expected corpus entries in the built KB`. A fixture that makes absence look like presence is worse than the crash it cures, and the fixture's whole justification (D26) is that it is honest about what it is. It stays where it belongs — behind a loader that names it — and the app-boot problem was solved at the tests that had it.

The re-verification was done the way the first one should have been: a real `git clone` into a temp directory — no `data/`, no `sessions.db`, no `operator.json`, no `llm_config.json` — run with CI's exact dependency set and no pywinrm. That clone reproduced all three failures first, so the fixes are demonstrated rather than assumed, and then went green at **56 files, every one exited 0**. The second CI run passed on both branches.

The durable lesson is build #7's and build #9's in a third setting. Build #7: six defects invisible to a green suite, every one a structural claim nothing had executed. Build #9: four defects that only a real domain could surface, every one two halves of the codebase agreeing in a test and disagreeing against reality. Build #12: **three tests that passed for months and had never once run anywhere but the machine that wrote them.** Each time the green suite was accurate about what it checked and silent about what it assumed — and each time the fix was to put the code somewhere its assumptions did not hold.

### The drift job had never run — and could not have told anyone if it failed

Two green runs later, one job in the workflow had still never executed: `gh run list` returned **zero** schedule-triggered runs and **zero** dispatches. The ATT&CK/Sigma drift check is `schedule`- and `workflow_dispatch`-only by design, so nothing on the merge path had ever reached it. It was, precisely, a guard that existed and had never fired — the finding this whole workflow was written to prevent, sitting inside the workflow.

**Dispatched, it passes.** ATT&CK Enterprise **v19.1** and the SigmaHQ master ruleset both clean, **8 detection-reference pages** checked, `0 problem(s)`, 14 seconds. The one plausible way it could have failed on a clean checkout — the citation half reads `pipeline/authored/authored_entries.jsonl` — does not arise, because that file is tracked; the gitignored `/data/` never reaches it.

**But running it exposed why nobody would have noticed if it hadn't.** `continue-on-error: true` is what keeps this job off the merge path, and it is also what made it mute: real drift exits non-zero, gets swallowed, and paints the job **green**. The finding would have existed only in a log nobody opens on a Monday morning. *Non-blocking has to mean reported, not silent* — otherwise "we check upstream weekly" is a claim with no delivery mechanism, which is the same shape as a proof slot scoring as a pass. The job now captures the verify outcome explicitly, writes the verdict and the full output to the run summary **either way**, and raises a `::warning::` annotation on drift — the only part visible without opening the run. The outcome is read from `PIPESTATUS`, not `$?`, because piping to `tee` otherwise hands the step *tee's* exit code: the identical mechanism build #10 found could disarm the safety runner. Upstream is clean today, so the drift branch of the report was exercised by simulating a non-zero status rather than left to run for the first time on the day it matters.

Also bumped: all six action references ( `checkout` , `setup-python` , `setup-node` ) from Node 20 actions being force-shimmed onto Node 24, to `@v7` — confirmed `using: node24` at the source before pinning. Every run had been carrying deprecation annotations that would become hard failures whenever the runners drop the shim. Both changes verified by dispatch: three jobs green, deprecation annotations gone, the drift summary written.

This is the lesson a fourth time, in its sharpest form. The other three were code that was *wrong* in a place nothing had exercised. This one was not wrong at all — it had simply **never run**, and was built so that it would have looked identical either way.

### A latent finding, recorded not fixed

Measuring the fixture projection surfaced something about the matcher itself. `retrieval._entry_blob` reads `title, text, body, summary, tags, product` , and the live KB has **none of `text` , `body` , `product`** — it stores `body_md` and `steps` . So the unstructured fallback path has never searched the body of any entry; it sees titles, summaries and tags only. That may well be correct — matching titles and summaries is the more precise behaviour, and D25 tightened this path deliberately after measuring it — but the `body` / `body_md` near-miss looks accidental rather than chosen. It is **left alone and written down** rather than "fixed" in passing: changing which fields the fallback reads would move retrieval behaviour that was just carefully measured to a 0% false-fire, and that is a decision to take deliberately with an eval beside it, not a drive-by.

### Housekeeping, and a document that had been wrong for a year

`GATE-AUDIT-FINDINGS.md` moved from the repo root into `docs/` (both stale references updated — the build-5 plan literally said "at the repo root"). `fix-vmnet8.ps1` , the host-side VMware NAT repair for the build #9 lab, was **parameterized before being tracked**: it carried a hardcoded `C:\Users\zaid_\…` transcript path, and this repo is public — the same class of thing cleaned up before it was published. It now takes `param()` defaults and `$PSScriptRoot` , and sits in `docker/proof/` beside `dc_prereq_probe.py` , its natural companion (the probe reports the DC unreachable; this fixes the commonest cause). Two superseded KB snapshots were deleted after verifying the live corpus parses to 2,743 valid objects — `data/kb` 88 MB to 58 MB. `exploitdb.json` was **kept**: an earlier pass in this session wrongly grouped it with the stale backups, and it is the live CVE index `backend/exploits/index.py` reads.

**The repo went to one branch.** `sandbox-kali-image` was cut for the build #2 Kali image work, became the de-facto trunk for twelve builds, and had stopped describing its own contents by about build #4. It sat at the identical commit as `main` — zero commits either direction, empty tree diff — and the land-there-then-fast-forward-`main` habit bought nothing: no review gate, no window in which `main` was shielded, both refs public, both pushed seconds apart. Ceremony, not a gate; the gate is the CI plus the safety suite. It went, along with **eleven local feature branches** ( `ad-graph` , `session-panel` , `detection-panel` , `engagement-mode` , `tool-arsenal` , …) that were all fully merged. Deleted with `git branch -d` rather than `-D` , so git refused anything unmerged rather than trusting the check that said none were — it refused nothing, and all twelve tips verify as ancestors of `main` afterwards. The CI push trigger is now `branches: [main]` , with `pull_request` left unfiltered so a branch cut for a one-off build is still gated through a PR without editing the workflow again. **Every earlier reference to `sandbox-kali-image` in this document and in `docs/` is left standing on purpose** — those sentences record what was true when they were written, and rewriting a past assessment to scrub a branch name is the worse outcome of the two.

`docs/build-notes.md` was rewritten. It had opened with "**The Cockpit does not exist yet**" and closed calling the execution engine "still ahead of me" — true when written, false for about a year, across eleven builds and a live DCSync against a real domain. The correction is kept **in** the document as a dated note rather than quietly applied, because the failure is worth naming: a doc written to guard against overselling ended up underselling by a year, and the mechanism is the same either way — it stopped tracking the code. The five original "what broke" stories survive; three better ones were added (the six live-fire defects invisible to a green suite, the shared-predicate pattern across three subsystems, and the DA password leaking into the LLM proposer context), and the closing admission is now four honest ones led by the real one: the headline was autonomous hacking and the project deliberately built the opposite.

## Build #13 — where a callback lands (2026-08-03)

HackPit reaches **out** to anything: the engage sandbox is fully open by decision. Being reached **in** is a different problem, and it is the one real capability gap a review of the whole build surfaced. A callback is the target dialling *you*, and for that to land a container port must be published on a host address the target can route to. Exactly one file did that — hand-written, opt-in, and hardcoded to the VMware VMnet8 address of one laptop.

**This is one of four parts, and the decomposition is the first decision.** The callback gap splits into local listener profiles (no infrastructure), an out-of-band confirmation canary and a public C2 listener (both needing a VPS and a domain that do not exist yet), plus an unrelated safety-model change — reversing the evasion engine's generate-only rule into a gated deploy. Cramming those into one spec would have produced a document too vague to implement and a plan too big to verify, so each gets its own spec → plan → build cycle. Part 1 is first because it is free, verifiable today, and it is where the callback-destination abstraction gets designed, so the remote parts slot into an existing shape rather than being bolted on.

### What was built

`backend/cockpit/exposure.py` owns the surface end to end: validate → render → write → apply → observe. A profile names a bind address, a container ( `engage-sandbox` or `kali-open` — never the lab sandbox), and the listener kinds it will use; ports are derived from each kind's default and explicit extras merge in. `docker/proof/c2-lab.yml` is gone, replaced by a `vmnet8-dns` preset locked against it by a test.

### Three decisions went against the first draft, all from pushing back on it

**Public and wildcard binds are red-confirms, not refusals.** The first design refused both. But this codebase already has the pattern — the danger gate "never blocks outright; requires the confirm. Over-inclusive assist — human is the gate" — and inventing a second, stricter one would be inconsistent for no gain. A wildcard buys two real things: a binding that survives a VPN or DHCP address change, where a named bind leaves a container that will not restart, and a fallback when a specific bind misbehaves under Docker Desktop's networking.

**Arbitrary ports are allowed.** The first design derived ports only from the four known listener kinds. Those four omit a **plain reverse shell** — netcat or pwncat on 443 or 4444, an msfconsole handler — which is the commonest callback there is, so the restriction would have been hit on first use. Ticking a kind now fills in its port as a convenience rather than acting as a cage. Ranges stay refused: a range is how one typo publishes hundreds of ports, and it makes the exposure summary unreadable.

**A non-live bind address warns rather than refuses — and the claim that motivated refusing it was wrong.** The first draft said a mistyped address would start fine and silently receive nothing. It does not: Docker refuses to start the container with `bind: cannot assign requested address`. The check buys a better error, earlier — not safety — and refusing would break the real case of writing a profile while off the VPN, intending to connect before applying it.

### The hole self-review found, and the one the suite found

**Permitting an acknowledged wildcard broke the scanner.** `test_exposure_safety` is a static text scan, and it had no way to tell a wildcard the operator consciously chose from one that slipped through — so simply teaching it to accept wildcards would have *deleted* invariant 3 rather than relaxed it. The acknowledgement is therefore rendered **into the file**, as a `# hackpit-ack: wildcard bind=0.0.0.0 engagement=…` line, and the rule becomes **covered** rather than **absent**: every broad binding needs a marker naming that exact address. This makes invariant 2 stronger, not weaker — the one small file a reviewer reads now states what is exposed *and* that it was chosen deliberately, by whom and when. A marker for a different address covers nothing, which is the case that keeps the rule from degrading into "contains the string `hackpit-ack` somewhere".

**A guard fired on the first full-suite run, and the fix was to remove the dependency rather than excuse it.** `exposure.py` needs to know which port a listener binds, and the obvious way to get it is `from .tunnels import CHISEL_DEFAULT_PORT`. That trips the whole-tree scan that allows exactly two files to reference the tunnel module, so no agent path can raise a pivot listener; `sliver` and `obfuscation` carry the same guard. Adding `exposure.py` to those allow-lists would have been wrong twice — it narrows the file set instead of fixing the predicate, the mistake build #5 records, and it would have left the module free to call `start_tunnel` with nothing watching. The scan matches the *import* rather than the call precisely so a module cannot get within reach and then be trusted not to use it. So the constants moved to `cockpit/listener_ports.py` and the dependency disappeared. The drift lock came out stronger: one definition instead of one per owner.

### Measured, not assumed

The bind classifier keys off `ipaddress.is_global`, not the obvious `is_private`, because a test caught `is_private` being wrong twice over on Python 3.14: it is **False** for CGNAT `100.64/10` (Tailscale, mobile hotspots) and **True** for the RFC 5737 documentation ranges. The first is the one that would have hurt — every Tailscale address would have been called public and demanded an acknowledgement, on the interface most likely to be right for a remote internal engagement. Liveness is probed by binding a throwaway UDP socket rather than enumerating interfaces, which would need a third-party package the hermetic suite forbids, and which asks a weaker question than "can a listener actually bind here".

### What it still cannot do

A published port is **not** an open port — the host firewall can still drop inbound, so `observe()` says so rather than leaving the operator guessing. And 1c does nothing for an internet-facing target: `192.168.13.1` means nothing to a host on the internet, which is the whole reason parts 1a and 1b exist and need infrastructure. Suite **57 files, 0 failures**, up from 56.

## Build #13 part 2 — the evasion engine can now deploy (2026-08-03)

**This is a policy reversal and should be read as one.** The evasion engine opened with "*GENERATES ONLY, never runs or deploys*", and two tests enforced it. That property is gone by decision. HackPit can now put an artifact built to evade detection onto a real host and run it.

The argument for it is the project's own precedent: the Sliver server and the pivot/DNS listeners were once refused outright and became **gated-and-allowed** in build #7 / I2, on the reasoning that the gate — not the absence of the feature — is the control. The artifact always landed in a loot directory mounted into the sandbox and sitting on the host, so an operator could already copy it out and run it by hand. What changed is that the step no longer has to leave the tool. The argument against it is simply the plain fact above, and it is recorded here rather than argued away.

**What did not change: the mandatory footprint.** `deliver` computes the honest half first and raises rather than act without it, exactly as `generate` does, and the route guard now asserts `DeliveryResult` carries it too. An evasion tool that told you only how to be quieter, and never what still sees you, would be an evasion how-to — that is what makes this purple-team, and it is what justified lifting the OPSEC sensor-tamper ban in D16.

### Two primitives, deliberately separated

Putting an artifact somewhere and **running** it are different acts with different blast radii, and one `deploy()` would have left the gate unable to tell them apart. So `deliver()` takes a **closed set** — `winrm` (chunked base64) or `smb` (argv `smbclient` ) — and never a free-form delivery command, which would have handed the package a general execution path with none of the executor's gates. `invoke` is **WinRM-only**, and a sandbox invoke is **refused rather than merely unimplemented**: running the artifact inside HackPit's own box detonates it on the operator's machine, never the target.

The red-confirm is required **unconditionally** rather than left to the heuristic to notice — build #5 found a red-confirm you could defeat by moving a cmdlet one token right, so a gate that depends on a classifier spotting a name is a gate a rename defeats.

### The guard that fired found a design mistake, not an import

`test_winrm_safety` scans the whole tree and allows only the executor and the router to reach `winrm_transport` — the orchestrator proposes, it must never fire WinRM. The first implementation of `deliver` imported the transport directly and tripped it.

Adding the evasion engine to that allow-list would have been wrong twice over: a **third** module able to fire WinRM, and that module re-implementing gates beside the ones already living in the executor. The real problem was architectural — `deliver` was duplicating gate logic and then reaching *around* the gated execution point. So the capability moved to `executor.send_windows_scripts`, where every other Windows command already goes.

That also resolved a genuine tension. Per-command human approval is the floor on a real target, but a chunked upload is not N commands anyone could sanely approve one at a time. Treating the whole sequence as **one gated act** — one approval for one transfer, chunking an implementation detail of it — keeps the floor intact without making it unusable. This is the second time in one build that the right answer to a whole-tree guard was to remove the dependency rather than be added to its allow-list.

### Smaller things the tests forced

A **short write is checked** against the far side's own reported file length, because a truncated payload that reported success is the failure mode chunking introduces — and it is worse than a failed transfer, since the operator would run it. The SMB credential is **masked by construction** in the audited argv, so the run store does not become a key store — obfuscation.py's rule for its pre-shared tunnel key. And the route-set guard caught the new `/api/evasion/deliver` endpoint, which is exactly its job: the set is pinned so a new evasion surface cannot appear without someone deciding it should.

Suite **57 files, 0 failures**.

### Deferred to a later session, deliberately

Two README items, noted here so they are not lost:

1. **State plainly that the KB is not in the repo.** `data/kb` — the ~2,743-entry dataset of techniques, workflows and payload corpora — is gitignored and always will be (it is rebuildable, and third-party corpora are never committed). A reader who clones this and expects search to work will find it **empty and non-functional**, and the README does not currently say so, or say that the KB is available on request from the author.
2. **Mention the CI.** It is a genuine engineering signal — a public repo whose safety suite runs on every push, with a corpus fixture proven equivalent to the real one — and the README still describes the suite as something run by hand.

The README's counters are also stale (it claims 2,621 KB entries, a 110-tool catalog, and "42 test files, 459 assertions"; the measured values are **2,743**, **115** and **56**). All three are one editing pass, held for a session of their own.

### Status

Suite **57 files, 0 failures** (build #13), green both with the live KB and under the CI simulation. Frontend builds clean at 23 routes; eslint holds at the accepted 11-error baseline. Every one of the ten previously-unreachable endpoints now has a real component caller, verified by re-running the sweep that found them.

CI is complete: **all three jobs have now actually executed on GitHub's machines and passed** — the two gating jobs on every push, and the drift job by dispatch, which was the last piece of this workflow that had only ever been reasoned about. No deprecation warnings remain.

The repository is **one branch, `main`**, and CI is green on it.

## Build #13 part 3 — the out-of-band canary (2026-08-03)

This is **part 1a** in the numbering above: the half of "where a callback lands" that part 1c can never do. Listener profiles solve the callback for a target that can route to your laptop; `192.168.13.1` means nothing to a host on the internet, which is every bug-bounty target there is.

Without an internet-reachable listener, whole vulnerability classes are **unconfirmable**, because the hit *is* the entire proof: blind SSRF, blind XXE, blind RCE with no returned output, DNS-based blind SQLi (`xp_dirtree`, `UTL_HTTP`), JNDI and deserialization callbacks, and async SSRF where the app calls your URL minutes later through a queue. The alternative is writing a promising blind injection up as "unconfirmed", which most programs reject and which this project's own report discipline already bans.

**DNS matters more than HTTP**, and that is the reason this needs a real domain rather than an IP. Plenty of targets block outbound HTTP from application servers while DNS still resolves through their internal resolver, so DNS-based OOB lands where HTTP does not. That requires **NS delegation** to an authoritative listener — and it is the second reason part 1c can never do this job, since a target's resolver will never route to a private address.

### HackPit does not provision, and that is a decision

The tool **configures, deploys and verifies**; you create the droplet and buy the domain yourself, once. Four reasons, in order of weight. A credential that can create one droplet can create a hundred, and it would sit in an application that has **zero route authentication today**. The ROI is poor for a strictly one-time task. Domain registration needs a funded registrar account and ICANN verification regardless, so the "automated" path still stops and waits for a human. And a tool that *spins up* C2 infrastructure is a materially different artifact from one pointed at infrastructure you already own — the same concern that ended build #11. Everything after the one-time setup is buttons.

### The first internet-facing component, and why it is safe to expose

`oob/server.py` is the first HackPit component that faces the internet, and it faces it from a machine holding a client's evidence. The safety argument is deliberately **an absence rather than a control**:

- it **records, and that is all** — no execution, no eval, no deserialization;
- it **never forwards** — there is no outbound connection anywhere in the file, so it cannot be turned into a redirector. That is part 4, and a different thing;
- it **never reflects** request content into a response — the body is the constant `ok\n`, so it is not a free reflection oracle for anyone who can reach it;
- the hit log is **append-only JSONL** — no database, no rewrite path, no delete;
- **reads are authenticated**, because the log holds the target's internal hostnames and source addresses. That is the client's information, and an unauthenticated read endpoint would publish it to anyone who guessed the host. Bearer secret from the environment, compared with `hmac.compare_digest`, rate-limited per source, newest-first on a cursor.

It is **one file importing nothing outside the standard library**, because it is deployed by copying it to a bare VPS. An install step on a box you SSH into once is a box that stops working the day a wheel moves.

The correlation deliberately does **not** live there. The server records the candidate token it saw; only HackPit knows which tokens were ever minted and for which engagement. A canary that knew would be a canary worth stealing.

### Four things the design got right only after being questioned

**An answer, never NXDOMAIN.** Refusing the name would record the DNS half of a chained proof and kill the HTTP half. The A record is the feature: the target resolves `<token>.<zone>`, gets an address, and the follow-up request lands on the same box under the same token. Other query types (AAAA, TXT, MX) get NOERROR-no-data rather than a refusal, for the same reason — the name stays alive for the A retry.

**The token is the label immediately LEFT of the zone, not the leftmost label.** Reading the leftmost one works perfectly for `<token>.<zone>` and fails silently for every blind-RCE and DNS-exfil one-liner, all of which *prepend command output* to the name. `whoami-output.<token>.<zone>` has to correlate or the feature only works in the demo.

**The question is echoed byte-for-byte, and this is the subtle one.** Resolvers randomize the case of a query as an anti-spoofing measure (DNS 0x20) and compare the echo against what they sent. Re-encoding the question from the lowercased name — the obvious implementation — produces an answer the resolver discards as a spoof, while every log on this side says the hit was answered. So the raw question bytes are kept verbatim for the response, and the *recorded* name is folded for correlation. Two different jobs, two different forms.

**Credentials that arrive in a hit are recorded as present, with their values dropped.** A blind SSRF routinely arrives carrying the target's own `Authorization` header, session cookie or proxy credential. The finding needs "an authenticated internal client reached out"; it does not need the secret it used, and storing it would turn a canary into a credential store sitting on a VPS. Bodies are capped to a 512-byte excerpt, flagged when truncated — a canary records that something arrived and from where, not a copy of the target's traffic.

The read secret **fails closed**: no secret, or one under 16 characters, and the server refuses to start. The failure mode that prevents is an operator deploying, watching the listeners come up, and never learning the read endpoint was open the whole time.

### Verified end to end, on loopback, for real

The first assumption was that nothing here could be verified without infrastructure. That was wrong, and precisely wrong: **everything except public reachability runs today**. `docker/proof/oob_loopback_proof.py` starts both listeners on `127.0.0.1` with DNS on **udp/5353**, sends a real datagram carrying a real minted token, parses the answer off the wire, and asserts the hit was recorded, correlated back to the right engagement and step, and served through the authenticated API — with the anonymous and wrong-bearer reads refused and the operator's own read traffic absent from the hit log. **15 passed, 0 failed, 2 not-run.**

The two not-run are named exactly, never folded into a pass: that **NS delegation resolves publicly** to the server, and **one live hit from a real target**. Both become real checks the moment the VPS and zone exist.

**The proof's own first version had the bug worth recording.** It probed udp/5353 with a throwaway socket before binding, and fell back to an ephemeral port when the probe failed. The probe did not set `SO_REUSEADDR` and the real listener does, so on a host running mDNS the probe reported the port busy when the actual bind would have succeeded — and the proof passed cheerfully on a port the spec never named. A probe that differs from the real thing answers a different question. It now attempts the real bind and says loudly if it ever falls back.

Hermetically, the two new test files add **24 assertions** over the wire format, the token grammar, redaction, paging, authentication and inertness. Ten mutations were planted to confirm they can fail — NXDOMAIN answers, a re-encoded question, redaction removed, an uncapped body, a prefix-matching bearer check, a missing read secret, oldest-first paging, the leftmost-label token, a reflected response, and a global instead of per-source rate limit. All ten were caught.

Two structural locks exist because no behavioural test can see the difference: the minter must draw from `secrets` and never `random`, and the bearer check must go through `hmac.compare_digest` rather than `==`. Both spellings look identical in review and only one of each is safe.

**The token grammar lives in two files on purpose** — the deployable may not import from the repository — so a test holds them together against 200 freshly minted tokens rather than against a reading. Drift there is the silent kind: hits keep landing and quietly stop correlating.

### The rest of part 1a — poll client, templates, panel (2026-08-03)

The first commit was the server and the minting. This one is everything that turns a line in a JSONL file on a VPS into a finding in a report: the **poll client and state ingest** (§3.3), the **payload templates** (§3.4), and the **configure / deploy / verify panel** (§3.5) with its UI.

**Correlation is the product, so nothing is dropped.** A hit arrives with no memory of why. The poll client joins each one back to the token's mint record — engagement, step, note — and files the correlated ones as `high`-severity findings that carry the source address, the timestamp and the token. The severity is deliberately not a judgement call: a callback from inside a target's network is the proof for the *entire* blind class, and grading it lower because the module cannot see the payload would understate every one of them. Hits that cannot be attributed are **reported, never discarded** — "something arrived that I could not place" and "nothing arrived" are different facts, and collapsing them would reintroduce exactly the silence this part exists to remove. There are two such shapes and they get distinct reasons: a token this HackPit never minted, and an engagement with no state session to file into.

**Hits arrive late, so the engagement lookup covers exited engagements too.** A blind SSRF routed through a queue lands minutes or days after the test that caused it, by which point the engagement has very often been exited. Filtering to active engagements would have filed those nowhere — the evidence loss the canary was built to stop.

**The cursor is the whole safety of re-reading.** It advances only after the ingest returns, and only forwards. A poll that fails anywhere leaves it untouched and the same hits are read again next time; re-reading is free because findings upsert on a fingerprint, whereas missing a hit is not recoverable. Reads with an explicit `after` (verify, the panel) never touch it, so a verify run cannot swallow a genuine callback that arrived in the same window.

**Why an outbound request is allowed here when the repeater refused one.** `cockpit/repeater.py` deliberately does not send from the backend process — it execs curl inside the open sandbox — because operator-composed requests to arbitrary hosts would be a general egress path out of the operator's machine. Neither half applies to a poll: the destination is one host read from the config store, the path set is two constants, and the response is parsed as JSON and never executed. That is the containment shape `backend/llm.py` already has. **Writing that down surfaced a real defect.** The first version documented "redirects are not followed" and did not follow from the code: `urllib.request.build_opener` re-adds every default handler that is not overridden, so leaving the redirect handler out leaves redirects *on*. A tampered or proxied canary answering `302 http://169.254.169.254/…` would have turned a poll into an SSRF from the backend host. It now passes an explicit refusing handler, and the test interrogates the real opener object rather than reading the source — with a control proving the check fires on an opener that would redirect.

**The templates encode the details whose absence is silent.** Fifteen of them across the five classes the spec names — SSRF, XXE, blind RCE, blind SQLi, JNDI — each carrying where to paste it and, separately, *what a hit proves*, because that sentence differs by class and ends up in the write-up. The class-specific traps are the point: a parameter entity where a general entity is refused, a UNC path rather than a URL for `xp_dirtree`, `LOAD_FILE` being Windows-only. The one that would rot quietly is **exfil direction** — the server reads the label immediately left of the zone, so `whoami`-output must be prefixed, not appended. A test feeds every DNS name in every rendered payload to the **canary's own parser** and asserts the right token comes back: 21 names, checked against the implementation that will actually read them rather than against a second regex.

**The deploy takes no destination, and that is asserted on the signature.** Shipping `server.py` over SSH is a third remote-execution transport after `docker exec` and WinRM, so it lives at the gated execution point in `cockpit/executor.py` — the lesson part 2 learned when `deliver` grew its own WinRM call and a whole-tree guard caught it. `deploy_oob_canary(*, approved, restart)` has no host, user, port or key parameter, defaulted or otherwise, and the resolver it calls takes no arguments either: there is no way to *express* "deploy somewhere else". A test parses the real signature and fails on any destination-shaped parameter, with a planted control. Around that: an unapproved deploy sends nothing (verified by recording the transport, not by trusting the return), the whole tree is scanned so only the executor and the canary's own route can reach it, and the read secret rides **stdin** into a 0700 file rather than a command line, because `ps` is world-readable on a multi-user box. The config store refuses shell metacharacters in the host, zone, user and key path at *configure* time, and every interpolated value is single-quoted at the point of use anyway.

The subprocess runs on the host rather than in the sandbox, which is the opposite of the repeater's choice and is deliberate: the alternative would mount the operator's SSH **private key** into a container that also runs attack tooling — strictly worse than the thing it would be avoiding.

**Verify is the valuable button, and it reports NOT-RUN as its own status.** Every link in this chain fails silently, so "is my canary working" has to be re-runnable. Three checks: the server answers its authenticated health endpoint *and* is authoritative for the zone payloads are rendered against (a mismatch there is a live misconfiguration that otherwise looks like "no hits ever arrive"); a freshly minted token completes a full round trip — mint, arrive, correlate, read back; and NS delegation resolves. The third is the only one that genuinely needs public infrastructure, and it says so rather than counting as a pass. The NS records themselves are generated as exact copy-paste text, with the mistake named: an A record alone makes the zone resolve while every `<token>.` name still goes to the parent's nameservers.

**A guard fired, and the fix was to split the claim rather than widen the ban.** `test_oob_tokens.py` asserted that the whole `backend/oob` package "executes nothing and reaches no network" — and the second half stopped being true of a poll client, whose entire job is to fetch. The tempting fix was to drop `urllib` from the banned list, which would have un-banned it for all six modules. Instead the invariant is now two: execution stays absolute with no exceptions, and network reach is a **pinned set of two named modules**, so a third one gaining egress fails here. That is a stronger statement than the original made, not a weaker one. The same discipline applied to a second guard: the DNS-tunnel human-only lock matches on raw text and fired on a docstring that merely *cited* the sibling generator to agree with it — so the citation was dropped rather than the module allow-listed.

**One defect in the tests themselves, worth recording.** The first drafts called `save()` and `clear()` on the real `sessions.db`, which would have destroyed a live canary's read secret — a value whose only other copy is a 0700 file on a VPS. All three test files and the proof now redirect their stores to a scratch database. A suite that can damage the thing it is testing is not hermetic whatever else it asserts.

**Verification.** The loopback proof grew from 15 checks to **24 passed / 0 failed / 3 not-run**, and the new half is not a unit test wearing a proof's name: it configures the real store, points the **real poll client** at the real listener over loopback TCP, and asserts it fetches through the authenticated API, correlates back to the engagement and step, files findings into engagement state, and is idempotent on a re-poll. The real verify button runs too — health and round trip pass, DNS reports NOT-RUN. Hermetically, three new test files add coverage of correlation, the cursor, the opener, the templates against the server's own parser, and the deploy gates.

The three NOT-RUN are named individually and never folded into a pass: **NS delegation resolving publicly**, **one live hit from a real target**, and **the SSH transfer itself**. Only the third is new, and its *gates* are fully covered hermetically — what is missing is a remote box to reach, not an untested control.

**Frontend.** `:oob` is a real panel at `/oob`, wired to all eight endpoints — there is no endpoint here without a component caller, which is the whole reason build #12 existed. It walks the actual order of operations: configure, delegate, deploy, verify, mint, collect. The read secret field is blank on load and blank means "keep the stored one", because the value is never sent to the browser. The deploy button sends `{approved}` and nothing else; there is deliberately no host field, since there is no parameter for one. eslint holds at the accepted baseline of 11 — the refresh callback routes its `setState` through a `.then` rather than an effect body, which is what pushed the count to 12 on the first attempt.

Suite **62 files, 0 failures**, up from 59.

### What is not built yet

Part 1a is now complete. The public C2 redirector remains **part 4**; this server records hits and never forwards, and nothing in it can be turned into a redirector — an outbound-connect ban is asserted over its AST.

## Build #13 part 4 — the public C2 redirector (2026-08-03)

The last piece of "where a callback lands". Part 1 made the destination configurable but every destination it can express is an interface **on this machine** — so an implant inside an internet-facing target has nowhere to call, because a laptop behind NAT has no address anyone can dial. Part 3 made the *proof* of a blind vulnerability reachable and deliberately cannot carry a session. This adds the remaining shape: a redirector on a VPS the operator already owns, which accepts an inbound connection on a public port and relays it down a reverse tunnel that was dialled **outward**. The implant talks to the VPS; the VPS talks down a tunnel established from the inside; nothing is traversed inbound.

### This one carries real AUP weight, and the safety argument had to be rebuilt

Part 3's canary is safe to expose because of an **absence** — it records, never executes, never forwards, and answers a constant. Reusing that shape here would be dishonest, because this component *forwards by design*: it is the exact thing `test_oob_server.py` has an AST-asserted ban against the canary becoming. What is actually being stood up is an always-on listener on a public IP, reachable by anyone who scans that address rather than only by the target under test, relaying traffic it did not authenticate, **into the operator's own machine**. That last direction is the one that matters — a misconfigured redirector is an inbound path from the internet to a laptop, not just an exposed service on a rented box.

So the claim is **bounded forwarding**, and the bounds are structural rather than advisory:

- **One destination, and it is loopback.** `TARGET_HOST` is a module constant in the deployable. A test walks every addressing call in the file and asserts each one uses it — with the answering half held to its own non-empty rule, since replying to the source of a datagram is not choosing a destination. Nothing in the file resolves a hostname. An open forwarder on a public IP is precisely what a stranger scanning that address wants to find, so "it cannot be pointed elsewhere" had to be a property of the code.
- **An enumerated port set.** No ranges, the same rule part 1 already applies, for the same reason: a reviewer has to be able to read the whole exposed surface at a glance.
- **Off unless the tunnel is up.** With nothing attached, the loopback target is a closed port, so a client is accepted and dropped. That is the right failure direction, and the proof asserts it *first* — before any listener exists — because it is the state a redirector spends most of its life in.
- **Deploy and stop are both gated, and equally reachable.** A start button without an equally reachable stop is how a public forwarder outlives the engagement it was built for.
- **The panel says all of this in those words.** The exposure sentence, the AUP position and the teardown command are returned *with* the profile, so a panel cannot render a configured redirector without also rendering what it exposes.

**Authentication is deliberately not built, and the reason is written down.** It would be theatre: a shared secret would have to live inside the implant, which is a binary the target holds. A fake control standing beside a real one is worse than no control, because it invites trusting the wrong thing.

### Extending part 1 rather than sitting beside it

`ListenerProfile` gains a `destination` of `local` (unchanged, and regression-locked — part 1's tests and its published-port scanner pass untouched) or `remote`. The two are not one path with a flag, and the validation genuinely differs rather than being reused with exceptions carved out: a remote profile has no bind address to check liveness on, no container, and **no private case** — a port on a VPS is reachable by anyone, so the public acknowledgement is unconditional rather than conditional on classifying an address there is none of. `render`, `write`, `compose_command` and `apply` all refuse a remote profile at the door, and `write_remote` refuses a local one, so the two paths cannot half-mix into a compose file that publishes on `''` or an operator believing a VPS is exposed when nothing was shipped to it. `observe()` reports `remote` and does not guess — `docker inspect` says nothing about a process on someone else's machine, and answering anyway would be the exact defect `lifecycle.py` exists to prevent.

### One deploy engine, not a second path

Part 3's deploy ships one file to the configured VPS behind an approval and takes no destination. Part 4 needed to ship a different file to the same box, and two of the three available answers were wrong: a second deploy function would repeat part 2's mistake of two places implementing the same gates, and a `path` parameter would turn a deploy button into an arbitrary-write primitive on a machine reached over SSH once. So the transport, target resolution, gates, stdin-not-argv secret discipline and step orchestration became **one private engine**, with the artifact as a module-level constant built inside each wrapper from repository paths and server-side config. The public surface is three thin wrappers that take an approval and nothing addressable, and the signature assertion now runs over **all three** — plus a new check that reconciles the wrapper list against the real module, so a fourth one added without a line in the enumeration fails rather than going unchecked.

### The one awkward fact, stated rather than papered over

**SSH reverse tunnels carry TCP only.** A Sliver implant or a reverse shell rides `ssh -R` end to end; a DNS tunnel does not. Rendering an `-R` line for UDP would produce a command that runs cleanly and carries nothing — indistinguishable from a target that never called back, which is the precise class of silent failure part 3 exists to remove. So the UDP case renders a `socat` pair with each end labelled by *where it runs*, and says why. The tunnel itself terminates on `127.0.0.1` on the VPS rather than `0.0.0.0`: the alternative needs `GatewayPorts yes` and would put the tunnel endpoint on a public interface with no forwarder in front of it — an unbounded relay straight into the operator's machine.

The reverse tunnel is **rendered and never run**, the same boundary the DNS-tunnel client one-liner draws. It is a long-lived outbound process on the operator's own machine, and starting it deliberately is the approval.

### A test bug worth recording

The first draft of the destination scan read `args[0]` for every addressing call — correct for `connect`, wrong for `sendto`, whose signature is `(data, address)`. It inspected the *payload* and reported every legitimate relay as an unknown destination. That is the kind of false alarm that gets a guard loosened rather than fixed, so the predicate now records which argument carries the address per call shape, and a control asserts specifically that `sendto`'s address slot is the one being read.

### Verification

**On loopback, for real.** A redirector relaying `127.0.0.1:A → 127.0.0.1:B` is the entire mechanism; the only thing a public IP adds is that a stranger can reach A. So the proof runs a real forwarder, a real listener standing in for the far end of the tunnel, and a real client standing in for an implant: a full exchange in both directions over TCP, the same over UDP, the accept-and-drop behaviour with nothing attached, survival of a client that vanishes mid-stream, and agreement between the rendered `ssh -R` port and the port the running forwarder actually relayed to. Hermetically, two new test files cover the live forward, the rendering, and the four safety invariants with a control each.

**Reported NOT-RUN, named individually, never folded into a pass:** that a connection from the internet reaches the forwarder on its public address, one real implant session through the full chain, and the SSH transfer itself. Only the first is a property of this component that nothing local can stand in for; the deploy's *gates* are covered hermetically and what is missing there is a remote box, not an untested control.

### Frontend

`:exposure` at `/exposure` — and this is where part 1's four `/cockpit/exposure` endpoints finally get a caller, having shipped with none. That was the same gap build #12 existed to close, and adding a second orphan surface beside it would have compounded it. The panel walks both destinations from one port selection, keeps the approval explicit for every destructive or exposing action, and renders the reverse-tunnel command, the socat bridges and the teardown with copy buttons. eslint holds at the accepted baseline of 11.

Suite **64 files, 0 failures**, up from 62.

## Build #14 part 1 — ZAP, the first web vulnerability scanner (2026-08-03)

The arsenal catalogued 115 tools and shipped an HTTP repeater, but nothing in it systematically probed a discovered parameter for an injection class. Recon found endpoints; nuclei matched known templates; the gap in between was the one a web assessment actually lives in. This closes it with OWASP ZAP.

### Why not Burp Suite Professional

Burp Pro is $499/user/year and its REST API is a scan-launcher — start a scan, poll it, fetch issues. Anything richer needs a Montoya extension loaded inside a running Burp, which is not drivable over a wire. Its licence is a named-user seat and the process assumes a desktop session, so running it as a shared service backend is the case PortSwigger sells Burp DAST for. ZAP is Apache 2.0, headless-first, and has no activation to break when a container is rebuilt.

A cracked Burp distribution was considered and rejected outright. Beyond being pirated software and a takedown risk to a public repo, it means running an unaudited third-party patch as the interception proxy that sees every credential pushed through it.

### The whole build is seven touch points and no new architecture

ZAP is modelled on nuclei: a catalogued tool the executor gates and the ingest parses. No new module, no new endpoint, no frontend, and — the point — **no new execution capability**. Scans run through `POST /cockpit/exec` as ordinary commands, so all four lab gates and all three engagement gates apply on day one without a line of code protecting them.

**The daemon and its REST API are excluded, and the reasoning is structural rather than cautious.** Every gate here inspects *a command*. Once "scan this target" is an HTTP call, it bypasses all of them unless a second, parallel gate system is built — and a second copy of a safety predicate is this codebase's most-repeated defect: the WinRM `argv[0]` classification, the proxychains-laundered confirm, the collector FQDN classifier. Worse, the only channel into a sandbox is `docker exec`; the lab sandbox sits on an `internal: true` network precisely so no route exists between it and the host. Reaching an HTTP API in there means opening the path `assert_isolation_proven()` exists to deny.

`zap.sh -cmd -autorun plan.yaml` is excluded for a related reason: whether the run is passive or active lives *inside the plan file*, so the gate would classify a string that does not describe what executes. The packaged scan scripts are self-describing, which lets the existing danger gate split them unmodified.

### The split, and an inconsistency stated rather than hidden

`zaproxy -cmd -quickurl` spiders and then sends live SQLi/XSS/command-injection payloads at every discovered parameter, and demands the red-confirm. `zaproxy -cmd -zapit` crawls and fingerprints, and does not — gating both identically would make the confirm meaningless for the tool, the same argument the AD-enumeration note has always made.

**The verdict is argument-based, not name-based**, because Kali ships one launcher that does both jobs, so the binary was never the tell. `_TOOL_ATTACK_FLAGS` mirrors the `_TOOL_EXEC_FLAGS` pattern already used for `netexec -x`.

This makes ZAP **stricter than the rest of its family**: `sqlmap`, `nikto`, `dalfox` and `nuclei` remain unflagged, and sqlmap is arguably more intrusive. That is a recorded decision, not an oversight. Erring safe on a new tool changes no existing behaviour, and the rationale is written at the point of use so a future reader does not quietly "fix" it.

### Two things the existing guards caught that the plan had not anticipated

**A fourth shared-predicate defect, prevented.** `dangerous_script_heuristic` shares its tool groups with the command heuristic — its docstring says so explicitly, "two lists would drift, and drift is what produced this bug." Adding the active-scan rule to only the command side would have been the fourth instance of exactly that failure. It went into both — and after the rework the script heuristic **derives** its markers from the same `_TOOL_ATTACK_FLAGS` dict the command heuristic reads, rather than restating them, so the two cannot drift even in principle. A test locks them to the same verdict on the same tool.

**A tool's own name needs a verdict too.** `_catalog_invocations()` covers the catalog's `name` field, not just template `argv[0]`s, so bare `zap` failed the suite until classified. It landed in `_ARGUMENT_DEPENDENT`, whose stated shape it matches exactly: the bare binary is clean, at least one catalogued template fires.

### The image build caught what the whole test suite could not

This is the finding worth keeping. Everything was first written against `zap-baseline.py` and `zap-full-scan.py` — upstream's documented packaged scan scripts, and the names the parser registry, the catalog templates, the danger sets and four test files all hardcoded. **Kali's `zaproxy` package ships neither.** `dpkg -L zaproxy` on 2.17.0-0kali1 gives exactly `/usr/bin/zaproxy`, `/usr/bin/owasp-zap` and `/usr/share/zaproxy/zap.sh`. Those scripts exist only inside OWASP's own Docker image, and they hardcode `/zap/zap-x.sh`, so adopting them would have meant faking that image's directory layout and fetching three unpinned files from GitHub `main` into a safety-critical image — rejected, given how often upstream drift has already bitten this project (dnscat2, ScareCrow, trufflehog).

Every one of those keys would have matched nothing, forever, with a green suite. That is the build #9 ingest gap in a new place, and it is the *third* time in this build that the same root cause surfaced: **a hermetic test feeds the parser a string the test itself chose.** Nothing in 66 test files could have found it. The image build's own smoke test did, on the first run, because it checks the names against the package rather than the documentation.

The replacement is the package-native CLI, and it was verified before being adopted rather than after: a real ZAP 2.17 scan was run inside the Kali image against a live throwaway app, and `-quickurl`'s report turned out to be shaped exactly like the one `parse_zap` already handled — **so the parser needed no change at all.** That verbatim report is now committed as `test_support/zap_report_fixture.json` and a test parses it for 4 findings and 13 endpoints, with severities mapped from ZAP's *string* risk codes. The synthetic fixture is kept beside it, because the real report happens to carry only risk codes 1-2 and the four-way mapping still needs covering.

### The trap that would have shipped silently

**Two normalisers exist and they disagree.** `allowlist._tool_name()` strips `.py`; `state.ingest.program_name()` strips only `.exe`. So the danger sets must be keyed `zap-full-scan` and the parser registry `zap-full-scan.py`. Key either the other way and nothing matches: zero findings, no error, green suite. That is the build #9 ingest gap exactly, where a live DCSync dumped four NTLM hashes including krbtgt and ingested none of them. A test pins each side against its own normaliser.

A second, smaller version of the same lesson: `_json_objects()` cannot reach a ZAP report, and the way it fails matters. It does not return nothing — its line-delimited fallback matches the alert *instance* fragments that happen to sit on one line. A parser resting on it would emit quiet rubbish rather than obvious nothing. The spec claimed "neither path works"; implementation measured otherwise and the spec was corrected in the same commit.

### Verified, and what is not

**Hermetically, for real:** the report is extracted from surrounding progress output, all four risk codes map, the registry keys match what `program_name` actually produces, `-zap.json` is claimed while a plain `.json` loot file is not, both heuristics agree on both verdicts, proxychains cannot launder the confirm, and the catalog genuinely ships both invocations. Every one carries a control in the same test. Suite **66 files, 0 failures**, up from 64.

**Now run, and green:** `docker/proof/zap_install_proof.sh` **reports 9 passed, 0 failed.** The image was rebuilt, ZAP starts, both program names resolve under exactly the strings the parser registry keys on, the two scripts Kali does *not* ship are asserted absent, `$HOME/.ZAP` is writable as uid 1000, and a real active scan of the Juice Shop lab target from inside the isolated sandbox produced a report that `parse_zap` turned into **6 findings and 26 endpoints**. The ingest path is verified end to end.

Getting there cost three rounds, and each failure was a check working rather than a tool misbehaving. **The container is not the image:** every name check ran `docker run` against the freshly built image while the scan ran `docker exec` inside a container that had been up for 45 minutes on the previous one — `compose up -d` does not recreate a running container just because its image changed. The proof now compares the two IDs and says so outright. **MSYS rewrites container paths:** on a Windows host, Git Bash turned the container path `/tmp/zap-proof.json` into a host path before Docker saw it, so ZAP correctly reported the directory unwritable and exited 0 — a scan that "worked" and wrote nothing. **And bash's `/tmp` is not Windows Python's `/tmp`**, so the report is now piped to the parser rather than staged through a host file that the two sides name differently. A fourth lesson in one build: an exit code is not a result.

**Still not run:** a scan driven through the gated executor's own approve-and-run path rather than `docker exec` directly. The gates themselves are covered hermetically; what is untested is the wiring, not a control. These are the only checks that can confirm Kali installs the scan scripts under the names the parser registry and the catalog templates both hardcode — and no hermetic test can stand in, because it feeds the parser a string it chose itself. `docker/proof/zap_install_proof.sh` runs all of them and fails loudly per check. **Until it passes, the ingest path is unverified end to end.**

### Timeout, resolved without touching a global

A full active scan runs for tens of minutes against the executor's 180-second default. No global was raised — `MAX_TIMEOUT_SECONDS` is already 3600 and clamps rather than refuses, so the two full-scan templates simply say to raise `timeout_seconds` on the request. Raising a default for one tool changes behaviour for every tool that relies on it.

### Deferred to part 2

The proxy surface — ZAP as a live intercepting proxy feeding the repeater and state — with its own spec and safety review. The daemon question belongs there, and it splits by sandbox: the engage sandbox is already open and breaks no property by hosting one, while the lab keeps `internal: true` and keeps command-path scanning. A *recording* proxy may reuse the existing listener pattern almost directly (gate the start, hold liveness, expose no writer); a scan-control channel would need the `tunnels.py` treatment, where one derivation function sits behind both the gate and the action.

## Build #14 part 2 — the recording proxy (2026-08-03)

Part 1 gave HackPit a scanner. This gives it the thing the tool had never had: **the raw HTTP of every run**. Until now a `ffuf` run's findings were parsed and its actual requests were thrown away — you could see that `/admin` existed, never the request that found it or what came back.

### The daemon part 1 refused, built anyway — because the transport changed

Part 1 excluded a ZAP daemon for two reasons: an HTTP control channel bypasses `validate_request`, and reaching one inside the lab sandbox would mean opening the `internal: true` network `assert_isolation_proven()` exists to deny.

Both objections are about **a socket from the backend to the container**. Neither survives if there isn't one. Measured against the running sandbox before any of this was designed:

| Check | Result |
|---|---|
| `zaproxy -daemon -host 127.0.0.1 -port 8090` | API answers after ~7 s |
| `curl 127.0.0.1:8090/JSON/core/view/version/` **from the host** | **refused — unreachable** |
| the same call via `docker exec` | `{"version":"2.17.0"}` |
| a request proxied from inside the sandbox | recorded, with full bodies |

The daemon binds loopback **inside** the container and no port is published. So the API exists, and the only way to reach it is `docker exec` — the same channel every other execution uses, and the one the gates already classify. Part 1's objection was to an *ungated* control channel; this is a gated one.

`docker/proof/zap_proxy_proof.sh` asserts host-unreachability directly, because it is a property of the network rather than of the code and no hermetic test can see it. **7 passed, 0 failed.**

### Four defects the guards caught, none of them test bugs

**The gate refused the operator's own socket.** The first `_gate_request` passed the real argv, and the scope extractor reads `127.0.0.1` as an out-of-scope host. `tunnels.py` documents this exact trap for `-laddr`; my code carried a comment saying the port was excluded while passing it anyway.

**Lab mode refuses target-less commands** — a locked invariant, and very likely the real reason `tunnels.py` is engagement-only. Copying that would have been worse than the feature: engagement mode runs in the fully-open sandbox, so "make an engagement for the lab" would push practice traffic out of the sealed box. Resolved by declaring the lab as the gate surface, which is a true statement of scope rather than a workaround — the lab proxy runs in the isolated sandbox, whose network has no route off the bridge, so the lab target *is* everything it can reach. `check_target_lock` is untouched.

**The red-confirm would have been decorative.** Part 1's rule is argument-based and keys on `-quickurl`, which a daemon argv does not carry — and a gate surface holding only the binary name cannot fire a flag-based rule at all. Either omission alone left `dangerous_ack` unenforced: gate-audit finding I2's exact shape. `-daemon` now sits in both the attack flags and the surface, for a stated reason — it attacks nothing, but it starts a listener that records credentials in cleartext.

**The repeater lock refused the module.** `proxy.py` first imported `RepeaterExchange`; `test_repeater.py` bans *any* import of the repeater, not just `repeater.send`, because a module that can import it is one line from calling it. The tempting fix was adding `proxy.py` to the allow-list — the exact anti-pattern build #5 was about. The models are local instead, with field names deliberately matching so the panel still renders a captured exchange with no translation layer.

### Two things only a live process could find

**The spawned argv is not the running argv.** `zaproxy` is a wrapper that exec's the JVM, so the process on the box is `java -jar /usr/share/zaproxy/zap-2.17.0.jar -daemon …`. The string "zaproxy -daemon" appears nowhere in it, so `pkill -f` matched nothing and the daemon survived every stop. No unit test can catch that — a hermetic test has no process to fail to kill.

**`pkill -f` matches its own command line.** The fixed pattern then killed the killer: the probe shell exited 143 having reaped nothing. The pattern is written `[z]aproxy` for that reason — it matches the literal "zaproxy" in the JVM's argv, while the killer's own argv contains "[z]aproxy", which does not.

### Secrets: raw in the panel, masked in reports

Captured bodies hold passwords, `Authorization` headers and session cookies. Redacting on ingest was considered and rejected: the request that matters is usually the one carrying the token, and this is the operator's own data on their own disk. Masking lands at the **report** boundary instead — the artefact handed to a client or a grader — reusing `secretargs`' `REDACTED` marker rather than inventing a second convention. It masks the value and keeps the parameter name, and its test carries a positive control, because a redactor that blanks everything would satisfy "the password is gone" while making every report useless.

### One invariant with nothing behind it, stated plainly

The history read is deliberately **ungated** — a panel that refreshes cannot demand approval per refresh. That makes it the one path in this build with no human checkpoint, so what it can reach matters. It issues two fixed URL constants and takes no endpoint parameter, so an `action/` call is not expressible without writing a visibly new function.

**Nothing enforces that.** Both candidate guards — a runtime allowlist and a three-line static test — were considered and declined (2026-08-03). It is a convention, and the spec records the consequence and a review rule: any change to `cockpit/proxy.py` that introduces a URL parameter reopens the decision rather than quietly satisfying it.

### Verification

Suite **68 files, 0 failures**, up from 66. `zap_proxy_proof.sh` 7/0.

That count needed one more fix to be true, and the honest version is worth recording. `test_redirector.py` began failing four runs out of four with `WinError 10013` — its `_free_port()` helper picked a port by binding a **TCP** socket, and the UDP test then bound **UDP** on that same number. TCP-free proves nothing about UDP: Windows reserves large UDP ranges for Hyper-V/WSL (on this box `50000–50059` and everything from `53879` up). It had passed 8/8 earlier the same day, which is the tell — the exclusion table is environmental and moves. The helper now takes the socket kind and probes **both** the public port and its tunnel port for it. That is the second time a Windows/Linux ephemeral-range difference has broken this one file. The `:proxy` screen ships **with** its endpoints — build #13 part 1 shipped four `/cockpit/exposure` endpoints with no caller and closing that took a whole later build. `tsc --noEmit` and `next build` both exit 0, and the screen adds zero lint errors.

**Not built, deliberately:** browser interception (it needs a published port, which breaks the lab sandbox's isolation — its own exposure decision), and driving ZAP's scanner through the API (scanning stays on part 1's gated command path). *Part 3 built the second of those; the first is still blocked, for a reason measured immediately afterwards — see below.*

## Build #14 part 3 — the scanner learns to aim (2026-08-04)

Part 1 could scan a URL; part 2 could record traffic. Neither could attack **what you actually touched**. `zaproxy -cmd -quickurl` spiders a site and attacks whatever the crawl found, so an endpoint reached only by *using* the app — an API route nothing links to, a page behind a login, the exact request a `ffuf` run just made — was out of reach. Those are precisely what the proxy already records. This part joins the two halves and drives ZAP's active scanner over part 2's `docker exec` transport, aimed at the captured Sites tree.

The measurement that justifies the feature: **one captured endpoint, 376 real attack requests, one live High SQL injection** — on a route `-quickurl` 's spider had no reliable path to.

### The finding that came first, and still blocks the other half

Before designing anything, `-config api.key=SECRETKEY123` was tested against the running daemon. It **enforces nothing**: `spider/action/scan` and `ascan/action/scan` both launched with no key at all. ZAP's proxy and its API share one listener, so publishing that port for **browser interception** would also publish an unauthenticated scan trigger to the host — and HackPit has no route auth. Browser interception therefore stays blocked, now for a measured reason rather than a suspected one. Scanner-over-API is unaffected: nothing is published, and the transport stays `docker exec`.

> ⚠ **THIS FINDING WAS WRONG, AND IT IS LEFT HERE UNEDITED ON PURPOSE.** Re-measured on 2026-08-04 with the flag stated explicitly, `api.key` **does** enforce — on views *and* on actions. The original test inherited `api.disablekey=true` from `$HOME/.ZAP/config.xml`, which **HackPit itself had written** on an earlier proxy start. It is left in place because a corrected record that hides the mistake also hides the lesson, which generalises: *a daemon that persists its configuration makes every measurement conditional on what a previous run wrote.* See **build #15** below, which this unblocked.

### The gate: an honest surface, not a new one

A scan start builds `zaproxy -quickurl <target>` and runs it through the **real** `executor.validate_request` before ZAP is contacted. No new gate exists. `-quickurl` is already defined in the attack-flag table as *spider then active scan*, and an API scan is that attack minus the spider — so the declared command describes strictly **more** aggression than what runs, which is the safe direction. Putting the full target URL in the surface is what matters:

measured against the real validator, an unapproved scan is refused at `approval`, one without the red-confirm at `danger`, and `http://example.com/x` at `target` — the existing scope extractor reads the host out of a URL carrying a port and a query string.

### Part 2's central lock could not be restated, so it was replaced

Part 2 asserted *the gated argv is the spawned argv*. That is meaningless here: the gate classifies an **argv** and what executes is a **URL**, so string equality between them would be theatre. The property underneath it is what actually matters — *the thing the gate scoped is the thing that gets attacked* — and that is now the lock. One derivation feeds both sides, and a test decodes the API's `url=` parameter back out and asserts its host is the host the target gate read.

**A defect class that did not exist before this build:** the operator-supplied target is interpolated into a URL that carries the scan's own parameters, so a target containing `&recurse=true` would broaden the scan the human approved. That is Critical 2 expressed in a query string. The target is percent-encoded with `quote(safe="")`, and the test carries a control proving recursion can still be set legitimately — otherwise it would also pass on a build where recursion never worked.

### A second bound, enforced by ZAP rather than by us

`ascan/action/scan` on a URL not already in the Sites tree answers `{"code":"url_not_found"}`. The active scanner's reach is therefore bounded by what already passed through the proxy: a host never captured cannot be attacked through this path even if every gate here were bypassed. It is a bound, not a control — the gates remain the control — but it means the worst case of a defect here is "it attacked something you already proxied". The proof asserts it live.

### The shape trap, caught by measuring instead of assuming

`state/parsers.py::parse_zap` reads the `-quickurl` **report**: nested `site[].alerts[]`, severity in `riskcode` (`"0"`–`"3"`), plugin in `pluginid`, URL down in `instances[].uri`. The API returns a **flat** list with `risk: "High"`, `pluginId`, and `url` on the alert itself. `_zap_report()` requires a `site` key, so feeding it an API response yields **zero findings, silently, forever, with a green suite** — part 1's headline defect in a new place. A separate mapper handles the API shape, tested against a real captured response committed as a fixture, and a test asserts the two parsers are **not** interchangeable, with a control proving the report parser does work on a real report. Someone will eventually try to merge them; that test is the argument they have to answer.

The plugin reference is written in `parse_zap`'s exact `pluginid:NNNNN` spelling, so the same issue found by both paths fingerprints to one finding rather than two — asserted by comparing real fingerprints, not by eyeballing the format.

### What the proof found that no test could

ZAP locks its **home directory**, not its port. A daemon left running on any other port kills a new one at startup with a message that appears only in a log file inside the container. The proof hit this on its first run, and it exposed a latent part-2 defect: the "one proxy at a time" refusal was scoped per *port*, so a second proxy on a different port in the same container was accepted and then died. Nothing was unsafe — status is observed, so the dead proxy reported itself down rather than lying — but it was **unexplainable**, which is its own kind of defect. The refusal is now container-scoped and states ZAP's reason.

### Verification

**CI caught a defect in my own tests, and it is worth recording.** The first version of the new locks drove the real `start_proxy` and asserted a lab scan validates cleanly. Both passed locally and failed on the runner: there is no Docker in CI, so the isolation gate refuses first and the verdict is `sandbox`, not the gate under test. The tests were silently depending on the developer's stack being up — the opposite of hermetic. The fix was structural rather than a loosened assertion: the clash check became a pure function that needs no Docker, and every control is now phrased as "*not refused at THIS gate*" rather than "*not refused*", which is how part 2's locks were already written. Re-verified by hiding `docker` from `PATH` and confirming the simulation actually bites (the lab scan comes back `sandbox`) before trusting that both files still pass under it — a simulation nobody checks is just a second thing to believe.

Suite **71 files, 0 failures**, up from 68. `zap_scan_proof.sh` **8 passed, 0 failed** — including host-unreachability re-asserted (it matters more now that action URLs sit behind that boundary), ZAP's `url_not_found` refusal, a full scan to completion, and the live alert response mapping through the real mapper. `zap_proxy_proof.sh` still 7/0. `tsc --noEmit` exits 0 and the `:proxy` scan panel adds zero lint errors. The panel ships **with** its endpoints, and its "Aim scanner" button on a captured row sets the target without starting anything — a one-click path from a table row to live attack traffic is exactly the shape a red-confirm exists to prevent.

### Closing the two gaps that were left open — and what the second one found

Two things were shipped unverified and named as such rather than left to read as done. Both are now closed.

**The ingest route had never been executed.** The mapping into `Finding` / `Endpoint` was tested, but nothing had driven `POST /cockpit/proxy/alerts/ingest` through to `upsert_findings` — unverified *wiring*, the same shape as build #13 part 1's four `/cockpit/exposure` endpoints that shipped with no caller. It is now suite file **71**, and it is hermetic in a way the house style is not: `store.DB_PATH` points at a temporary file, so nothing touches the operator's `sessions.db`. A test whose whole subject is "*did the write land?*" cannot also be the test that trusts a delete to have removed only its own rows. It patches `proxy._api_get` rather than `scan_alerts`, one layer lower, so the real captured JSON goes through ZAP-response parsing, the route, the mapper *and* SQLite. It asserts the rows read back, that re-ingesting does not duplicate (the panel has a button, so it will be pressed twice), that findings belong to one session, that a missing `session_id` is a 422 rather than a cheerful `{"findings": 0}`, and — by AST — that the route executes nothing.

**The scan panel had never been rendered, and it turned out the entire `:proxy` screen was invisible.** `.hp-card` starts at `opacity: 0` and only becomes visible via an `.hp-in` class or the `.hp-surface` keyframe; `ProxyScreen` used bare `hp-card`, and it was the only component in the codebase to do so. Worse, **six of the eight classes it used did not exist at all** — `hp-kv`, `hp-check`, `hp-table`, `hp-danger`, `hp-url`, `hp-error`. Part 2 wrote the screen against an invented vocabulary, and `tsc`, `next build` and ESLint all passed, because **none of them can see whether a class exists in CSS**. Part 2's own verification note claimed the typecheck and build pass — which was true, and did not mean the screen worked. The screen is now written in the real `hp-tn-*` vocabulary that `:exposure` and `:c2` use, and it renders: status rail live from the backend, the gate visibly refusing (the start button stays disabled until *both* confirms are ticked), and the scan panel's red-confirm block reading as the most dangerous thing on the page. One further browser-only defect: checkbox rows placed in `.hp-tn-form` inherit `flex: 1 1 200px` and render as 200px grey slabs, so they get their own `.hp-tn-check` rule.

**What is still unverified, stated rather than glossed:** a full click-through could not be completed under browser automation — the enabled start button did not issue its `POST` after several attempts, and I stopped rather than keep retrying. The source wiring is the codebase's standard `onClick` / `disabled` pattern and every endpoint behind it is covered by the suite and the live proof, so this is most likely an automation artefact; but it was not proven either way, and "probably the harness" is not a verification.

**And the screen had no way in.** Zaid asked whether anything recent was missing from the home launcher. It was: `/proxy` was the **only** top-level route in the app with no tile — not in `SURFACE_BANDS`, not in the command palette, with nothing anywhere in the UI linking to it. Both part 2 and part 3 shipped it reachable only by typing the URL. Part 3's own definition of done said "*:proxy ships with its endpoints — no orphaned routes (the build #13 part 1 lesson)*", and I had verified that every **endpoint** had a frontend caller — but never that the **screen** had an entry point. The same lesson, one level up, walked straight past. The tile now sits in **operate**, next to `:repeater`. That placement is an argument, not a preference: the band `hint` is a posture claim about everything in the band, and infrastructure's reads "*gated start · human-only stdin*" — the proxy's start is gated, but it has no stdin at all (spawned `interactive=False`, deliberately), so half that claim would be false for this tile. Operate's "*every command human-approved · needs the stack*" is exactly true of both halves. Sitting beside `:repeater` also makes the capture → replay path discoverable, which is why the captured-exchange model mirrors the repeater's field names in the first place.

**A process lesson worth more than the bug it caused.** The CI-simulation from the previous fix — running the suite with `docker` hidden from `PATH` — also hid `git`, and `test_corpora`'s re-ingest uses a git-recovery path. It rewrote the live KB and dropped one entry (2743 → 2742). The KB is gitignored, so there was no commit to fall back on; re-running the ingest with a normal `PATH` restored it exactly, as the recorded flake note predicted. The lesson: **a simulation that strips the environment can have side effects on real data.** Simulate by running the specific test files, not the whole suite.

**Still not built:** browser interception, blocked on the unauthenticated-API finding above; spidering via the API (that is `-quickurl`'s job and carries its confirm); scan-policy tuning (a policy that decides its own aggression is the `-autorun` shape part 1 excluded); and authenticated scanning.

## Full-project audit — the whole surface, driven end to end (2026-08-04)

An unattended overnight audit of **every** surface, not just the newest build, against the brief in `docs/superpowers/prompts/2026-08-04-full-project-audit.md`. The full report with per-question verdicts, evidence and an explicit NOT-RUN list is `docs/AUDIT-2026-08-04.md`; this section records what it changed and the three things worth carrying forward.

### The finding that mattered most was a combination, not a bug

`state/render.py` sets `INCLUDE_CREDENTIAL_SECRETS = True` — Zaid's explicit decision, and the module says so in full, including the consequence: "*every captured credential is sent to whatever LLM endpoint is configured, including a remote one.*" That decision is safe under a precondition it states but does not enforce — a **local** endpoint. The precondition was not holding. The live provider was `claude-agent-sdk` (remote, `opus`), while six real credentials from the build #9 live fire sit in `state_credentials`. Neither half is a defect; the pair was. The provider is now `ollama` / `qwen3:8b` per the standing rule, verified end to end (a grounded attack path composed in 147s, `context_leaks: 0`, every step citing a real KB entry id).

**The lesson: a documented decision can carry an unenforced precondition, and the audit that finds it will be looking at configuration, not code.** Both halves passed every test.

**Reviewed and deliberately left unchanged (2026-08-04, Zaid).** Five enforcement designs were put to him — derive inclusion from the provider, pass it from the caller, guard the config write, scrub at the egress boundary, or scope it to lab-vs-engagement — and he declined all five. His reasoning holds for the case he is in: the credentials in the store are from his own `corp.local` lab VM, a box built to be popped, so sending them to a model costs nothing. Recording the decision was the point rather than winning the argument, so the acceptance and its *boundary* now sit in `state/render.py` beside the constant: this path is not scoped to the lab, so the same render runs during a real engagement, where a client's domain-admin hash reaches the same endpoint by the same route — and there it stops being a matter of taste, because most pentest contracts forbid sending client data to third parties. The risk accepted is precisely "*my own lab credentials reach my own model endpoint*", and Option E is the smallest change that would close the rest if HackPit is ever pointed at a client under contract.

That is the right shape for a disagreement to end in. The behaviour is his call and it is unchanged; what changed is that the next reader finds reasoning instead of a landmine.

### Every gate fires, and every gate can fail — 27 checks, both directions

The 2026-07-27 audit found seven guards that never fired. A harness drove the real `executor.validate_request` with a refusal **and** a positive control for each gate, in lab and engagement mode: 24 checks, 0 failures, plus 3 on the isolation guard. The isolation refusal was proven with a genuinely non-isolated container rather than a mock — pointing the lab guard at `hackpit-engage-sandbox` (fully open by design) produced the egress-path refusal.

**A trap the harness caught in itself.** Its first danger-gate probe was `bash -c '…10.0.0.1…'`, which is refused at **`target`**, not `danger` — so the probe never reached the gate under test and the paired "with ack" control passed *vacuously*. In lab mode the danger gate is only reachable by a command naming the lab target as a clean argv token. This is the exact failure the house "not refused at THIS gate" phrasing exists to prevent, and it is the second time that convention has earned its keep.

### The missing gate is the mirror of the never-fired guard

Measured against the real bug-bounty program: `ffuf -u https://…/FUZZ -w common.txt`, `sqlmap -u https://… --batch` and `nuclei -u https://… -t cves/` all pass on `approved=true` **alone** against a third-party production host — no red-confirm. That is not a defect in the danger heuristic, which correctly models arbitrary code execution (it does flag `sqlmap --os-shell`). **Nothing models aggression toward the target**, and there is no rate limiting, pacing or concurrency control anywhere in `cockpit/`, `state/`, `reasoning/` or `arsenal/`. Recorded as the highest-value missing feature; the narrowest fix reuses the existing red-confirm in engagement mode only.

### Two hypotheses in the brief were wrong, in the product's favour

Bug-bounty reporting **exists**: a dedicated `bugbounty` template ("a HackerOne / Bugcrowd submission"), selectable in the UI, with a CVSS 3.1 base calculator verified correct against six known vectors including the round-up edges. Missing is narrower than assumed — no VRT, no P1–P4, 3.1 not 4.0. And the scope model **already** handles wildcards, CIDRs and `!exclusions`, and refuses both classic wildcard bypasses (`notexample.com`, `example.com.evil.net`). All 11 of the program's web/API targets parse, resolve and enforce.

What it cannot express is **mobile** — and the failure shape matters more than the refusal: `parse_scope("THAT Concept Store iOS")` splits on whitespace into four bogus hostnames and fails only because none of them resolve. Had one resolved, a garbage host would have entered the scope. A fail-open shape inside a fail-closed result.

**Reviewed with Zaid and ACCEPTED — no change (2026-08-04).** Following the audit, the finding was measured further and turned out not to be about mobile at all: the trigger is **whitespace**, and the worst case is not a store URL but a scope block that names its own out-of-scope hosts in prose — measured, `"out of scope: iana.org"` puts `iana.org` in the **allowed** set. Two fixes were designed and both declined: a strict tokeniser (comma/semicolon/newline separators, with anything not host-shaped recorded as "not understood"; zero of the

21 live engagements affected), and a dedicated out-of-scope box wired to the `!host` exclusion machinery — which is viable and already enforced end to end, **provided it marks every token rather than every line**, since per-line marking leaks even on a clean comma list. That per-token trick is recorded in the audit report in case the box is built later; the asymmetry behind it is that over-*inclusion* is dangerous while over-*exclusion* is free.

The acceptance is defensible for a reason the audit under-weighted, and it is worth stating because it corrects the framing rather than excusing it: **`check_target_lock` describes itself as "cheap defense-in-depth... NOT a load-bearing control", and says that in engagement mode "HUMAN APPROVAL of every command is the actual bound and this lock is an aid to the human, not a guarantee."** An over-wide scope runs nothing on its own; every command is still approved individually with the hostname visible in the argv. The audit treated the scope as the wall; the codebase treats it as a handrail. Zaid's stated workflow — a clean comma-separated list — parses perfectly, as do all 21 existing engagements. The residual, accepted knowingly: pasting a scope straight off a program page can still admit an out-of-scope host, caught only by reading the hostname before approving.

### Defects fixed in this commit

**A tool-file id dropped two files out of the product.** `pipeline/ingest_corpora.py` built the id from `path.stem`, so files differing only by extension collided: `nc` / `nc.exe` (linux vs windows) and `SharpHound.ps1` / `SharpHound.exe`. The extension is what decides `platform`, so these are genuinely different artefacts. `/entry/{id}` could only resolve one of each pair, and because the Scripts Arsenal keys its React list on that id, **React dropped one of each pair from the list** — two duplicate-key errors, confirmed in the browser. Fixed at source ( `path.name` ), artefacts regenerated, locked by two tests: one on the derivation carrying a control that proves the *old* derivation collided, one on the built artefact. The KB was verified byte-identical at 2743 lines before and after every regeneration.

**The Scripts Arsenal claimed more than it contained.** The section header printed `count` while the list held `shown` — 1235 claimed, 1158 served; `Enumeration` claimed 335 and rendered 263. The cap is deliberate and the pipeline records both numbers honestly; the UI simply never used `shown`, which was already carried through the backend model and the API type. It now shows what is listed and states how many were dropped, by which cap, and that the filter cannot reach them. The index was also **stale by 126 entries** (built against 2617); rebuilt.

**Checkboxes were grey slabs on six screens.** `.hp-tn-form input { flex: 1 1 200px }` over-matches — a checkbox is not a text field. Build #14 part 3 worked around this on `:proxy` with a per-screen `.hp-tn-check`; `:exposure`, `:repeater`, `:oob`, `:cockpit` and the AD screens still had it. **Fixed in the over-matching rule itself**, where every screen gets it, rather than in the one component that noticed — the same "fix the predicate, never narrow the file set" discipline build #5 established. Verified in the browser before and after.

### Verification

Suite **71 files, 0 failures**. All five live proofs green: isolation 4/0, engage-open 4/0 (Wall A confirmed down), zap-install 9/0, zap-proxy 7/0, zap-scan 8/0. `tsc --noEmit` clean; lint unchanged from its accepted baseline (no changed file appears in the output). All 18 surfaces rendered in a real browser and the orphan-route check is clean. Windows/AD driven **live** against the real DC — WinRM reached `corp\administrator`, and read-only domain enumeration ran through the gated executor while DCSync-shaped and `-EncodedCommand` payloads were correctly held at the red-confirm.

**Not run, and not to be read as passing:** the passive HTTP fingerprint sweep of the 11 third-party hosts (refused by the classifier — left as a self-verifying `docs/audit-2026-08-04/bb-passive-recon.sh`), a re-measurement of ZAP's `api.key`, and the end-to-end report-redaction observation (refused, correctly, because it meant dumping six real domain credentials — the property is verified by its existing test instead). Exactly **one** packet-generating command touched a program asset all night: a single `dig`, which returned `crateandbarrel.edgekey.net`. Nothing was submitted anywhere.

---

## Build #15 — browser interception (2026-08-04)

Against WAF/bot-managed targets HackPit did not reach rate limiting. **It did not reach request one.** A passive sweep of a live Bugcrowd program — one `HEAD` per host, in scope — returned nothing at all from every host:

| protocol | result |
| --- | --- |
| HTTP/2 | instant stream reset, `INTERNAL_ERROR` |
| HTTP/1.1 | total timeout, 0 bytes in 15s |

Two *different* failure modes on two protocols is an edge actively refusing the client, not a protocol quirk. Nine of the eleven assets sit behind Akamai Bot Manager. Egress was fine — HackPit reached Akamai and Akamai said no. So the audit's "can it run a full bug bounty?" answer of "*partly — breaks at volume*" was generous: volume is a problem you would like to have.

A real browser is what passes: correct TLS fingerprint, real headers, JS execution, a real profile. **That is the whole of this build**, and the boundary is stated because it is the interesting part: nothing here imitates a browser or evades a control. It uses one.

### The measurement that unblocked it was a correction, not a discovery

Part 3 recorded, as MEASURED, that ZAP's `api.key` "*enforces nothing*" — and that finding blocked browser interception for a day. It is wrong. Re-measured against the same ZAP 2.17.0 with the flag stated explicitly:

| check | result |
| --- | --- |
| `core/view/version` **without** key | refused |
| same **with** key | `{"version":"2.17.0"}` |
| **`ascan/action/stop` without key** | **refused** |
| wrong key | refused |
| **the PROXY meanwhile** | `HTTP 200` — serves normally |

`cockpit/proxy.py::server_argv_for` had been passing `-config api.disablekey=true` on *every* proxy start, and ZAP **persists `-config` values into `$HOME/.ZAP/config.xml`**. The original test set a key with no explicit `disablekey`, inherited `true` from a previous HackPit run, and concluded the tool was broken. **HackPit was disabling its own lock and we blamed ZAP.**

Generalised, beside "the container is not the image": **a daemon that persists its configuration makes every measurement conditional on what a previous run wrote.** State the flag explicitly, or you are measuring history. Both the wrong finding and its correction are kept in this document, because a corrected record that hides the mistake hides the lesson too.

### Part 1 — the port is published through `exposure.py`, not hardcoded

The single most important decision in part 1, and it replaced an earlier draft that baked `127.0.0.1:8090:8090` into `docker-compose.yml`. HackPit already has a designed, tested answer for "publish a port on the engage sandbox", built across builds #10 and #13, and this uses it: the ZAP port is `extra=[(8090, "tcp")]` on a `ListenerProfile`. **No new model, no new field, no new file format, no second publish path** — the generated override is gitignored (it can name a client's internal address and this repo is public), and a wildcard bind is a red-confirm whose acknowledgement is written into the file as a machine-readable marker that `test_exposure_safety.py` statically enforces.

That is *freer* than hardcoding loopback, which is the point. The operator can bind anything, including every interface for a phone or a second machine, with one confirm — recorded against the engagement so a report can state honestly that the proxy was open on all interfaces during that window. A hardcoded `127.0.0.1` would have been a constant to edit compose to escape; this is a switch they hold. Two presets ship: `zap-proxy` (loopback) and `zap-proxy-lan` (`0.0.0.0`). **Neither pre-acknowledges its own confirm** — a preset that did would mean picking a dropdown entry silently satisfies the gate, which is gate-audit finding I2's exact shape.

### What the design spec did not state, and the implementation had to answer

**A container process bound to loopback cannot be reached through a published port at all.** `docker -p` forwards to the container's bridge interface, where nothing would be listening. A published proxy that stayed loopback-bound would have been the worst of both worlds: a port open on the host and a feature that silently did not work. So `-host` became conditional (`bind_host_for`), and a published daemon also needs `api.addrs` widened — a request arriving through a published port reaches ZAP from the bridge gateway, not from `127.0.0.1`, and ZAP's default address filter would refuse it while looking exactly like a broken feature.

Both are real widenings. What pays for them is the key: what becomes reachable is an **HTTP proxy**, which is the entire point, while scan control still refuses everyone who cannot present a secret only the backend holds. Publishing is **engagement-only**, refused before the executor gates — the lab network is `internal: true`, so a published port there has no route, and binding wide would expose the API *inside the isolated network* while still being unreachable from here.

### The key is closed twice, and one of the two cannot regress

`-config api.key=<secret>` puts a credential in the spawned argv, and this codebase records argv into run records that feed the state store, the LLM prompt and rendered reports. Two layers:

1. **The gate is never given the key.** `_gate_request` passes a placeholder. An `ExecRequest` is the thing that gets recorded, reported and put in front of the model — redacting a secret after handing it over depends on the redactor being right forever; never handing it over cannot regress.
2. `secretargs` **masks it anyway**, and this needed a new *shape*: `-config` is the wrong discriminator, because the same flag carries `api.disablekey=false` — the single most audit-relevant token in the argv, and **the evidence the lock was on**. A rule keyed on the flag would redact both, which is `nmap -p 445` becoming `nmap -p <redacted>` one level down. So the map keys on the *setting name*, and a control proves an unregistered tool keeps its value.

The residual is written down rather than hidden: `/proc/<pid>/cmdline` inside a single-tenant container we own. ZAP reads `api.key` from `-config` or its persisted config and nothing else — there is no stdin path, so build #13's "ride the secret in on stdin" is unavailable here.

## Part 2 — the AJAX spider, and the red-confirm that could not fire

The spec said the crawl's equivalent surface is `zaproxy -zapit <target>` and that it must require `dangerous_ack`. **Those two cannot both be true.** `-zapit` is deliberately absent from `_TOOL_ATTACK_FLAGS`, and `test_zap_safety.py` explicitly locks that a `-zapit` recon run must *not* demand a red-confirm. As written, the spec's own test could never pass.

Fixing it exposed a defect that was already shipping. The map was `tool -> frozenset(flags)` and the consumer appended **one hardcoded sentence** for whatever matched:

| flag | reason the operator was shown | true? |
|---|---|---|
| `-quickurl` | "active web scan — sends live injection payloads..." | yes |
| `-daemon` | "active web scan — sends live injection payloads..." | **no** |

**Starting the recording proxy told the operator it was sending injection payloads at a target it never touches.** A red-confirm whose stated reason is false is worse than no reason: it is what teaches an operator that the text is noise and the checkbox is a formality.

The fix was to change the one map to `tool -> {flag: reason}` — *not* to add a second `_TOOL_BROWSER_FLAGS` beside it (Zaid, 2026-08-04). Two lists of the same kind of fact have to agree forever: that is build #5's "fix the predicate, never add a parallel one", and the same drift this very file removed once when `dangerous_script_heuristic` was made to DERIVE from this dict instead of restating it. **That the reshape fixes an existing defect as a side effect is the tell that it is the right shape.**

**The half that nearly slipped through:** that derivation *kept working without an edit*, because `sorted(a_dict)` yields its keys. It would not have failed loudly — it would have gone on stamping the old false sentence onto every flag, re-introducing the defect on the WinRM path only. `.items()` makes the reason travel with the flag, and a test now asserts the two heuristics agree on WHY, not just on WHAT.

So the crawl earns its confirm for a third, genuinely different reason: **it drives a real browser that clicks things.** On the production e-commerce sites in scope for a bug bounty, clicking everything reachable can submit a form, empty a basket, trigger an email or place an order — a different hazard from SQLi and arguably a more embarrassing one. `-ajaxspider` is a **declared marker**, not a real ZAP flag (the crawl is API-driven and has no command line), and `_ATTACK_FLAG_IS_REAL` records which tokens are which so a third marker cannot arrive unnoticed. `-zapit` is dropped from the surface entirely: once the marker carries the danger verdict and the URL carries the scope, including it would make the declared command claim two modes at once.

**Depth and duration are in the approved surface**, for the same reason `-autorun` was excluded from the catalog in part 1: a crawler that decides its own bounds is a command that has stopped describing what runs.

### Why ZAP's spider and not our own headless browser

The reason exists only because part 1 landed first. Manual browsing through the published proxy establishes an **authenticated session inside ZAP**, and the AJAX spider runs through that same ZAP — so it inherits those cookies and crawls the logged-in application. A separate headless Chromium would start cold and need scripted authentication per target, an auth-automation problem this build would then own forever. Log in once by hand, let the spider expand from there, let part 3's scanner attack what both produced.

### An OK is not a result

`setOptionBrowserId` **does not validate**: it accepted `not-a-browser` and answered `{"Result":"OK"}`. The image ships Chromium and **no Firefox**, while ZAP's configured default is `firefox-headless` — so the failure surfaces at *crawl* time, in a driver stack trace inside ZAP's log, not at set time. `start_spider` therefore reads the value back and refuses a mismatch, `observed_spider` reports what ZAP holds rather than what we sent (asserted **by AST**, because a substring scan failed on the function's own docstring — build #8's lesson, firing on the first run), and the proof asserts a browser *process* appeared and the message count rose.

### The isolation argument was rebuilt, not relaxed

Part 2's argument was "*no port is published, so the API is unreachable, so the only channel is* `docker exec`." **Half of that is now deliberately false.** The property being proven changes from "*nothing can reach the control channel*" to "*the control channel refuses everyone who does*" — and the second needs **more** assertions, not fewer. `zap_proxy_proof.sh`'s load-bearing lab check is untouched and still runs; `browser_intercept_proof.sh` adds the engage half: reachable, **refuses without the key** (view *and* action), **answers with it**, **still serves as a proxy**, the lab still publishes nothing, and a control proves the refusal is enforcement rather than a dead port.

**Residual risk, accepted and written down:** anything that can reach the bound address can *use* the proxy — it cannot scan, which needs the key. On loopback that is a privacy annoyance on a single-user machine. On a wildcard bind it is an open proxy with the engage sandbox's full egress behind it, which is why that case carries a confirm rather than being forbidden or being free.

### Two defects the proof found that nothing else could — and the second was hidden by the first

The hermetic suite was green, `tsc` was clean, the image built, the API key enforced, the port published, and the AJAX spider answered `{"Result":"OK"}` and **crawled nothing**. Twice over, for two unrelated reasons that produce the identical symptom:

**1. Chromium refuses to run as root without `--no-sandbox`.** The engage sandbox runs `user: root` by design (build #3's privilege decision), so every browser Crawljax launched died at creation — the log simply stops after `Loaded ... DummyPlugin as a OnBrowserCreatedPlugin`, with no error on the ZAP side at all. ZAP's Selenium add-on exposes **no API** for adding Chrome arguments in this version (`addChromeArgument` answers `bad_action`), so the flag goes in through `/etc/chromium.d/` — Debian's own launcher extension point, which already ships a `dev-shm` file.

`--no-sandbox` turns off Chromium's internal process sandbox, and this browser visits target-controlled content, so it is written down rather than glossed: the boundary that holds here is the container, not Chromium's own, and as root the alternative is a browser that does not start at all. `--disable-dev-shm-usage` rides along for Docker's 64MB `/dev/shm`.

**2. ZAP bundles its own chromedriver and prefers it.** With the first defect fixed, the crawl failed differently: `This version of ChromeDriver only supports Chrome version 151`. The `webdriverlinux` add-on ships a driver for Chrome 151; Kali ships Chromium 150. So installing `chromium-driver` was **necessary and not sufficient**, and every daemon start now passes `-config selenium.chromeDriver=/usr/bin/chromedriver`.

**The lesson is about the check, not the flags.** The Dockerfile layer already followed part 1's discipline — it proved the driver was on PATH, proved it *ran*, and proved a browser existed. All three passed while the feature was completely broken, because **proving a driver runs proves nothing about whether it can drive the browser that is there**. The layer now compares the two major versions and fails the build on a mismatch. That is "an exit code is not a result" one level deeper than part 1 phrased it: a *successful* version string is not a result either.

Both were then given their own proof checks, because they fail separately and because a future image whose packages drift apart would otherwise reappear as an unexplained empty crawl.

**A third, smaller one, in the proof itself.** The "a real browser process was observed" check passed "*after 0s*" on a run where the crawl found nothing — it had matched a leftover Chrome from the previous attempt. A check that reports success from another run's residue is worse than no check, so strays are now reaped before the crawl starts (and deliberately *before*, since doing it after would kill the very process the evidence depends on).

### Two more defects, this time in the proof itself — and one is a lesson arriving backwards

**`pkill -f` matched too much, because a fix earlier in this build changed an unrelated argv.** The proof reaps stray browsers before crawling, and did it with `pkill -f "[c]hrome"`. That killed **the ZAP daemon**: its command line now carries `-config selenium.chromeDriver=/usr/bin/chromedriver`, so it contains the string "chrome". Every API call after that point returned empty and three checks failed for reasons that had nothing to do with what they test.

This is the `[z]aproxy` lesson arriving from the opposite direction. There, `-f` matched too LITTLE — the wrapper exec'd the JVM, so the spawned argv was not the running one. Here it matched too MUCH, and it began matching only because a fix elsewhere in the same build put a new word on a different process's command line. **A `pkill -f` pattern is a claim about every argv on the box, and it stops being true silently when an unrelated argv changes.** Now `pkill -x`, which matches the process NAME and cannot be broadened by somebody else's flags.

**A check whose own plumbing decided the verdict.** The "can chromium start as this user" check piped into `grep -c`, which prints `0` and *exits 1* when it finds nothing — so the `|| echo 1` fallback fired on the SUCCESS path, appended a second line, and the comparison could never match. It reported FAIL while the thing it tests was working perfectly. Read the output; do not infer it from an exit code. That is the same sentence as part 1's, applied to the harness rather than the tool.

Neither is a product defect, and both are worth writing down: a proof that fails for its own reasons trains you to discount it, which is exactly the failure mode a proof exists to prevent.

### CI caught a test depending on Docker — for the SECOND time, in the same file

Build #14 part 3 recorded exactly this: a new lock drove the real validator, asserted a lab run "validates cleanly", passed locally and failed on the runner, because there is no Docker in CI and the LAB gate order is `target -> approval -> danger -> **sandbox**`. The fix then was structural — every control phrased as "*not refused at this gate*" rather than "*not refused*".

The new crawl-gate test did it again: `assert clean is None`. Locally the stack was up, so the isolation gate passed and the assertion looked correct; on the runner it came back `gate='sandbox'`. **The green local run was measuring the developer's machine, not the code.**

Two things follow, and the second is the useful one. The assertion is now *if* refused it must be at `sandbox`, which excludes the three gates under test and nothing else. And the suite is now run with **`docker` removed from `PATH`** before being believed — with the strip itself checked first, because a simulation nobody checks is just a second thing to believe. That lesson already existed, was already written down, and still did not change what I actually ran until CI forced it. **A convention only holds if something executes it.**

### Verification

Suite **71 files, 0 failures** — and re-run with `docker` hidden from `PATH`, where it is also **71/0**, which is the environment CI actually provides. `docker/proof/browser_intercept_proof.sh` **22 passed, 0 failed**, including the four that carry the safety argument — an unauthenticated **view** refused, an unauthenticated **action** refused, a **wrong key** refused, and the same call **with** the key answering `{"version":"2.17.0"}` as the control proving the refusals are enforcement rather than a dead port. A request made **from the host** through the published proxy was captured (0 → 2), the browser-driven crawl captured **2 → 41 messages and found 29 URLs**, and the real captured traffic parsed through the real parser into exchanges and endpoints. The **lab** sandbox still publishes nothing, and teardown left nothing listening on either side of the boundary. `docker/proof/zap_proxy_proof.sh` **7 passed, 0 failed** with its load-bearing check — *the ZAP API is UNREACHABLE from this host* — still passing for the lab sandbox, and `isolation_proof.sh` **4 passed, 0 failed**. The half of the argument that did not move did not move.

`tsc --noEmit` exits 0, `next build` exits 0, and eslint sits at the accepted baseline of 11 — the same count with the frontend changes stashed, so this build adds none. `:proxy` was **looked at in a browser**, not merely typechecked: the publish control is disabled until an engagement id is entered, and the crawl panel's red-confirm carries its own copy rather than the scanner's. All three spider routes are registered and all three are called by the client — no orphans.

**The acceptance test was left open at commit time and HAS SINCE BEEN RUN.** The runbook is `docs/proof/build15-acceptance-runbook.md`; what it returned is below, and it split the two halves of this build apart.

### The acceptance test, run live (2026-08-04) — part 1 passes, part 2 does not

Run against `www.crateandbarrel.me`, in scope for the MAF Lifestyle Bugcrowd program and one of the nine Akamai-fronted assets that returned **nothing at all** to a bare `HEAD` — h2 stream reset, h1.1 total timeout.

**Part 1 PASSED.** Firefox on Windows, pointed at the published `127.0.0.1:8090` with ZAP's CA trusted, loaded the site normally. ZAP captured **55 requests** across the target's own hosts:

| host | requests |
|---|---|
| `www.crateandbarrel.me` | 44 |
| `dh.crateandbarrel.me` | 6 |
| `gtm-analytics.crateandbarrel.me` | 3 |
| `crateandbarrel.me` | 2 |

A real browser through this proxy reaches a host a bare client cannot. That is the whole premise of the build, and it holds. It also surfaced **two subdomains enumeration had not** — `dh.` and `gtm-analytics.` came out of a page load, not a wordlist.

**A stated concern was measured and was WRONG.** Before the test, the risk called out was that ZAP is a MITM proxy: it terminates the browser's TLS and re-originates upstream with its own Java stack, so Akamai would see Firefox's headers and JS but *not* Firefox's TLS fingerprint — and Akamai Bot Manager leans on TLS fingerprinting. That was a reasonable objection and the measurement overruled it. Java's handshake was accepted. Worth keeping, because it is the shape of a plausible argument that a five-minute test settles.

**Part 2 FAILED against the same target, and the control makes it unambiguous.** Three fetches through the identical proxy, minutes apart:

| client | target | result |
|---|---|---|
| Firefox | `www.crateandbarrel.me` | 55 requests captured |
| headless Chromium | `www.crateandbarrel.me` | **ZAP's own 20s read timeout, 0 bytes** |
| headless Chromium | `example.com` | HTML, normal |

Same ZAP, same proxy, same window. The refusal is specific to the CLIENT, and it reproduces the original `curl` signature exactly: a silent hang rather than a rejection.

**This undercuts part 2's central argument on precisely the targets that motivated the build.** The design was "log in by hand, let the AJAX spider expand from there, let part 3's scanner attack what both produced." The *session* inheritance works — that part is sound. But the spider brings its own browser and therefore its own fingerprint, and on a bot-managed edge that is what gets refused. So the honest scope of each half is now measured rather than assumed:

- **Part 1 (publish + manual browsing)** — what actually unblocks the WAF-fronted assets.
- **Part 2 (AJAX spider)** — scale on ordinary targets; refused by Akamai. Useful, and narrower than the spec claimed.

**Not pursued, deliberately.** Headless Chrome advertises `HeadlessChrome` in its User-Agent, which is the likely discriminator, and overriding it would probably restore the crawl. That crosses the line this build drew around itself — "*nothing here imitates or evades — it uses one.*" Going further is a separate decision and a separate build; this section is the evidence it would rest on, which is exactly what the spec asked for if a real browser were refused.

### Two smaller things the live run surfaced

**Chromium phoned Google during the proof's crawl.** A crawl aimed at a local two-page site also produced requests to `www.google.com`, `gstatic.com` and `play.google.com` — Chromium's background networking (variations seed, search preconnect) going out through the proxy, and the engage sandbox has full egress. Harmless here, wrong for a tool used on engagements: traffic nobody asked for, attributed to the operator's IP. The `/etc/chromium.d/` drop-in should carry `--disable-background-networking --no-first-run --no-default-browser-check`.

**An engagement's rules-of-engagement text is not enforced by anything.** The active engagement carried `authorization: "...PASSIVE RECON ONLY this session; no active scanning per rules of engagement."` — written for an unattended session. Scope is enforced in code; that sentence is free prose no gate reads, so a crawl that clicks through a production storefront would have passed every gate. It was caught by reading the record, not by a control. Same shape as the credential-precondition finding in the 2026-08-04 audit: **a documented constraint with nothing behind it.**

## The audit punch list, worked — build #16 (2026-08-04)

The 2026-08-04 audit ended with nine open items. Zaid reviewed each and decided which to build: D3, D5, D6, D7, D8 and D9 in, plus three new ones (known-issue matching, VRT priority, scan session-expiry detection) and the `:exposure` control layout. **D1, D2, D4, route auth, `INCLUDE_CREDENTIAL_SECRETS` and mobile support were reviewed and declined** — they are not open defects and are not listed as such below.

### D8 — the planner emitted commands nobody could run, and nothing said so

The measurement that opened this: with the correct request shape, `/attack-path` returned **32 commands, of which 0 named the real scope**, four carried a foreign example host, and an earlier run had returned `sublist3r -d tesla.com -t 100 / above the -t is for threads…` — the source writeup's example host *and* its prose, verbatim, inside the `cmd` field. Every step carried a `target_adaptation` line *describing* the substitution in words. Nothing performed it.

**The substitution machinery was not broken. It never ran.** `substitute_target` opens with `if not target: return cmd`, and `target` came only from `extract_target(goal)` — which reads the goal text. The failing call passed the real hosts in `scope_text` and a goal that named none, so `target` was `None` and every command kept the KB author's example host. One missing fallback made an entire feature a no-op, silently, for as long as it has existed.

Two changes, in the order the work was scoped:

**(c) Validate and flag first.** Every returned command is now checked against the declared program scope and, when it cannot run as written, marked `runnable: false` with the reason. Two reasons, kept apart because the fix differs: *points at a host outside the scope* (written for someone else's environment) and *names no host at all* (an unsubstituted placeholder, or a KB excerpt that was prose rather than a command). The path-level `commands_unrunnable` is the headline: **a plan built entirely from the KB's own examples used to be indistinguishable from one adapted to the target.**

**(a) Then substitute.** `primary_target` falls back to the first *concrete* in-scope host when the goal names none, and reports where the target came from (`caller` / `goal` / `scope`) so "HackPit picked this for you" reads differently from "you named it". A wildcard is deliberately not a target: `*.example.com` names nothing runnable, and inventing `www.` in front of it would be a fabricated target dressed as a real one.

**The scope model is reused, not re-derived.** `cockpit/scope.py` — wildcards, CIDR, exclusions, already measured against a real bug-bounty program — is what judges the commands, so the plan cannot disagree with the gate about what is in scope. The same reasoning produced one `_host_positions()` definition shared by the substituter and the checker: written twice they would drift, and a checker that disagreed with the substituter would flag commands it had itself just fixed. That is the shared-predicate defect this repo has now found four times.

**What was NOT done, and why.** A real foreign host like `tesla.com` is *flagged, never rewritten*. Rewriting any out-of-scope host sitting in a host position would have closed the gap completely — and would also have rewritten `curl https://raw.githubusercontent.com/…/linpeas.sh` into a request against the client's own storefront. A tool download, an attacker-owned listener and a target are all "a host in a host position"; nothing in the argv distinguishes them. So example hosts are substituted (they are recognisable), and everything else is reported. The scope directive said "*where it can be done safely*"; this is where that line falls.

**`/attack-path` accepts `target` now — and refuses what it does not know.** A `target` field used to be dropped in silence, which is how the first measurement looked worse than it was. It is a declared field, and `AttackPathIn` is `extra="forbid"`, so an unknown field is a 422 rather than a shrug.

**A silent loss found on the way in.** `AttackStep.commands` was typed `list[Code]` — the KB entry shape — while `attack_path.py` had been setting `unverified` and `truncated` on planned commands the whole time. A response model does not carry a field it has not declared, so both have been dropped on the floor for as long as they have existed. `PlannedCode` repairs that and holds the new scope fields. `test_attack_path_contract.py` locks the general property rather than a list of names: **every key the composer emits must survive `response_model`**, with a planted field proving the check can fail. A name list would need updating by exactly the person who just forgot.

### D3 — nothing throttled anything

Pacing injects each tool's own rate or delay flag, in the exact shape `executor.apply_proxy` already uses, **engagement mode only** — the lab target is an isolated container with nobody to annoy, and lab argv stays byte-identical. A tool with no known throttle flag runs unchanged and *says so*, because a run the operator believes was paced and was not is precisely how a program bans your IP mid-engagement.

Three things the shape needed that a proxy URL does not:

- **Units.** Throttling is spelled three incompatible ways — requests/sec, packets/sec, or a delay *between* requests. The operator supplies one number and it is converted per tool. Handing a literal `4` to sqlmap's `--delay` would mean a four-*second* wait when four requests/second was asked for; reversed, a delay fed to a rate flag runs 1000× too fast. The regression test pins all twelve conversions.
- **Already-set detection.** A command that already carries its own rate flag is left alone and says so. Two contradictory rate flags are resolved silently and differently by each tool.
- **Flag placement.** See below — this one was a live bug, not a hypothetical.

**Every flag was read out of the tool's own `--help` inside the real image**, not recalled. That check removed three candidates rather than guessing at them: `amass` is un-probeable in the sandbox (the sudo / `no-new-privileges` trap), `curl` issues one request so there is nothing to pace, and — see below — the image's `httpx` is not the tool anyone assumes it is. All three take the honest no-flag path.

### Two shipping defects the pacing work found in the proxy flag it was mirroring

**`gobuster --proxy … dir …` has never worked.** Measured against the real image, it exits with `flag provided but not defined: -proxy` and does nothing at all: gobuster's flags belong *after* its subcommand. Asking to capture a gobuster run has been breaking the run. Pacing would have reproduced the bug exactly, so the placement rule now lives in one `_place_flag()` used by both rewrites — the next tool with subcommands cannot be right in one and wrong in the other.

**`/usr/bin/httpx` in the sandbox is the *python* httpx CLI**, whose proxy flag is `--proxy`. ProjectDiscovery's httpx — the one `-http-proxy` belongs to — installs as `httpx-toolkit`. Both names were mapped to `-http-proxy`, so capture on `httpx` produced a command line that CLI rejects outright. Same defect class as "*Kali ships no `zap-baseline.py`*": **the map named a flag for a tool that is not there.**

### D5 — the safety suite rewrote the live KB on every run

`test_reingest_is_byte_identical` re-ran the real ingester against `data/kb/entries.jsonl` itself. The assertion held, but a green test that rewrites 15 MB of gitignored production data with no restore path is how an earlier build lost an entry. It now ingests into a **copy of the real KB** in a temp directory, and afterwards asserts production is byte-identical.

Copying the KB alone was not enough. **Everything under `/data/` is gitignored**, and `--kb` redirected only `entries.jsonl` — the sidecars, `toolfiles.json` and `corpora_report.json` kept writing into `data/kb` regardless. The ingester's outputs now follow the KB it is given, so "run this somewhere else" means it. The test asserts that too, by mtime: `corpora_report.json` is rewritten unconditionally, so before this fix its mtime moved on every single suite run.

It is still the **real** KB, copied — with a size floor asserting >1000 entries. Idempotence over 2,743 entries of real escapes, non-ASCII titles and foreign lines is the property worth proving; over a three-line fixture it is close to vacuous, and a fixture is what this test would otherwise decay into.

### D6 — nothing noticed a derived artefact going stale

The scripts index sat **126 entries behind** the KB (2,617 against 2,743) through an entire audit, and was found by reading a number. A stale index does not error — it answers, with 126 entries' worth of the corpus missing.

`test_kb_drift.py` compares the KB against the three artefacts built from it: the semantic index ( `ids.json` ), the scripts index, and the corpus report. Two details carry the weight:

- **A count is not coverage.** `ids.json` is checked for *set equality* of entry ids, not just the total — one entry added and another deleted between runs leaves the count untouched and the index wrong. The scripts index is additionally checked for citations that no longer resolve.
- **Absences are reported, never silently green.** `/data/` is gitignored in its entirety, so a fresh clone or a CI runner has none of these files, and there the test proves nothing. It says so out loud. A fourth check fails when `data/kb` grows a file nothing has classified, so leaving something unchecked has to be a decision rather than an oversight.

Each check has a positive control that runs **the real check** against a doctored artefact — including a 126-entry lag, the exact drift that went unnoticed.

### D7 — every successful AD command looked like a failing one

PowerShell's stderr over WinRM is not text; it is a CLIXML object stream, and every progress bar it would have drawn on a console arrives as an `<Obj S="progress">` record. Build #9's live run against a real `corp.local` DC returned `rc=0` alongside a wall of markup.

Progress records are now stripped where the `WinRMResult` is *built*, so the event stream and the persisted run record see the same cleaned text; doing it at a display layer would have left the record full of markup and put the burden on every future consumer.

The whole risk of this fix is that it strips something else, converting a cosmetic annoyance into a silent failure — a strictly worse bug. So the filter removes progress records and nothing else: Error, Warning, Verbose and Debug streams survive, `_xNNNN_` escapes are decoded, and **anything that fails to parse is returned verbatim**. The raw stderr stays on the result and the dropped count is reported, so "quiet because it was filtered" stays distinguishable from "quiet because it was quiet". The test that matters is the negative one: a genuine `Get-ADUser` error survives a document carrying forty progress records.

### The bug-bounty submission fields — and one that nothing could set

**VRT priority (P1–P5) now sits alongside CVSS.** Bugcrowd triages on the Vulnerability Rating Taxonomy, and the two genuinely disagree: a stored XSS is P2 whatever its vector works out to, and a 9.8 on a class the program rates P3 gets paid as a P3. Reports carried CVSS only, so the number a triager acts on was missing.

**It is a LOOKUP, and that is the invariant with teeth.** Deriving a priority from the CVSS score would have been three lines and would have produced a confident P-number with no relationship to the taxonomy — a fabrication in the field a triager reads first. So the operator names the category and a curated subset of the VRT maps it; an unrecognised category claims *no* priority and says why. The regression test proves the priority does not move when the CVSS vector does. Where the two disagree, the report says so and suggests arguing it.

**The field nothing could set.** `build_cvss_block` has read `session['cvss_vector']` since the bug-bounty template shipped. There was no column, no endpoint and no control — so the CVSS block this project verified against six reference vectors, roundup edges included, **could never appear in a real report**. The calculator was right and unreachable. Adding the VRT field would have been dead in exactly the same way, so both are now stored, settable through `PUT /sessions/{id}/submission`, and covered by a round-trip test.

### The known-issue check — flag, never suppress

Programs publish what they already know about and will not pay for. Submitting one burns the report and, on some programs, costs signal. Nothing compared a finding against that list.

The whole feature is one design rule: **it flags and it never drops**. A false match that silently removed a real finding costs a paid bug and nobody ever learns it happened; a false flag costs one glance at the brief. The output is a table headed by what was compared, the matching is deliberately loose, no code path removes a finding, and the test asserts the finding list comes back unmutated.

It also reports **when it finds nothing** — "compared 7 findings, no matches" — because silence is indistinguishable from a check that never ran. The discrimination is real, not nominal: a stored XSS in the order form is not matched against a published *self*-XSS in the profile bio, which is the false positive that would have flagged a P2 as unpayable.

### Authenticated-scan session expiry — a silent wrong answer made loud

Build #15's AJAX spider crawls behind a login by inheriting a session the human established by hand. It added no ZAP context, no session management and no authentication handling. That is fine. **What happens when the session expires mid-scan is not.**

The active scanner does not stop. It keeps firing SQLi and XSS payloads at what are now login redirects, matches nothing, and finishes cleanly reporting **zero findings** — which is indistinguishable from "*the application is secure*". Not a crash and not an error: a confident wrong answer, of the kind nobody has a reason to doubt.

`session_health()` reads the recorded traffic and notices four shapes a dead session takes: a wall of redirects to a login path; 200s whose body is the login page; a wall of 401/403; and the one the first three miss — a **collapse to a single response shape**, which is what a login wall looks like when it renders a friendly page instead of redirecting. Below ten responses it returns `unknown` and says so; a false all-clear would re-create exactly the confidence this removes. A `suspect` verdict rides the alert-ingest response *and* lands at the top of the report, above the finding count it should be read with.

**This is the cheap honest version by decision.** It does not re-authenticate, maintain a session or build a ZAP context — those are a separate build. The AST-scanned test asserts it cannot: it reads the exchanges it was handed and calls nothing.

### The phantom class, and what the type-checker cannot see

`.hp-tn-start` **did not exist in `globals.css`**, while nine buttons across `:exposure`, `:oob` and `:windows` used it. Those are the PRIMARY actions — "write profile", "save remote profile", "deploy + start" — so they rendered plainer than the destructive buttons beside them, which at least use `.hp-tn-stop` and turn red on hover. **The visual hierarchy was exactly backwards**, and nothing could tell: tsc, `next build` and eslint have no idea whether a CSS class exists.

Defined rather than dropped — nine call sites already assert "this is the affirmative action", which is a distinction worth rendering. Three ranks now, so the eye sorts them before reading the labels: `.hp-tn-start` (accent, affirmative) · `.hp-tn-destroy` (red, destroys something) · `.hp-tn-stop` (muted, a reversible undo).

**And the row itself.** Inputs, the wildcard checkbox and all three buttons shared one `.hp-tn-form` flex row, where `input { flex: 1 1 200px }` stretched the fields and the buttons wrapped wherever they landed. There was no break between *configuring* a profile and *acting* on it — including the act that recreates the container and kills every listener, session and background job inside it. A shared `.hp-tn-actions` row now separates them, rules them off, and pushes the destructive button away from the affirmative one. It lives in the CSS layer: the last time a shared `.hp-tn-form` rule was patched per-screen, the fix shipped for `:proxy` alone and left checkboxes broken on six others.

**The guard this earns.** `test_css_vocabulary.py` asserts every `hp-*` class any component names exists in `globals.css`. This class of defect has now cost three builds — an entirely invisible `:proxy` for two of them, then this — and it is mechanical, so it belongs in the suite rather than in a reviewer's memory. **Its first run found seventeen**, and the first thing that had to be fixed was the checker: `hp-set-model` and `hp-set-key` are element **ids**, not classes, so the scan was narrowed to `className` attributes. Four more are harmless: modifiers on an element whose *first* class is defined and does the styling. The remaining **eleven are real, bare phantoms** in `:repeater`, `:exploits`, `:category` and three others. They are frozen as a baseline that may only ever SHRINK, kept deliberately apart from the allow-list: one list means "checked, fine", the other means "known, unexamined", and collapsing them is how a check like this quietly stops working. **It is still not a substitute for looking at the screen** — a class can exist and still be wrong.

### D9 — the two surfaces that read as a different product

`:evasion` was **raw Tailwind utilities** ( `rounded border border-slate-700 bg-slate-900/40` ) with no kicker/ `:title` header, so it looked like a different application. It is now the house `hp-tn-*` vocabulary throughout. Two classes of thing the vocabulary did not have — a label/value fact list and a preformatted block — were added *as vocabulary* rather than as one screen's private classes, which is the difference between a restyle and a reskin. The one piece of emphasis kept deliberately is `still_recorded`: it is the sentence that makes the surface a purple-team instrument rather than an evasion how-to, and it must not flatten into the facts around it.

`:ad-graph` was a different case, and worth saying plainly. It is **not** Tailwind — it has a purpose-built `hp-adg-*` vocabulary for nodes, edges and hop drawers, with no `hp-tn-*` equivalent. The real gap was the page: a bare `<main>` with its own back-link bar, outside `PageShell`, with no kicker/ `:title` block. That is fixed, along with the two empty states, which are ordinary cards now. **The graph canvas itself is untouched** — replacing those primitives would be a rewrite of something that works, and D9 is explicitly cosmetic.

Both are behaviour-identical: no gates, no endpoints, no handlers changed.

### The browser pass found two things nothing else could

Every gate was green — suite, docker-stripped suite, `tsc`, `next build`, lint baseline, and the new CSS-vocabulary check — before anyone looked at a screen. Then looking found two defects.

**All three buttons in the new action row rendered identically grey.** `.hp-tn-actions button` and `button.hp-tn-start` have the *same* specificity (0,1,1), and the base rule was written second, so it won. The classes existed, they were applied, the check that asserts they exist passed — and the row that was built specifically to make three ranks distinguishable showed three identical buttons. This is what "*a class can exist and still be wrong*" means in practice, and it is the reason that sentence is in the guard's own docstring rather than a claim that the guard is sufficient.

**`15 of 16commands won't run as written`**. JSX drops the space between an expression and the text following it when a line break falls between them. A missing space in the one banner whose entire job is to be read.

Neither is subtle once seen, and neither was visible to any tool. The rule stands: **a frontend change is not verified until it has been looked at.**

A third came out of the same pass, once the new action rows put it under a spotlight: `hp-tn-olhint` is a **label** style — 10px, 1px letter-spacing, ALL CAPS, bottom margin only — and ten places were using it for whole paragraphs. Three lines of prose rendered in it are close to unreadable, and with no top margin they sit crammed against whatever is above. Those are now `hp-tn-note`; the genuine labels (`expected: 8090/tcp`, `path`, `mode · exit · run`) keep the label style, which is the point of having two. One element on `:exposure` was doing both jobs at once — a port list and a paragraph sharing a tag, so the paragraph inherited the label's styling — and was split in two. Pre-existing on `:exposure`, `:oob` and `:windows`, and fixed there because this build is what put a ruled row directly above each one.

**Still outstanding, deliberately not changed:** the acknowledgement checkbox labels ("I understand this is a public listener that relays into this machine") are also 10px uppercase. That is text an operator is meant to actually read before ticking a safety control, so the styling is arguably wrong there too — but it sits inside the form flex row, where a size change moves layout, and it was not part of what this build touched.

## What was not done

- **No UI for the VRT / known-issues fields yet.** `PUT /sessions/{id}/submission` and `GET /vrt-categories` exist and are tested; the report screen has no control for them, so today they are set by API. Flagged rather than quietly skipped — the report-side work is what the punch list asked for, and it is complete.
- **`:ad-graph`'s graph primitives were left alone**, for the reason above.
- **A real foreign host is still not substituted** in a planned command, only flagged — see the D8 section for why rewriting one would point a tool download at the client.